

“Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

MONISHA RAGHINI CHELLE 2520030537

BHUPATHIRAJU DEVI CHARITHA 2520030462

PRASHANTHI DABBUGUNTA 2520030301

Under the Esteemed Guidance of

Dr Yerragudipadu Subbarayudu

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development”** is a record of Bonafide work of **MONISHA RAGHINI CHELLE -2520030537, BHUPATHIRAJU DEVI CHARITHA-2520030462, PRASHANTHI DABBUGUNTA - 2520030301** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

MONISHA RAGHINI CHELLE – 2520030537

BHUPATHIRAJU DEVI CHARITHA -2520030462

PRASHANTHI DABBUGUNTA - 2520030301

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certified that the mini project-based report entitled **“Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development”** is a bonafide work done and submitted by **MONISHA RAGHINI CHELLE -2520030537, BHUPATHIRAJU DEVI CHARITHA-2520030462, PRASHANTHI DABBUGUNTA - 2520030301** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Yerragudipadu Subbarayudu**, and my guide **Dr Yerragudipadu Subbarayudu**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

MONISHA RAGHINI CHELLE – 2520030537

BHUPATHIRAJU DEVI CHARITHA -2520030462

PRASHANTHI DABBUGUNTA - 2520030301

Abstract

Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development is a web-based application designed to simplify the process of booking and managing medical appointments. The system provides a centralized platform through which patients can access doctor information, check appointment availability, and manage their appointments online.

The application allows patients to register, log in securely, view doctors and their specializations, check available appointment slots, and book, reschedule, or cancel appointments. Patients can also view their appointment history and receive appointment-related notifications. Administrators can manage patient records, doctor profiles, doctor availability, appointment statuses, and other administrative operations through a dedicated dashboard. The system also includes a demonstration payment workflow with payment history and receipt information.

The frontend is developed using React.js, HTML, CSS, and JavaScript, with Vite as the development tool. The backend is implemented using Python and FastAPI to provide REST APIs, while MySQL is used for relational database management. JWT-based authentication and role-based authorization control access to protected functionalities.

The application follows a layered architecture that separates the frontend, backend, and database components. This modular design supports maintainability and future expansion. By integrating appointment scheduling, doctor availability, notifications, authentication, and administrative management, the project aims to reduce manual work, minimize scheduling conflicts, and improve the organization of healthcare appointments.

Contents

CHAPTER -1 INTRODUCTION	11
1.1 Background of the Project	11
1.2 Problem Statement.....	12
1.3 Scope of the Project	12
1.3.1 Patient Management	13
1.3.2 Doctor Management	13
1.3.4 Notification Management	14
1.3.5 Administrator Management	14
1.3.7 Limitations of the Current Scope.....	15
1.4 Objectives of the Project.....	15
1.5 Importance of the Application	17
1.5.1 Importance for Patients.....	17
1.5.2 Importance for Doctors.....	17
1.5.3 Importance for Administrators	17
1.5.4 Security Importance.....	18
1.5.5 Technical Importance	18
1.5.6 Maintainability and Scalability.....	19
1.5.7 Overall Importance	19
1.6 Target Users / Audience	19
1.6.1 Patients.....	19
1.6.2 Doctors / Healthcare Providers.....	20
1.6.3 Administrators	21
1.6.4 Healthcare Organizations.....	21
1.7 Chapter Summary	21
CHAPTER -2 SYSTEM REQUIREMENTS	23
2.1 Hardware Requirements	23
2.1.1 Development Computer.....	23
2.1.2 Storage Requirements.....	23
The storage requirements include:.....	23
2.1.3 Internet Connectivity	24
2.1.4 Client Hardware Requirements.....	24
2.1.5 Server Hardware Requirements	24
2.2 Software Requirements.....	24
2.2.1 Operating System.....	25
2.2.2 Frontend Software	25
2.2.3 Backend Software	26
2.2.4 Database Software	26

2.2.5 Authentication Software	27
2.2.6 Web Browser	27
2.2.7 API Testing Software	27
2.3 Development Tools and Frameworks	28
2.3.1 Visual Studio Code	28
2.3.2 Python	28
2.3.3 FastAPI	28
2.4 Summary of System Requirements	30
2.5 Overall Requirement Conclusion	31
CHAPTER 3 – TECHNOLOGY STACK.....	32
3.1 Front-End Design – React.js, HTML, CSS and JavaScript.....	32
3.1.4 React.js and Vite	33
3.2 Back-End – Python and FastAPI	33
3.3 Database – MySQL.....	35
3.4 Authentication – JWT.....	36
3.6 Version Control – Git and GitHub.....	38
3.7 API Testing – Postman	38
3.8 Integrated Technology Stack	40
3.9 Summary.....	40
CHAPTER -4 SYSTEM ARCHITECTURE	42
4.1 High-level architecture diagram	42
Figure 4.1: System Architecture	42
4.2 Users of the System	43
4.2.1 Patient	43
4.2.2 Doctor	44
4.2.3 Administrator.....	44
4.3 Frontend Layer.....	44
4.4 Backend Layer	45
Authentication APIs.....	45
Patient APIs	45
Doctor APIs	45
Appointment APIs	45
Notification APIs	45
Admin APIs	46
4.5 JWT Authentication and Authorization.....	46
4.6 Database Layer	46
4.7 Doctor and Appointment Management	47
4.8 Notification and Admin Module.....	48

4.9 API Testing and Development Tools	49
Postman	49
Git	49
GitHub	49
4.10 Overall Data Flow.....	49
4.11 Summary.....	50
CHAPTER -5 DESIGN	51
5.1 Use Case Diagram	51
5.1.1 Patient Use Cases.....	52
5.1.2 Doctor Use Cases.....	52
5.1.3 Administrator Use Cases	52
5.1.4 Use Case Representation	53
5.2 Entity Relationship (ER) Diagram.....	53
5.2.1 Patient Entity	54
5.2.2 Doctor Entity	54
5.2.3 Specialization Entity	54
5.2.5 Notification Entity	54
5.2.6 Entity Relationships.....	54
5.3 Database Schema	55
5.3.1 Patient Table	55
5.3.2 Doctor Table	56
5.3.3 Specialization Table.....	56
5.3.4 Appointment Table	56
5.3.5 Notification Table	56
5.4 Data Flow Diagram.....	57
5.4.1 Level 0 Data Flow	57
5.4.2 Level 1 Data Flow	57
5.5 Module Design.....	58
5.5.1 Authentication and Patient Module	58
5.5.2 Doctor Module.....	59
5.5.3 Appointment Module.....	59
5.5.4 Notification Module.....	60
5.5.5 Administrator Module	60
5.6 System Workflow	60
5.6.1 User Registration	60
5.6.2 User Login	60
5.6.3 Doctor Selection	61
5.6.4 Appointment Booking	61

5.6.5 Appointment Update and Cancellation.....	62
5.6.6 Notification Workflow.....	62
5.6.7 Overall System Workflow	63
5.7 Summary.....	64
CHAPTER -6 IMPLEMENTATION	65
6.1 – Module-Wise Implementation	65
6.1.1 Authentication and Patient Module	65
6.1.2 Doctor and Appointment Module.....	66
6.1.3 Notification and Admin Module.....	68
6.2 Front-End Logic and User Interface Implementation.....	69
6.4 Database Integration	72
6.5 Authentication and Authorization Implementation	74
6.6 Notification Implementation.....	75
CHAPTER 7- FEATURES	77
7.1 List of Main Features.....	77
7.2How Each Feature Works.....	77
7.3Feature Integration.....	83
CHAPTER -8 TESTING	84
8.1. Postman Testing (Frontend/Backend)	84
8.2 Integration Testing.....	86
8.3 Tools Used for Testing	89
CHAPTER -9 DEPLOYMENT.....	94
9.1Steps to Deploy.....	94
9.2 Environment Configuration.....	95
9.3 Hosting of Frontend and Backend	96
CHAPTER -10 CHALLENGES & LIMITATIONS.....	100
10.1 Challenges Faced During Development	100
10.1.1 Frontend–Backend Integration	100
10.1.2 Database Connectivity	100
10.1.3 Authentication and Authorization.....	101
10.1.4 Appointment Scheduling	101
10.1.5 Notification Management	101
10.1.6 API Testing.....	101
10.1.7 Team Integration.....	102
10.2Solutions Applied	102
10.2.1 Layered Architecture	102
10.2.2 REST API Communication	103
10.2.3 Database Relationships.....	103

10.2.4 Authentication.....	103
10.2.5 Appointment Validation	103
10.2.6 API Testing.....	103
10.2.7 Version Control	103
10.3 Current Limitations or Known Bugs	104
10.3.1 Deployment Limitation.....	104
10.3.2 Notification Limitation	104
10.3.3 Scalability Limitation	104
CHAPTER -11 FUTURE ENHANCEMENTS	105
11.1 Advanced Notification Features	105
11.2 Reports and Analytics.....	105
11.3 Cloud Deployment.....	106
11.4 MongoDB Integration if Required	106
CHAPTER -12 CONCLUSION	108
12.1 Summary of the Project	108
12.2 Major Features Implemented.....	108
12.3 DBMS Concepts Applied	109
12.4 Backend and API Concepts Applied	110
12.5 Overall Outcome.....	111
12.6 Skills Applied	111
CHAPTER -13 REFERENCES.....	113
13.1 Books	113
13.2 Tutorials.....	113
13.3 FastAPI Documentation	113
13.4 MySQL Documentation	113
13.5 Postman Documentation.....	114
13.6 Other Resources Used.....	114
CHAPTER -14 APPENDICES	116

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Healthcare is an essential part of everyday life, and timely access to doctors and medical services is important for effective healthcare delivery. With the increasing use of digital technologies, healthcare services are gradually moving from traditional manual processes toward online platforms. Appointment scheduling is one of the areas where digitalization can significantly improve convenience for both patients and healthcare providers.

Traditionally, patients often need to contact hospitals or clinics through phone calls, direct visits, reception desks, or manual registration systems to schedule appointments. These methods can be time-consuming and may result in long waiting times, scheduling conflicts, difficulty in tracking appointments, and additional administrative work. Patients may also have limited information about doctor availability and specialization before making an appointment.

The proposed project, “**Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development**,” is designed to address these challenges by providing a centralized web-based appointment scheduling platform. The system allows patients to register, securely log in, view available doctors and their specializations, check appointment slots, and manage their appointments online.

The system also provides administrative functionality for managing patient information, doctor information, appointments, and notifications. Appointment status management and notification features are included to support communication between patients, doctors, and healthcare services.

From a technical perspective, the application follows a layered architecture that separates the frontend, backend and database components. The main frontend is developed using React.js, HTML, CSS and JavaScript, with Vite as the development tool. Python and FastAPI are used to develop the backend REST APIs, while MySQL stores patient, doctor, appointment and other application data. JWT-based authentication and role-based authorization help protect access to patient and administrative functionality.

The project combines several important areas of computer science, including:

1. Web application development
2. REST API development
3. Database management
4. Authentication and authorization
5. Distributed backend development
6. Software engineering and modular design
7. User interface and user experience design

The overall purpose of the project is to create a reliable, secure, and scalable digital solution that reduces manual appointment management and makes healthcare scheduling more convenient.

1.2 Problem Statement

Traditional healthcare appointment scheduling often depends on phone calls, physical visits, reception desks, or manually maintained records. These processes can create several difficulties for patients and healthcare administrators.

Patients may not have easy access to information about available doctors, their specializations, or available appointment slots. They may need to repeatedly contact the hospital or clinic to confirm availability. Manual scheduling can also result in duplicate bookings, scheduling conflicts, missed appointments, and difficulties in maintaining accurate records.

The system must provide patients with accurate doctor information and available appointment slots while enabling authorized administrators to maintain doctor profiles, availability and appointment records. It should also support secure authentication, appointment updates, cancellations and appointment-related notifications.

From the administrative perspective, managing patient information, doctor availability, appointment schedules, and notifications manually requires significant effort. Maintaining separate records can also make it difficult to retrieve updated information quickly.

Another major problem is the lack of centralized appointment management. When information is stored across different systems or maintained manually, communication between patients, doctors, and administrators becomes less efficient.

Security is also an important concern because healthcare applications handle personal information. Unauthorized access to patient records and appointment information must be prevented through appropriate authentication and authorization mechanisms.

Therefore, there is a need for a centralized healthcare appointment scheduling system that can:

1. Reduce dependency on manual appointment booking.
2. Allow patients to view doctors and their specializations.
3. Display available appointment slots.
4. Allow patients to book, update, and cancel appointments.
5. Help administrators manage healthcare-related information.
6. Provide appointment status and notification features.
7. Protect user information through secure authentication.
8. Provide a scalable backend capable of handling multiple requests.

The proposed system addresses these requirements through a secure web-based platform using **REST APIs, FastAPI, MySQL, and JWT authentication.**

1.3 Scope of the Project

The scope of the project covers the development of a **web-based healthcare appointment scheduling application** that connects patients, doctors, and administrators through a centralized platform.

The primary scope includes **patient registration and authentication**. Patients can create accounts, log in securely, and manage their personal information.

The system also covers **doctor management**, where doctor information, specializations, and availability can be maintained. Patients can use this information to identify suitable doctors and available appointment slots.

The **appointment management** functionality allows patients to book appointments according to available slots. Patients can also view their appointments and update or cancel them when required.

The system includes a **notification module** that can provide appointment-related notifications, reminders, and status updates. This helps improve communication between the healthcare service and patients.

An **administrator module** is included for managing patient, doctor, appointment, and notification-related information. This provides centralized control over important system operations.

1.3.1 Patient Management

The patient management component allows patients to create accounts and access the system securely.

The patient-related functionality includes:

- Patient registration
- Patient login
- Secure authentication
- Viewing doctor information
- Viewing doctor specializations
- Checking appointment availability
- Booking appointments
- Viewing appointments
- Updating appointments
- Cancelling appointments
- Receiving appointment-related notifications

1.3.2 Doctor Management

The Doctor Management Module allows authorized administrators to create and update doctor profiles, maintain specialization information and configure available appointment slots. Patients can view doctor details and check availability before booking appointments. The current implementation does not include a separate doctor login or doctor dashboard.

The doctor-related functionality includes:

- Managing doctor information
- Maintaining specialization details
- Managing doctor availability

- Viewing appointment-related information according to authorized access

This functionality allows patients to identify doctors and available appointment slots before making an appointment.

1.3.3 Appointment Management

Appointment management is one of the primary functions of the application.

The system allows patients to select an available doctor and appointment slot and create an appointment. Appointment information is maintained in the database and can be retrieved when required.

The appointment management functionality includes:

- Appointment booking
- Viewing appointments
- Appointment status management
- Appointment updating
- Appointment cancellation
- Maintaining appointment information
- Generating appointment-related notifications

1.3.4 Notification Management

The notification module supports communication related to appointments.

Notifications may be associated with events such as:

- Appointment confirmation
- Appointment updates
- Appointment cancellation
- Appointment reminders
- Appointment status changes

This functionality helps users remain informed about important appointment-related activities.

1.3.5 Administrator Management

The Administrator Module provides a centralized interface for managing patients, doctors, doctor availability, appointments, holidays and payment records. Administrative operations are protected using authentication and role-based authorization.

Administrators can manage relevant information related to:

- Patients
- Doctors
- Appointments

- Notifications
- Other system-related records

Administrative functionality is protected through authentication and authorization mechanisms.

1.3.6 Technical Scope

The technical scope of the project includes:

- Frontend development using HTML, CSS, and JavaScript
- Backend development using Python and FastAPI
- REST API development
- MySQL database integration
- JWT-based authentication
- Role-based access control
- Appointment management
- Notification management
- API testing using Postman
- Version control using Git and GitHub

The application follows a modular architecture that separates the frontend, backend, authentication, and database components. This organization supports easier development, testing, maintenance, and future extension.

1.3.7 Limitations of the Current Scope

The current application includes a demonstration payment workflow, payment history and receipt information. However, it does not integrate a live payment gateway or process verified real-money transactions. Video consultations, electronic medical records and external hospital-system integration are outside the current scope.

Advanced healthcare features such as online payment processing, video consultation, electronic medical records, AI-based diagnosis, and integration with external hospital systems are outside the current scope unless they are specifically implemented as part of the project.

The system is therefore focused on providing the core functionality required for appointment scheduling and related management activities.

1.4 Objectives of the Project

The main objective of the project is to develop a **secure, scalable, and user-friendly healthcare appointment scheduling system** that simplifies the process of booking and managing medical appointments.

Frontend Development: To develop an interactive and responsive frontend using React.js, HTML, CSS and JavaScript, with Vite as the development tool.

Backend Development: To implement REST APIs using Python and FastAPI for authentication, doctor information, appointment scheduling and administrative operations.

Database Management: To use MySQL for storing and managing application records.

Security: To implement JWT-based authentication and role-based authorization for protected operations.

Appointment Management: To allow patients to book, view, reschedule and cancel appointments.

Payment Demonstration: To implement a simulated payment workflow with payment history and receipt information.

The specific objectives of the project are:

1. **To develop a centralized healthcare appointment platform** for managing patients, doctors, and appointments.
2. **To provide secure patient registration and login** using JWT-based authentication.
3. **To allow patients to view doctors and their specializations** before booking appointments.
4. **To provide appointment slot availability** so that patients can select suitable appointment times.
5. **To enable online appointment booking, updating, and cancellation.**
6. **To provide appointment status management** for better tracking of scheduled appointments.
7. **To implement notification functionality** for appointment confirmations, updates, cancellations, reminders, and status changes.
8. **To provide administrative functionality** for managing patients, doctors, appointments, and notifications.
9. **To use MySQL for structured and reliable data management.**
10. **To develop REST APIs using FastAPI** for communication between the frontend and backend.
11. **To reduce manual administrative work** associated with appointment scheduling.
12. **To minimize scheduling conflicts** by providing centralized appointment and availability management.
13. **To provide secure access to system functionality** based on user roles.
14. **To provide a modular architecture** in which the frontend, backend, authentication, and database components are logically separated.
15. **To support future scalability and enhancement** of the healthcare appointment scheduling system.

These objectives are consistent with the documented project functionality, including patient management, doctor availability, appointment booking, notifications, administration, MySQL, FastAPI, REST APIs, and JWT authentication.

1.5 Importance of the Application

The proposed healthcare appointment scheduling application is important because it provides a centralized digital platform for managing appointments and related healthcare scheduling information.

Traditional appointment management can require patients to make phone calls or visit healthcare facilities simply to check availability or schedule an appointment. A web-based system provides a more organized way of accessing doctor information and managing appointments.

The application provides patients with a centralized interface for finding doctors, checking availability and managing appointments. It also provides administrators with tools for maintaining doctor information, availability and appointment records. Its layered architecture separates the React frontend, FastAPI backend and MySQL database, supporting maintainability and future development.

1.5.1 Importance for Patients

For patients, the system provides convenient access to healthcare appointment information.

Patients can:

- View available doctors
- View doctor specializations
- Check appointment availability
- Book appointments
- View scheduled appointments
- Update appointments
- Cancel appointments
- Receive appointment-related notifications

This reduces the need for repeated communication with healthcare staff for routine appointment management.

1.5.2 Importance for Doctors

For doctors and healthcare providers, the system provides an organized method of managing information related to their specialization, availability, and scheduled appointments.

Maintaining structured appointment information can help doctors and healthcare providers access relevant scheduling information through the system according to their authorized roles.

1.5.3 Importance for Administrators

For administrators, the system provides centralized management of important healthcare scheduling information.

Administrators can manage:

- Patient information
- Doctor information
- Appointment information
- Notification-related information

This can reduce the dependence on separate manual records and make appointment-related information easier to manage.

1.5.4 Security Importance

Security is an important part of the application because the system handles user and appointment-related information.

The implementation of **JWT-based authentication and authorization** provides a mechanism for identifying authenticated users and controlling access to protected functionality.

Role-based access helps ensure that patients, doctors, and administrators access functionality appropriate to their respective roles.

1.5.5 Technical Importance

The project is also important from a technical and academic perspective because it demonstrates the practical implementation of several computer science concepts.

These concepts include:

- Web application development
- Frontend development
- Backend development
- REST API development
- Database management
- Authentication
- Authorization
- Role-based access
- API testing
- Version control
- Modular software architecture

The project therefore provides practical experience in developing a multi-layer web application using commonly used software development technologies.

1.5.6 Maintainability and Scalability

The separation of frontend, backend, authentication, and database components provides an organized architecture.

The frontend is responsible for user interaction, the FastAPI backend processes application requests, JWT provides authentication and authorization, and MySQL provides structured data storage.

This separation makes the application easier to maintain because changes to one component can be managed independently of other components where appropriate.

The modular architecture also provides a foundation for adding additional features in the future without completely redesigning the existing application.

1.5.7 Overall Importance

Overall, the application is intended to:

- Reduce manual appointment management
- Improve appointment accessibility
- Provide centralized appointment information
- Help users track appointments
- Improve appointment-related communication
- Provide controlled access to system functionality
- Support organized management of patients and doctors
- Provide a structured backend architecture
- Provide a foundation for future enhancements

The documented project specifically identifies improved accessibility, reduced manual work, appointment tracking, notifications, secure access, and future scalability as important aspects of the application.

1.6 Target Users / Audience

The healthcare appointment scheduling system is designed primarily for **patients, doctors or healthcare providers, and administrators**. Each category of user has different responsibilities and access requirements within the application.

The system uses authentication and role-based access so that users can access functionality appropriate to their roles.

1.6.1 Patients

Patients are the primary users of the healthcare appointment scheduling system.

The main purpose of the system for patients is to provide a convenient method of finding doctors, checking availability, and managing appointments.

Patients can:

- Register for an account
- Log in securely
- View doctor information
- View doctor specializations
- Check available appointment slots
- Book appointments
- View scheduled appointments
- Update appointments
- Cancel appointments
- View appointment status
- Receive appointment-related notifications

The patient can access their own appointment-related information through the authenticated system.

1.6.2 Doctors / Healthcare Providers

Doctors and healthcare providers are another important category of users.

The system allows doctors to maintain relevant professional information and availability.

Doctors and healthcare providers are represented in the application through their professional profiles, specializations and available appointment slots. Patients use this information to identify doctors and schedule appointments. In the current implementation, authorized administrators maintain doctor information and availability. A separate doctor account or doctor-facing dashboard is not provided.

Doctor-related activities can include:

- Managing doctor profile information
- Maintaining specialization information
- Managing availability
- Viewing scheduled appointment information
- Managing appointment-related information according to authorized permissions

Doctor information is important because patients use the available doctor details and specialization information when selecting a suitable appointment.

1.6.3 Administrators

Administrators are responsible for managing important system information and administrative operations.

The administrator can manage relevant information related to:

- Patients
- Doctors
- Appointments
- Notifications
- Other system-related operations

Administrative functions are protected through authentication and authorization so that only authorized users can perform administrative operations.

1.6.4 Healthcare Organizations

Healthcare organizations such as clinics or hospitals can be considered potential organizational users of the system.

The system provides a centralized structure for managing appointment-related information and can support the organization of:

- Doctor information
- Patient information
- Appointment schedules
- Appointment status
- Notifications
- Administrative activities

The application can therefore provide a foundation for digital appointment management within a healthcare organization, depending on the deployment and configuration of the system.

The documented project identifies patients, doctors or healthcare providers, administrators, and healthcare organizations as the intended target audience.

1.7 Chapter Summary

This chapter introduced the Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development project and explained the need for an online healthcare appointment scheduling platform. It discussed the limitations of manual appointment management, including scheduling conflicts, limited visibility of doctor availability and difficulties in maintaining appointment records.

The scope and objectives of the application cover patient registration and authentication, doctor discovery, appointment booking and management, notifications, administrator-managed doctor

information and availability, and a demonstration payment workflow. The application uses React.js, HTML, CSS and JavaScript with Vite for the frontend, Python and FastAPI for backend REST APIs, MySQL for database management, and JWT for authentication and authorization. The chapter also described the application's importance and identified patients and administrators as its implemented user roles, with doctors and healthcare organizations represented through the services and information managed by the system.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware requirements define the physical resources required to develop, test, and operate the **Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development** application.

Since the proposed system is a web-based application, it does not require specialized hardware. A standard computer with sufficient processing power, memory, storage, and network connectivity is adequate for development and testing.

The development environment may need to run several applications simultaneously, including Visual Studio Code, Python, FastAPI, MySQL, a web browser, and Postman. Therefore, a moderately powerful computer is recommended for smooth development and testing.

2.1.1 Development Computer

The development computer is used by the project team to write source code, run the FastAPI backend, manage the MySQL database, test APIs, and access the frontend.

The following configuration provides a suitable development environment:

Hardware Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3 / Equivalent	Intel Core i5 or higher
RAM	8 GB	16 GB
Storage	256 GB	512 GB SSD
Display	1366 × 768	1920 × 1080
Network	Stable Internet	High-speed Internet
Keyboard	Standard keyboard	Standard keyboard
Mouse	Standard mouse	Standard mouse

A multi-core processor is useful because several development processes may run simultaneously. For example, the FastAPI server, MySQL database, Postman, web browser, and code editor may all be active during development.

At least **8 GB of RAM** is suitable for basic development, while **16 GB of RAM** provides better performance when multiple applications are running simultaneously. SSD storage is preferred because it provides faster file access and improves application startup and database operations.

2.1.2 Storage Requirements

Storage is required for maintaining the different components and resources associated with the project.

The storage requirements include:

- Project source code
- Python packages and dependencies
- MySQL database files
- API testing data
- Application documentation
- Application screenshots
- Git repositories
- Backup files

A minimum of **256 GB of storage** is sufficient for basic development, while **512 GB SSD storage** is recommended for smoother operation and future project expansion.

The source code can also be maintained using Git and GitHub, which helps preserve development history and reduce the risk of losing important project files.

2.1.3 Internet Connectivity

A stable Internet connection is required during development and testing.

Internet connectivity may be required for:

- Installing Python packages and dependencies
- Accessing technical documentation
- Synchronizing code with GitHub
- API testing
- Downloading development tools
- Sharing project resources
- Future cloud deployment

A reliable Internet connection is also useful if the application is later extended to communicate with external healthcare services or cloud-based services.

2.1.4 Client Hardware Requirements

Patients, doctors, and administrators do not require high-end hardware because the application is accessed through a web browser.

The system can be accessed using:

- Desktop computers
- Laptops
- Tablets
- Smartphones

A basic client device with approximately **4 GB RAM**, a modern web browser, and a stable Internet connection should be sufficient for normal usage.

Since the application is web-based, most application processing is handled by the backend server rather than requiring high processing capability on the user's device.

2.1.5 Server Hardware Requirements

During development, the FastAPI backend and MySQL database can run on the development computer. For larger-scale deployment, the backend and database can be hosted on a dedicated server or cloud infrastructure.

A basic server configuration may include:

- Multi-core processor
- 8–16 GB RAM
- SSD storage
- Stable network connection
- Appropriate operating system
- Database storage and backup facilities

As the number of patients, doctors, and appointments increases, server resources can be upgraded according to the requirements of the deployed application.

2.2 Software Requirements

Software requirements define the software environment required to develop, test, run, and maintain the healthcare appointment scheduling application.

The proposed system uses a combination of frontend technologies, backend frameworks, database software, authentication mechanisms, API testing tools, version-control tools, and web browsers.

2.2.1 Operating System

The application can be developed and tested using commonly available operating systems such as:

- Windows 10
- Windows 11
- Ubuntu Linux
- macOS

Windows provides convenient access to Visual Studio Code, Python, MySQL, web browsers, and Postman. Linux can also be used, particularly for backend and server deployment environments.

The operating system should support the installation and execution of the project's development tools and dependencies.

2.2.2 Frontend Software

The frontend of the healthcare appointment scheduling system is developed using:

The main frontend of the healthcare appointment scheduling system is developed using React.js, HTML, CSS and JavaScript. Vite is used as the frontend development and build tool. Node.js and npm are required to install frontend dependencies and run the development server.

React.js is used to develop reusable components and interactive interfaces for patient registration, login, doctor discovery, appointment booking, appointment management, notifications and administrative operations.

HTML defines the structure of the application's pages and components, CSS controls their appearance and layout, and JavaScript handles interactive functionality and communication with the FastAPI backend through REST APIs.

React.js and Vite

React.js is a JavaScript library used to build component-based user interfaces. It allows the application to organize its frontend into reusable components and update the displayed information when application data changes.

Vite provides the frontend development environment and build tooling. Node.js and npm are used to install the required dependencies and execute the project's frontend development commands.

HTML, CSS, and JavaScript

HTML

HTML is used to define the structure of the web application.

It can be used to create:

- Login forms
- Registration forms
- Navigation menus
- Doctor information sections
- Appointment forms
- Appointment tables
- Patient dashboards
- Administrative interfaces
- Notification sections

CSS

CSS is used to control the visual presentation and layout of the application.

It provides styling for:

- Buttons
- Forms
- Tables
- Navigation elements
- Doctor information cards
- Appointment sections
- Spacing and alignment
- Responsive layouts

JavaScript

JavaScript provides dynamic functionality for the frontend.

It can be used to:

- Validate user inputs
- Send requests to the FastAPI backend
- Receive API responses
- Display doctor information
- Retrieve appointment availability
- Submit appointment requests
- Update appointment information
- Cancel appointments
- Display notifications dynamically
- Handle authentication-related information

The frontend communicates with the FastAPI backend through REST APIs.

2.2.3 Backend Software

The backend is developed using:

Python and FastAPI

Python provides the programming environment for implementing the business logic of the application.

FastAPI is used to develop RESTful APIs that handle communication between the frontend and the database.

The backend is responsible for functions such as:

- User registration
- User login
- Authentication
- Authorization
- Patient management
- Doctor management
- Doctor availability
- Appointment management
- Appointment status management
- Notification management
- Administrative operations

The backend acts as an intermediary between the frontend and the MySQL database.

2.2.4 Database Software

MySQL is used as the relational database management system for the project.

The database stores important information required by the application, including:

- Patient details
- Doctor details

- Doctor specialization
- Appointment details
- Appointment date and time
- Appointment status
- Notification information
- User authentication information

MySQL provides structured data storage and supports relationships between different entities. For example, an appointment can be associated with a particular patient and doctor. This relationship allows the system to identify which patient has booked an appointment with which doctor.

2.2.5 Authentication Software

The system uses **JSON Web Token (JWT)** for authentication.

JWT provides a mechanism for verifying whether a user has successfully logged in before allowing access to protected resources.

The authentication process can be represented as:

User Login → Credential Verification → JWT Generation → Token Sent to Client → Protected API Access

After successful login, the backend generates a JWT token. The token can then be used when accessing protected API endpoints.

Authentication is particularly important because different users have different levels of access within the application.

2.2.6 Web Browser

A modern web browser is required to access and test the web application.

Supported browsers can include:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

The browser is used to:

- Access the frontend
- Test registration and login forms
- Check the user interface
- Verify application responses
- Test responsive layouts
- Debug JavaScript functionality

A modern browser also provides developer tools that can help identify frontend errors and inspect API requests and responses.

2.2.7 API Testing Software

Postman is used for testing the REST APIs developed using FastAPI.

Postman allows developers to send different HTTP requests, including:

- GET
- POST
- PUT
- DELETE

Postman can be used to test APIs such as:

- Registration API
- Login API

- Doctor API
- Doctor availability API
- Appointment booking API
- Appointment update API
- Appointment cancellation API
- Notification API
- Administrative APIs

Testing the backend APIs independently helps identify errors before connecting the APIs to the frontend.

2.3 Development Tools and Frameworks

Development tools provide the environment required to create, test, manage, and maintain the application.

2.3.1 Visual Studio Code

Visual Studio Code (VS Code) is used as the primary code editor for development.

It provides a convenient environment for working with:

- HTML
- CSS
- JavaScript
- Python
- FastAPI
- SQL

VS Code also provides features such as:

- Syntax highlighting
- Debugging
- Extensions
- Integrated terminal
- Project file management
- Code navigation

These features make it suitable for managing the different components of the project.

2.3.2 Python

Python is the main programming language used for backend development.

Python is used to implement:

- Authentication logic
- Appointment processing
- Doctor management
- Database communication
- Notification logic
- REST API endpoints

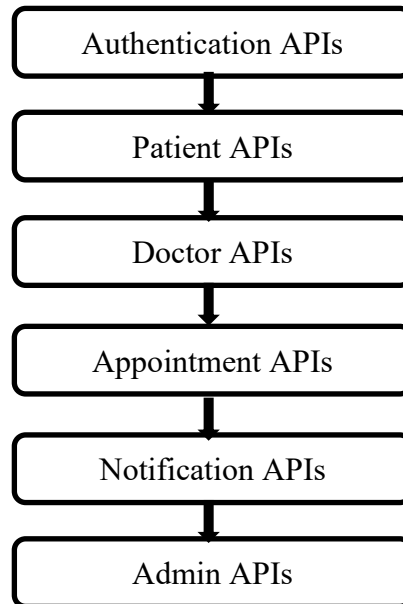
Python provides a relatively simple programming environment and supports libraries and frameworks required for backend application development.

2.3.3 FastAPI

FastAPI is the main backend framework used in the project.

It provides a structured approach for creating REST APIs and handling HTTP requests.

The backend can be organized into different API components, such a



This modular approach makes the backend easier to develop, test, debug, and maintain.

2.3.4 MySQL

MySQL is used for database management.

It provides features such as:

- Structured data storage
- SQL querying
- Table relationships
- Data retrieval
- Data modification
- Data consistency
- Database administration

The database acts as the permanent storage layer for patient, doctor, appointment, and notification information.

2.3.5 Git

Git is used for version control.

Git allows the development team to:

- Track changes
- Create branches
- Commit code
- Merge changes
- Restore earlier versions
- Maintain development history

Since the project contains multiple modules, version control helps the development team manage changes to the project source code.

2.3.6 GitHub

GitHub is used as the remote repository for the project.

It provides:

- Online source-code storage
- Team collaboration
- Backup
- Version history
- Repository management
- Code sharing

GitHub allows project team members to synchronize their work and maintain a centralized copy of the source code.

2.3.7 Postman

Postman is used throughout backend development and testing.

The development team can create and test an API in Postman before integrating it with the frontend.

Examples of API operations include:

POST /login

POST /register

GET /doctors

GET /appointments

POST /appointments

PUT /appointments/{id}

DELETE /appointments/{id}

GET /notifications

Postman makes it easier to verify whether individual backend services are functioning correctly.

2.3.8 MySQL Database Management Tools

Database management tools are used to create tables, execute SQL queries, inspect records, and troubleshoot database-related problems.

They help developers verify whether:

- Patient records are stored correctly.
- Doctor records are available.
- Appointment records are created.
- Appointment updates are reflected.
- Cancelled appointments are updated correctly.
- Notification records are stored properly.

These tools are useful during database development and testing.

2.4 Summary of System Requirements

The major system requirements can be summarized as follows:

Category	Technologies / Requirements
Operating System	Windows / Linux / macOS
Frontend	HTML, CSS, JavaScript
Backend	Python, FastAPI
Database	MySQL

Category	Technologies / Requirements
Authentication	JWT
Code Editor	Visual Studio Code
API Testing	Postman
Version Control	Git
Repository	GitHub
Browser	Chrome / Edge / Firefox
RAM	8 GB minimum, 16 GB recommended
Storage	256 GB minimum, SSD recommended
Internet	Stable Internet connection

The combination of **HTML, CSS, JavaScript, Python, FastAPI, MySQL, JWT, Postman, Git, and GitHub** provides the required development and testing environment for the healthcare appointment scheduling application.

2.5 Overall Requirement Conclusion

The proposed healthcare appointment scheduling system requires moderate hardware resources and commonly available software tools. The development environment can support the frontend, backend, database, authentication, API testing, and version-control requirements of the project. The combination of **HTML, CSS, and JavaScript** provides the frontend environment, while **Python and FastAPI** provide the backend services. **MySQL** provides structured database management, and **JWT** provides authentication and authorization. **Postman** supports API testing, while **Git and GitHub** support version control and collaborative development.

CHAPTER 3 – TECHNOLOGY STACK

The technology stack defines the programming languages, frameworks, database technologies, authentication mechanisms, testing tools, and development platforms used to build the **Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development** project.

The project uses a relatively simple and practical technology stack consisting of **HTML, CSS, JavaScript, Python, FastAPI, MySQL, JWT, Git, GitHub, and Postman**. This matches the technology stack specified in the project submission.

3.1 Front-End Design – React.js, HTML, CSS and JavaScript

The frontend is the user interface layer of the healthcare appointment scheduling system. It allows patients to register, log in, view doctors, check appointment availability, book appointments, manage existing appointments and view notifications. It also provides a separate administrative interface for managing doctors, patients and appointments.

The main frontend is developed using React.js, HTML, CSS and JavaScript. React.js is used to build reusable components and interactive interfaces, while HTML provides page structure, CSS controls presentation and JavaScript handles user interactions and communication with the backend. Vite is used as the frontend development and build tool.

The React frontend communicates with the FastAPI backend through REST APIs. This separation allows frontend components and backend services to be developed and maintained independently.

The main responsibilities of the frontend include:

Providing registration and login pages.

Displaying patient information.

Displaying doctor information and specializations.

Showing available appointment slots.

Providing appointment booking forms.

- Displaying booked appointments.
- Allowing appointment updates and cancellation.
- Displaying notifications.
- Providing administrative interfaces.

The frontend communicates with the FastAPI backend through REST APIs. When a user performs an operation, the frontend sends a request to the backend, receives the response, and updates the interface accordingly.

3.1.1 HTML

HTML (HyperText Markup Language) provides the basic structure of the application.

It is used to create:

- Login pages
- Registration pages
- Navigation bars
- Forms
- Doctor information sections
- Appointment tables
- Patient dashboards
- Admin dashboards

- Notification sections

HTML forms allow users to enter information such as their name, email, contact details, appointment date, and other required information.

The semantic structure of HTML also makes the application easier to organize and maintain.

3.1.2 CSS

CSS (Cascading Style Sheets) is used to control the visual appearance of the application.

CSS is responsible for:

- Colors and fonts
- Page layouts
- Buttons
- Forms
- Tables
- Navigation menus
- Doctor cards
- Appointment cards
- Spacing and alignment
- Responsive layouts

Responsive CSS allows the application to adapt to different screen sizes such as desktops, laptops, tablets, and smartphones.

The objective is to provide a simple interface in which users can easily find doctors, appointments, and notifications.

3.1.3 JavaScript

JavaScript provides the interactive functionality of the frontend.

It can be used to:

- Validate form inputs.
- Send requests to FastAPI APIs.
- Receive API responses.
- Display doctor information.
- Retrieve available appointment slots.
- Submit appointment booking requests.
- Update appointment information.
- Cancel appointments.
- Display notifications dynamically.
- Handle login and authentication information.

For example, when a patient selects a doctor, JavaScript can send a request to the backend to retrieve that doctor's available appointment slots and display them on the webpage.

3.1.4 React.js and Vite

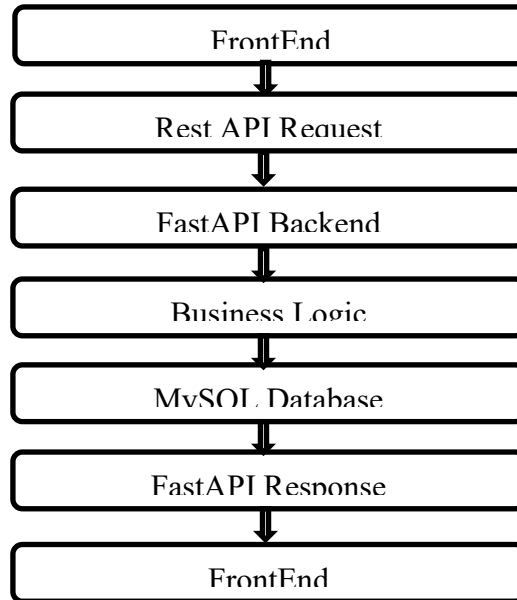
React.js is a JavaScript library used to develop component-based user interfaces. In this project, React.js provides reusable components for authentication, doctor discovery, appointment booking, appointment management, notifications and administrative operations.

React's component-based structure helps organize the frontend and update displayed information in response to user interactions and backend API responses.

Vite is used as the frontend development and build tool. Node.js and npm are used to install frontend dependencies and run the development server.

3.2 Back-End – Python and FastAPI

The backend is the main processing layer of the healthcare appointment scheduling application. The project uses **Python** as the backend programming language and **FastAPI** as the web framework. The backend acts as an intermediary between the frontend and MySQL database. The basic communication structure is:



The backend is responsible for implementing the main functionality of the system. These functions include:

1. User registration
2. User login
3. Authentication
4. Authorization
5. Patient management
6. Doctor management
7. Doctor availability
8. Appointment booking
9. Appointment updating
10. Appointment cancellation
11. Appointment status management
12. Notification management
13. Administrative operations

The project specifically identifies **Python** and **FastAPI** as its backend technologies.

3.2.1 Python

Python is used to implement the backend logic of the application.

Python is responsible for processing user requests and performing operations such as:

- Checking login credentials
- Generating authentication tokens
- Validating appointment requests
- Checking doctor availability
- Storing appointment information
- Retrieving patient information
- Updating appointment status
- Generating notifications

Python also provides access to libraries and frameworks that simplify database communication, authentication, and API development.

3.2.2 FastAPI

FastAPI is the framework used to build the REST API layer.

It receives HTTP requests from the frontend and sends appropriate responses.

The backend can be divided into multiple API groups:

- Authentication APIs
- Patient APIs
- Doctor APIs
- Appointment APIs
- Notification APIs
- Admin APIs

This modular structure makes the backend easier to develop and maintain.

FastAPI also provides automatic API documentation and structured request handling, which helps during development and API testing.

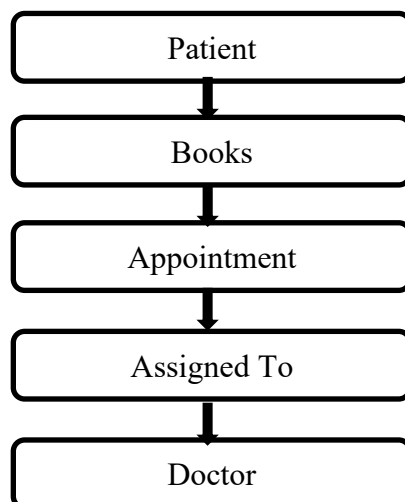
3.3 Database – MySQL

MySQL is used as the database management system for the project. The database stores and manages the information required for the healthcare appointment system. The main types of information stored include:

- Patient information
- Doctor information
- Specialization information
- Appointment information
- Appointment status
- Notification information
- User authentication information

A relational database is suitable for this application because patients, doctors, and appointments have clearly defined relationships.

For example:



This relationship allows the system to identify which patient has booked an appointment with which doctor.

3.3.1 Patient Data

The patient-related table can store information such as:

- Patient ID
- Patient name
- Email
- Contact details
- Password information
- Other required profile details

Each patient can have multiple appointment records.

3.3.2 Doctor Data

The doctor-related table can contain:

- Doctor ID
- Doctor name
- Specialization
- Availability
- Contact information

This information allows patients to identify suitable doctors before booking appointments.

3.3.3 Appointment Data

The appointment table is one of the most important components of the database.

It can contain:

- Appointment ID
- Patient ID
- Doctor ID
- Appointment date
- Appointment time
- Appointment status

The appointment status can help identify whether an appointment is booked, updated, cancelled, or completed.

3.3.4 Notification Data

The notification table stores appointment-related messages.

It can contain:

- Notification ID
- User ID
- Notification message
- Notification date
- Notification status

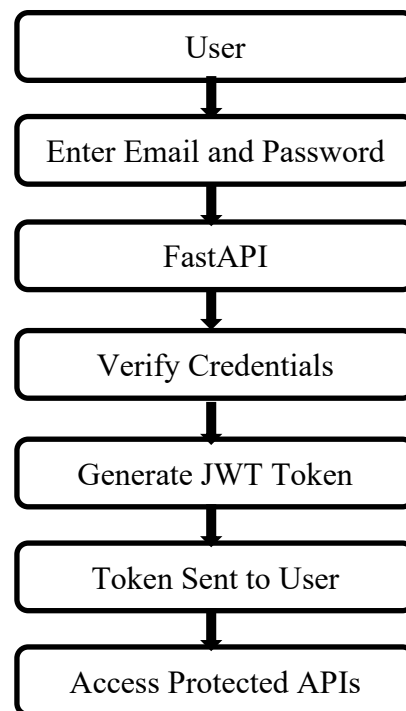
This allows users to view important updates related to their appointments.

3.4 Authentication – JWT

JSON Web Token (JWT) is used for secure authentication in the application.

Authentication ensures that only registered and authorized users can access protected parts of the system.

The authentication process can be represented as:



When a user logs in successfully, the backend generates a JWT token. The token can then be used when accessing protected API endpoints. For example, a patient may need authentication before accessing their appointment information.

3.4.1 Authorization

Authentication verifies **who the user is**, while authorization determines **what the user is allowed to access**.

The system can provide different access levels for different users.

User	Main Access
Patient	Profile and own appointments
Doctor	Doctor and availability-related functions
Administrator	Patient, doctor, appointment and notification management

This helps prevent unauthorized users from accessing administrative functions or another user's private information.

JWT authentication and authorization are included in the project's defined technology stack.

3.5 REST API

REST APIs provide communication between the frontend and backend.

The frontend sends HTTP requests to FastAPI, and FastAPI processes those requests and returns responses.

Typical operations include:

HTTP Method	Purpose
GET	Retrieve information
POST	Create new information
PUT	Update existing information

HTTP Method	Purpose
DELETE	Remove or cancel information

Examples of possible API operations include:

POST /register
 POST /login
 GET /doctors
 GET /doctors/{id}/availability
 GET /appointments
 POST /appointments
 PUT /appointments/{id}
 DELETE /appointments/{id}
 GET /notifications

Using REST APIs keeps the frontend and backend separate. This makes the system easier to maintain and allows the backend to potentially serve other applications in the future.

3.6 Version Control – Git and GitHub

Git is used for version control, while **GitHub** is used to store and collaborate on the project's source code.

Git allows the development team to track changes made to the project.

Important Git operations include:

- Creating repositories
- Committing changes
- Creating branches
- Merging changes
- Tracking previous versions
- Restoring earlier versions

GitHub provides a centralized location where the project code can be stored and shared among team members.

The work is divided into three major modules:

1. Authentication & Patient Module
2. Doctor & Appointment Module
3. Notification & Admin Module

Each team member can work on their assigned component and synchronize their changes through GitHub.

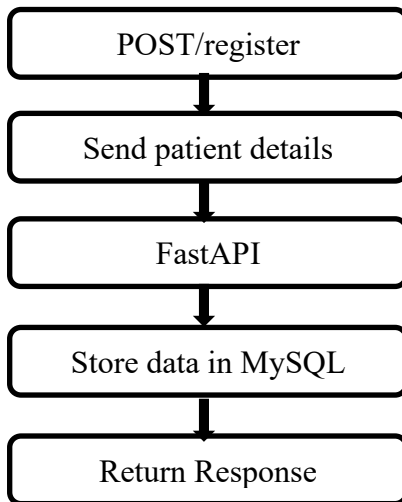
3.7 API Testing – Postman

Postman is used to test the REST APIs developed using FastAPI.

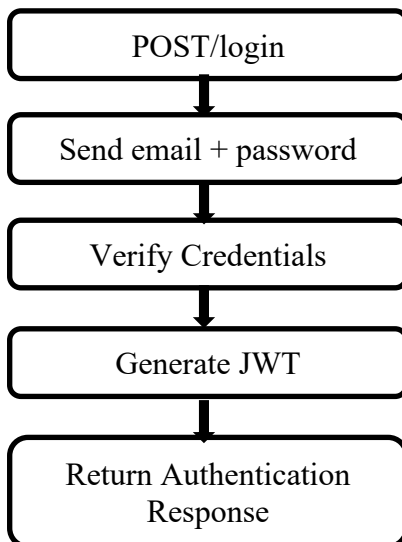
Before connecting the backend to the frontend, individual API endpoints can be tested using Postman.

Postman is an API testing tool that can be used to send HTTP requests, inspect responses and verify backend functionality. FastAPI also provides interactive Swagger documentation, which allows developers to inspect available endpoints and test API requests. The project includes evidence of selected backend responses, API documentation and protected-endpoint authorization behavior.

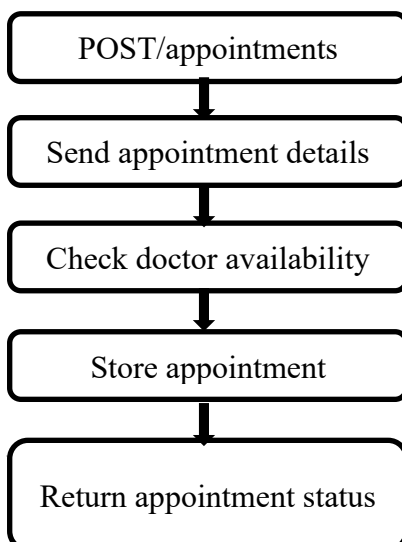
Registration Testing



Login Testing



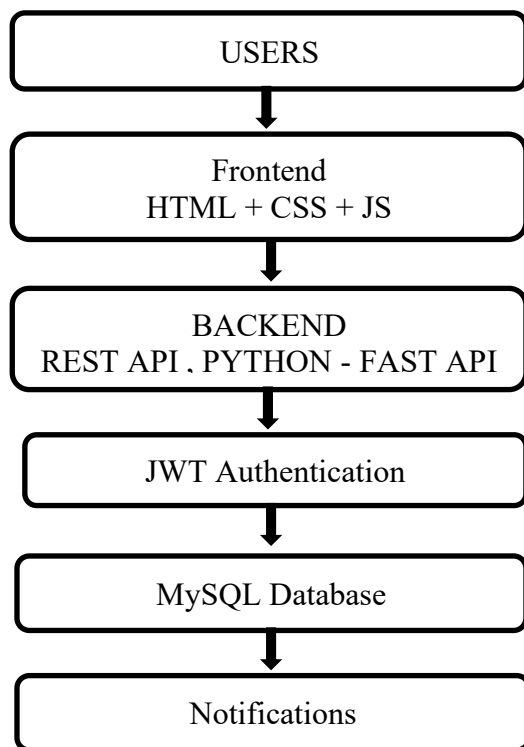
Appointment Testing



Postman helps developers identify API errors, incorrect responses, authentication problems, and database-related issues before frontend integration.

3.8 Integrated Technology Stack

The complete technology stack of the proposed system can be represented as:



The development and testing environment is supported by:

VS Code → Development

Git/GitHub → Version Control

Postman → API Testing

3.9 Summary

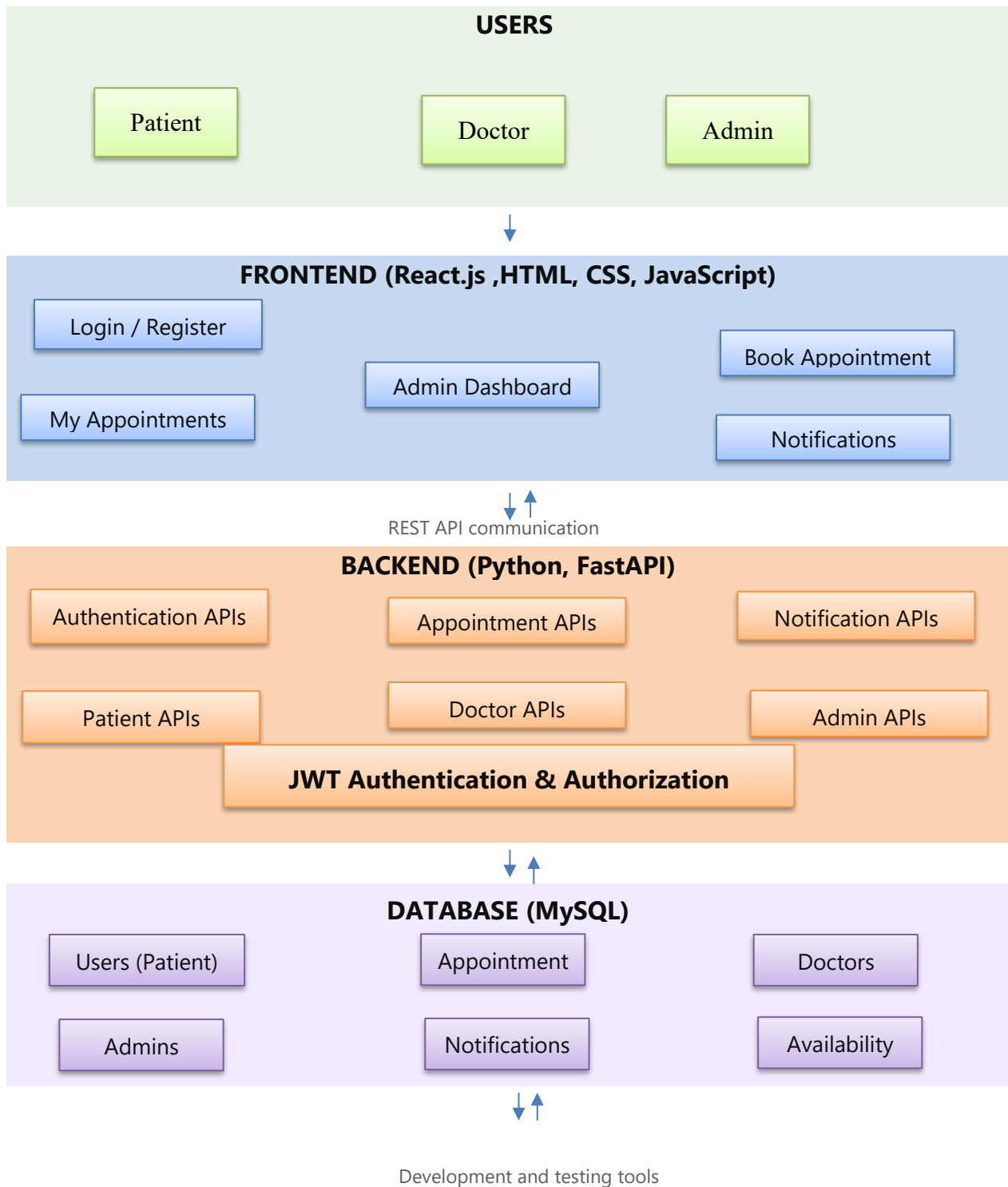
This chapter described the technologies used to develop the healthcare appointment scheduling system. The main frontend uses React.js, HTML, CSS and JavaScript, with Vite providing development and build tooling. Python and FastAPI implement the backend REST APIs, while MySQL provides relational database storage.

JWT-based authentication and role-based authorization protect patient and administrative functionality. Postman and FastAPI Swagger UI support API inspection and testing, while Git and GitHub support source-code management. Together, these technologies provide a modular foundation for the application's appointment scheduling, doctor availability, notification and administrative features.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram

Figure 4.1: System Architecture





The proposed **Secure and Scalable Healthcare Appointment Scheduling System** follows a layered architecture in which the frontend, backend, database, authentication, and supporting tools work together to provide a complete appointment scheduling platform.

The system consists of three main layers:

1. **Frontend Layer**
2. **Backend Layer**
3. **Database Layer**

The frontend provides the user interface, the backend processes requests and implements the application logic, and the database stores and manages the system information. The frontend communicates with the backend through **REST APIs**, while the backend communicates with the **MySQL database** to store and retrieve information.

The architecture also includes **JWT authentication and authorization** to protect user accounts and restricted system functions. **Postman** is used for API testing, while **Git and GitHub** support version control and collaborative development.

The healthcare appointment scheduling system follows a layered architecture consisting of a React.js frontend, a Python and FastAPI backend, and a MySQL database. The frontend communicates with the backend through REST APIs. The backend processes requests, performs application logic, verifies access to protected resources and interacts with the database.

The system provides interfaces for patients and administrators. Patients can search for doctors, check appointment availability, book appointments and manage their bookings. Administrators can manage doctor information, availability, patients and appointments.

JWT-based authentication and role-based authorization protect restricted functionality.

Separating the frontend, backend and database supports modular development, maintenance and future expansion.

4.2 Users of the System

The system is designed for three main categories of users:

4.2.1 Patient

Patients are the primary users of the appointment scheduling system. They can:

- Register for an account.
- Log into the system.
- View available doctors.
- View doctor specializations.
- Check available appointment slots.
- Book appointments.
- View their appointments.

- Update appointments.
- Cancel appointments.
- Receive appointment notifications.

The patient interacts with the system through the frontend, while all appointment-related processing is performed by the FastAPI backend.

4.2.2 Doctor

Doctors are responsible for providing healthcare services through the appointment platform. The doctor-related functionality includes:

- Maintaining doctor information.
- Providing specialization details.
- Managing availability.
- Viewing scheduled appointments.

Doctor information and availability are stored in the MySQL database and can be accessed through the appropriate backend APIs.

4.2.3 Administrator

The administrator is responsible for managing the overall application. The administrator can manage:

- Patient information.
- Doctor information.
- Appointment information.
- Notifications.
- Other administrative operations.

The admin functions provide centralized management of the healthcare appointment scheduling system.

These three user groups and their associated functions are consistent with the project's defined modules and requirements.

4.3 Frontend Layer

The frontend is the presentation layer of the system and is developed using React.js, HTML, CSS and JavaScript, with Vite as the development tool. It provides separate interfaces for patients and administrators.

The frontend supports registration, login, doctor discovery, appointment booking, appointment management, notifications and administrative operations. React components communicate with the FastAPI backend through REST APIs and display the responses to users.

The main frontend functions include:

- Login and registration
- Doctor information
- Appointment booking
- Appointment management
- Notifications
- Admin dashboard

When a user performs an action, JavaScript sends a request to the FastAPI backend through a REST API. The response received from the backend is then displayed on the frontend.

4.4 Backend Layer

The backend is developed using **Python and FastAPI**.

It acts as the central processing layer between the frontend and the database. The backend receives requests, validates information, performs business logic, communicates with MySQL, and sends responses back to the frontend.

The backend also supports the demonstration payment workflow, receipt information and administrator-managed holidays. Administrative APIs provide protected operations for managing patient records, doctor information, doctor availability and appointment statuses.

The backend contains different API components:

Authentication APIs

Handle:

- Registration
- Login
- JWT generation
- Authentication
- Authorization

Patient APIs

Handle:

- Patient information
- Patient profile
- Patient appointments

Doctor APIs

Handle:

- Doctor information
- Specializations
- Doctor availability

Appointment APIs

Handle:

- Appointment creation
- Appointment retrieval
- Appointment updating
- Appointment cancellation
- Appointment status

Notification APIs

Handle:

- Appointment notifications
- Status updates

Admin APIs

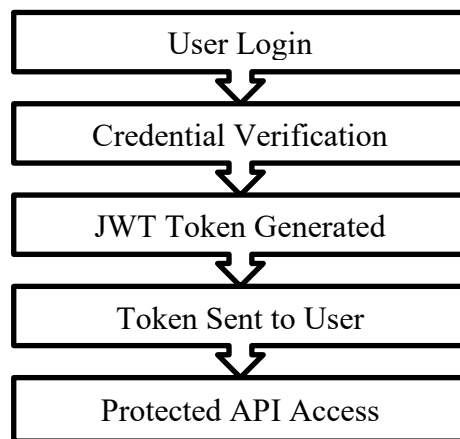
Handle administrative operations related to patients, doctors, appointments, and notifications. The project's submission specifically defines FastAPI as the backend framework and REST APIs as the communication mechanism.

4.5 JWT Authentication and Authorization

JWT (JSON Web Token) is used to provide secure authentication.

When a user logs in, the FastAPI backend verifies the submitted credentials. After successful authentication, a JWT token is generated.

The process is:



The token is used when accessing protected resources.

Authorization is used to control access based on the user's role. For example, a patient should not be able to access administrative functions.

JWT authentication is part of the specified project technology stack.

JWT-based authentication verifies users before they access protected resources. Role-based authorization restricts administrative operations to authorized administrator accounts. Patients can access their permitted appointment and profile functionality but cannot access protected administrator endpoints.

4.6 Database Layer

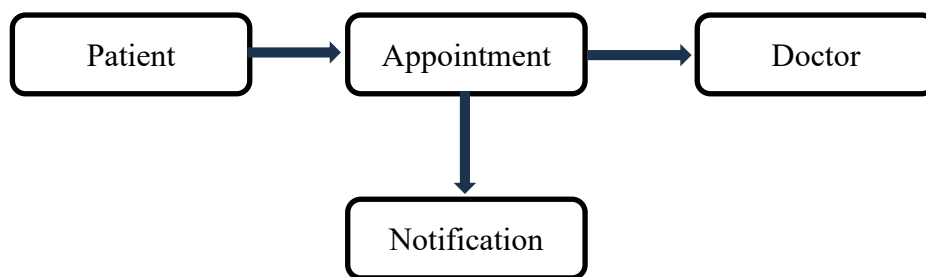
The database layer uses **MySQL** to store the information required by the application.

The major information stored includes:

- Patient details
- Doctor details
- Doctor specializations
- Doctor availability
- Appointment details
- Appointment status
- Notification information
- Administrator information

The backend communicates with MySQL whenever information needs to be created, retrieved, updated, or deleted.

A simplified relationship is:



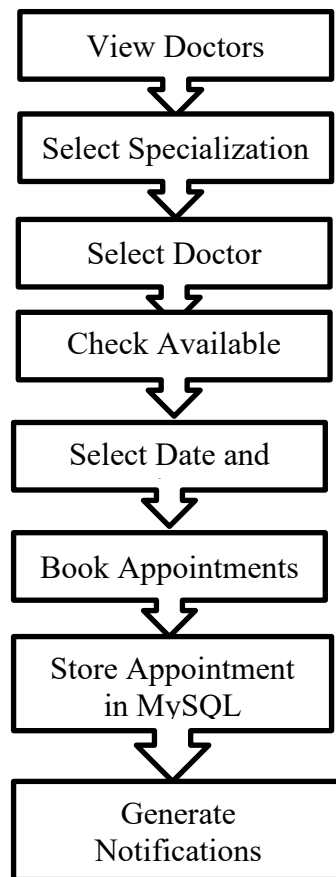
This structure allows appointments to connect patients with doctors and allows appointment-related notifications to be maintained.

4.7 Doctor and Appointment Management

The Doctor and Appointment module forms a major part of the system.

Patients can first view doctors and their specializations and then check their availability.

The appointment process is



The system also allows patients to update or cancel appointments according to the implemented functionality.

The project requirements specifically include doctor availability, appointment booking, updating, cancellation, and appointment status management.

4.8 Notification and Admin Module

The notification module provides appointment-related updates to users.

Notifications can be associated with:

- Successful appointment booking
- Appointment updates
- Appointment cancellation
- Appointment status changes
- Appointment reminders

The admin component provides centralized management of system information.

Administrators can manage:

- Patients
- Doctors
- Appointments
- Notifications

This module improves communication and allows the system to be managed from a centralized interface.

The administrator module provides centralized management of patient records, doctor profiles, doctor availability and appointment statuses. It also supports holiday management and access to payment records. Appointment-related notifications are available through the application's notification functionality.

4.9 API Testing and Development Tools

The architecture also includes supporting development tools.

Postman

Postman is used to test FastAPI REST endpoints independently from the frontend.

It can be used to test:

- Registration
- Login
- Doctor APIs
- Appointment APIs
- Notification APIs
- Admin APIs

Git

Git is used for tracking changes in the project source code and managing different versions.

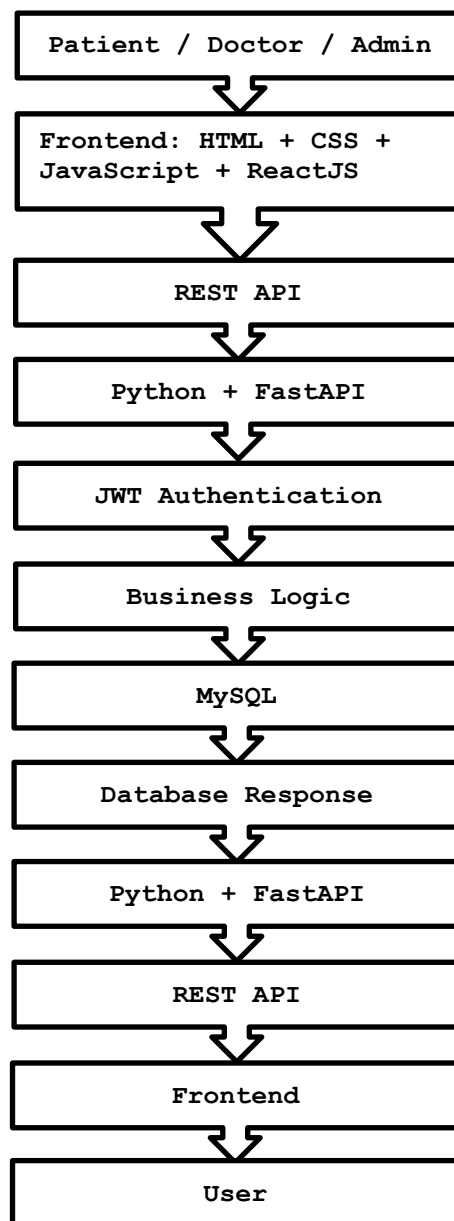
GitHub

GitHub is used as the remote repository for storing and sharing the project source code among team members.

These tools support testing, collaboration, and version management throughout the development process.

4.10 Overall Data Flow

The complete data flow of the system can be summarized as:



This architecture ensures that users do not directly access the database. All requests pass through the backend, where authentication, validation, and business logic are applied.

4.11 Summary

This chapter explained the layered architecture of the healthcare appointment scheduling system. The React.js frontend provides patient and administrator interfaces, while the Python and FastAPI backend handles REST API requests, application logic, authentication and database operations. MySQL provides persistent storage for the application's records.

JWT-based authentication and role-based authorization protect restricted functionality. The appointment, notification, demonstration payment and administrative modules support the main application workflows. The separation of frontend, backend and database components provides a maintainable foundation for future development.

CHAPTER -5 DESIGN

HEALTHCARE APPOINTMENT SCHEDULING SYSTEM

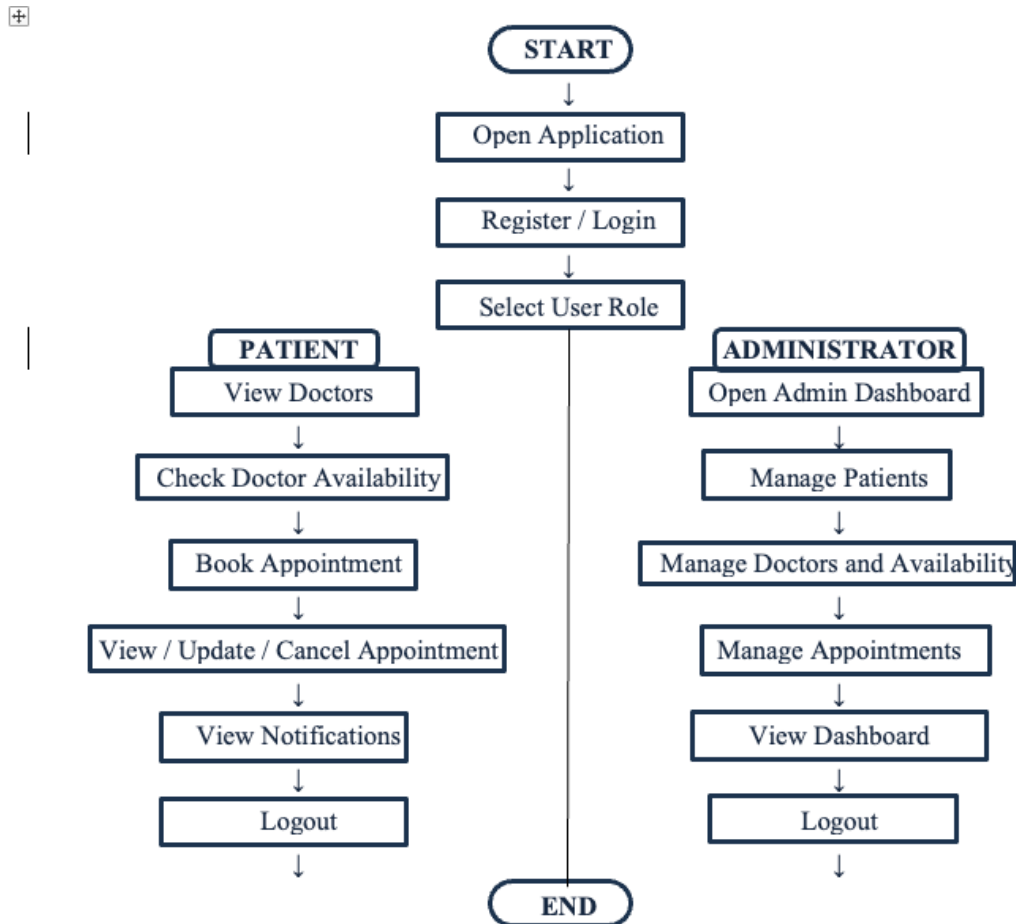


Figure 5.1: Patient and Administrator Workflow

This chapter describes the design of **the Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development project**. The system design explains how patients, doctors, and administrators interact with the application and how the frontend, backend, authentication, database, appointment, and notification components work together.

The main design components covered in this chapter are **the Use Case Diagram, Entity Relationship (ER) Diagram, Database Schema, Data Flow Diagram, Module Design, and System Workflow**.

5.1 Use Case Diagram

This chapter explained the layered architecture of the healthcare appointment scheduling system. The React.js frontend provides patient and administrator interfaces, while the Python and FastAPI backend handles REST API requests, application logic, authentication and database operations. MySQL provides persistent storage for the application's records. JWT-based authentication and role-based authorization protect restricted functionality. The appointment, notification, demonstration payment and administrative modules support the main application workflows. The separation of frontend, backend and database components provides a maintainable foundation for future development.

5.1.1 Patient Use Cases

The patient interacts with the system to perform the following activities:

- Register an account
- Login securely
- View profile
- View doctors
- View doctor specializations
- Check doctor availability
- Book an appointment
- View appointments
- Update an appointment
- Cancel an appointment
- View appointment status
- View notifications

The patient is therefore the main user who uses the system to search for available doctors and manage appointments.

5.1.2 Doctor Use Cases

The doctor interacts with the system to perform activities related to doctor information and appointments.

Doctors are represented in the system through their professional information, specialization and available appointment slots. In the current implementation, this information is maintained by authorized administrators rather than through a separate doctor login or doctor dashboard. Patients can view doctor information and availability when selecting an appointment.

The major doctor functions include:

- Login
- Manage doctor profile
- Manage specialization information
- Manage availability
- View scheduled appointments
- Update appointment status
- View appointment-related notifications

These functions allow doctors to maintain their availability and manage appointments assigned to them.

5.1.3 Administrator Use Cases

The administrator has access to centralized management functionality.

The administrator can:

- Login
- Manage patient information

- Manage doctor information
- Manage doctor specializations
- Manage appointments
- Manage appointment status
- Manage notifications
- Monitor the overall system information

5.1.4 Use Case Representation

The overall interaction can be represented as:

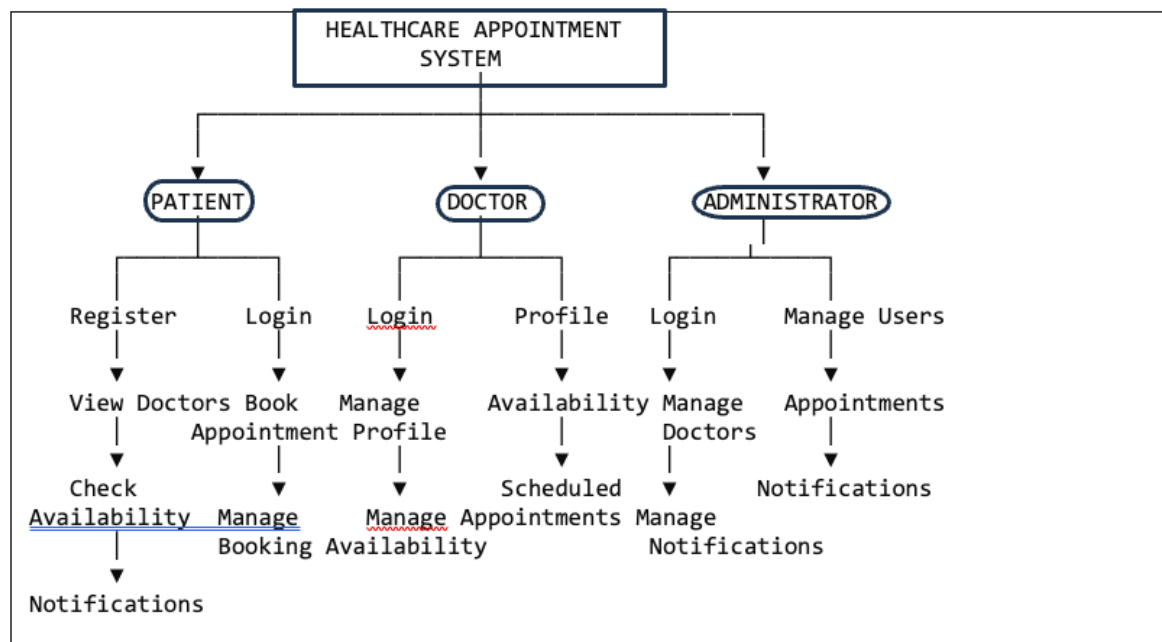


Figure 5.2: Use Case Representation of the Healthcare Appointment Scheduling System

The diagram should be inserted as an actual figure in the final Word document if you have a prepared use case diagram. Do not manually recreate the diagram using large amounts of spaces.

5.2 Entity Relationship (ER) Diagram

The Entity Relationship (ER) Diagram represents the major entities used by the healthcare appointment scheduling system and the relationships between them. The design represents patient information, doctor information, specialization, appointments and notifications. The relationships allow appointment records to associate patients with selected doctors and allow appointment-related information to be maintained.

The major entities include:

- Patient
- Doctor
- Specialization
- Appointment
- Notification
- User / Authentication information

The ER design helps organize the database and establishes relationships between patients,

doctors, appointments, and notifications.

5.2.1 Patient Entity

The Patient entity stores information related to registered patients.

Typical information includes:

- Patient ID
- Patient name
- Email
- Contact details
- Password information
- Profile information

A patient can have multiple appointments.

5.2.2 Doctor Entity

The Doctor entity stores information about healthcare providers.

Typical information includes:

- Doctor ID
- Doctor name
- Specialization
- Contact information
- Availability information

A doctor can have multiple appointments.

5.2.3 Specialization Entity

The Specialization entity represents the medical specialization associated with doctors.

Examples may include:

- Cardiology
- Dermatology
- General Medicine
- Orthopedics

The exact specializations depend on the data implemented in the project.

5.2.4 Appointment Entity

The Appointment entity connects patients and doctors.

It can contain:

- Appointment ID
- Patient ID
- Doctor ID
- Appointment date
- Appointment time
- Appointment status

An appointment belongs to a patient and is associated with a doctor.

5.2.5 Notification Entity

The Notification entity stores messages related to system activities and appointments.

It can contain:

- Notification ID
- User ID
- Notification message
- Notification date
- Notification status

Notifications allow users to receive important appointment-related information.

5.2.6 Entity Relationships

The major relationships can be represented as:

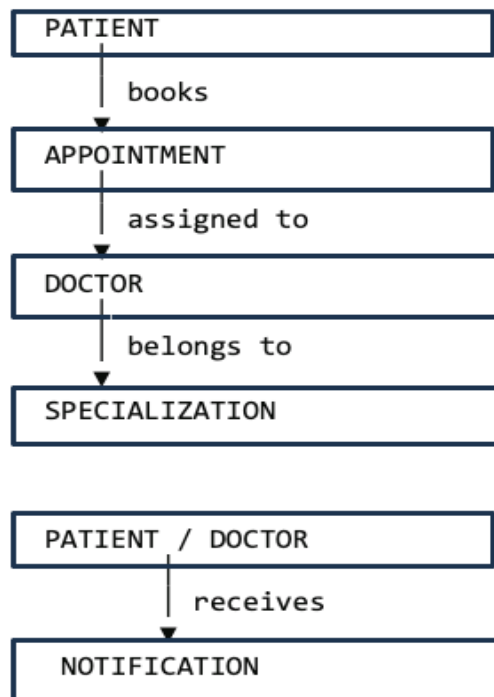


Figure 5.3: Major Entity Relationships

The relationship between patient, appointment, and doctor is particularly important because the appointment record connects the patient with the selected doctor.

5.3 Database Schema

The database schema defines how the application's information is organized into tables.

The main tables required by the system include:

1. Patient
2. Doctor
3. Specialization
4. Appointment
5. Notification
6. User / Authentication information, where applicable

The actual table and column names should match the database implemented in the project.

5.3.1 Patient Table

The Patient table stores registered patient information.

Attribute	Description
Patient ID	Unique identifier of the patient

Attribute	Description
Name	Patient name
Email	Patient email address
Contact	Patient contact information
Password	Authentication-related information
Profile Details	Additional patient information

5.3.2 Doctor Table

The Doctor table stores information about doctors.

Attribute	Description
Doctor ID	Unique identifier of the doctor
Name	Doctor name
Specialization ID	Associated medical specialization
Contact	Doctor contact information
Availability	Doctor availability information

5.3.3 Specialization Table

The Specialization table stores medical specialization information.

Attribute	Description
Specialization ID	Unique identifier
Specialization Name	Name of the specialization
Description	Description, if implemented

5.3.4 Appointment Table

The Appointment table stores appointment information.

Attribute	Description
Appointment ID	Unique appointment identifier
Patient ID	Patient associated with the appointment
Doctor ID	Doctor associated with the appointment
Appointment Date	Date of appointment
Appointment Time	Time of appointment
Status	Current appointment status

Possible appointment statuses may include:

- Booked
- Updated
- Cancelled
- Completed

The exact status values should match the values actually implemented in your database.

5.3.5 Notification Table

The Notification table stores notifications generated for users.

Attribute	Description
Notification ID	Unique notification identifier
User ID	User receiving the notification

Attribute	Description
Message	Notification content
Notification Date	Date/time of notification
Status	Notification status

5.4 Data Flow Diagram

The Data Flow Diagram represents how information moves through the healthcare appointment scheduling system.

The system receives information from patients, doctors, and administrators through the frontend. The frontend sends requests to the FastAPI backend through REST APIs. The backend processes the request and communicates with the MySQL database.

5.4.1 Level 0 Data Flow

At the highest level, the system can be represented as:

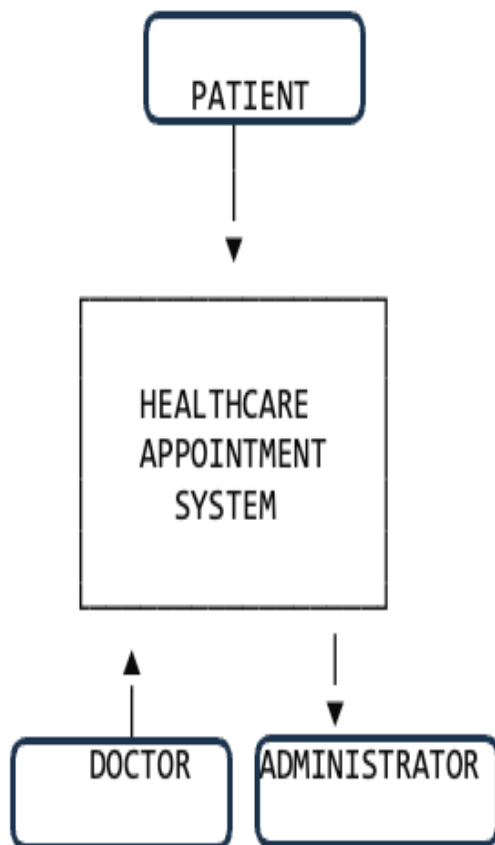


Figure 5.4: Level 0 Data Flow Diagram

The three primary external entities interact with the healthcare appointment scheduling system.

5.4.2 Level 1 Data Flow

The internal flow of the system can be represented as:

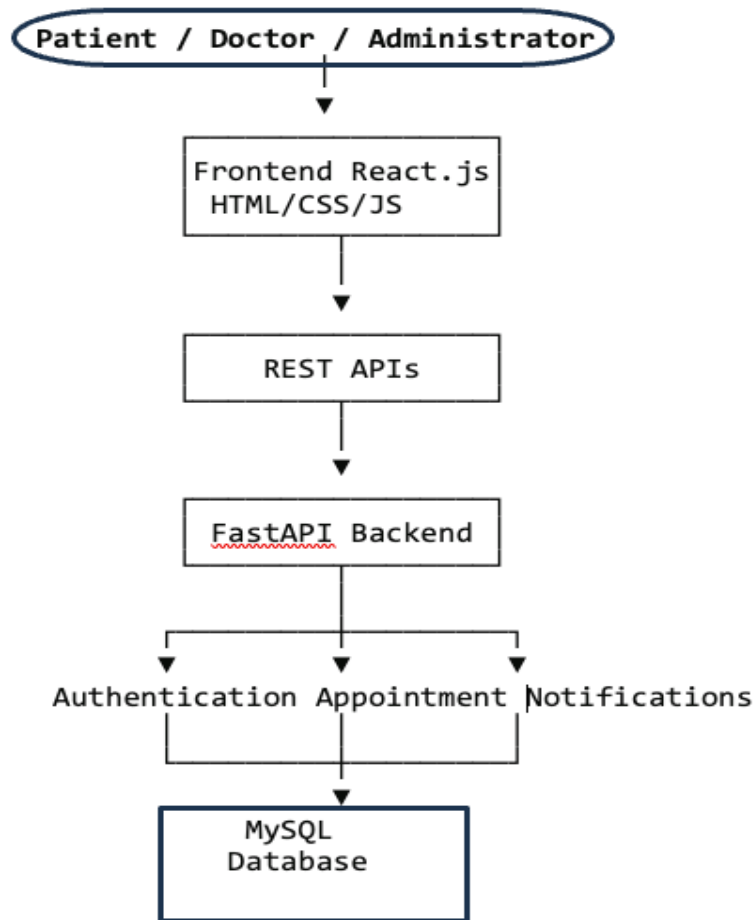


Figure 5.4: Level 1 Data Flow Diagram

The backend acts as the main processing layer between the frontend and database.

5.5 Module Design

The healthcare appointment scheduling system is divided into functional modules. Modular design makes the application easier to develop, test, maintain, and extend.

The major modules are:

1. Authentication and Patient Module
2. Doctor Module
3. Appointment Module
4. Notification Module
5. Administrator Module

5.5.1 Authentication and Patient Module

This module handles user authentication and patient-related functionality.

Main functions

- Patient registration
- Patient login
- JWT authentication
- Patient profile management
- Access to protected patient functionality

- Viewing appointments
- Viewing notifications

The authentication component verifies user credentials and provides the required authentication token for protected operations.

5.5.2 Doctor Module

The Doctor Module manages doctor-related information used by the appointment scheduling system. In the current implementation, doctor records and availability are maintained by authorized administrators.

Main functions include:

- Adding and updating doctor information
- Managing specialization information
- Managing doctor availability
- Displaying doctor information to patients
- Providing available appointment slots for booking

A separate doctor login and doctor-facing dashboard are not included in the current implementation.

5.5.3 Appointment Module

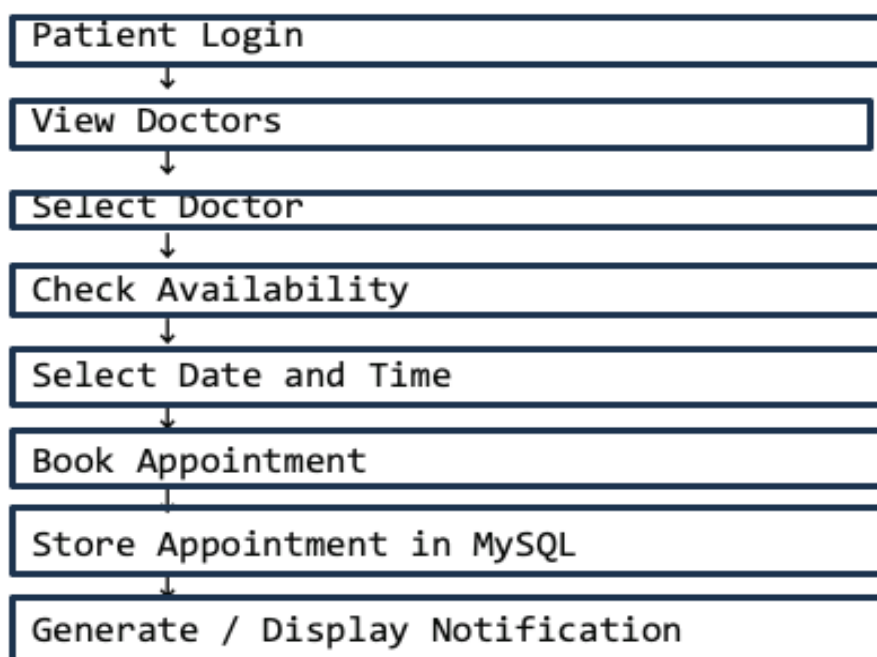
The Appointment Module is one of the main modules of the system. It handles the complete appointment management process.

Main functions

- View available doctors
- Check doctor availability
- Book appointments
- View appointments
- Update appointments
- Cancel appointments
- Manage appointment status

Associate appointments with patients and doctors

The basic appointment process is



:

Figure 5.5: Appointment Booking Workflow

The appointment module helps organize the relationship between patients and doctors.

5.5.4 Notification Module

The Notification Module handles appointment-related notifications.

Notifications may be generated for events such as:

- Appointment booking
- Appointment updates
- Appointment cancellation
- Appointment status changes

The notification information is stored and made available to the appropriate user through the application.

This module helps users remain informed about important appointment activities.

5.5.5 Administrator Module

The Administrator Module provides centralized management functionality.

The administrator can manage:

- Patient information
- Doctor information
- Appointments
- Notifications

Other administrative information supported by the application

The administrator module provides a centralized view of important system information.

5.6 System Workflow

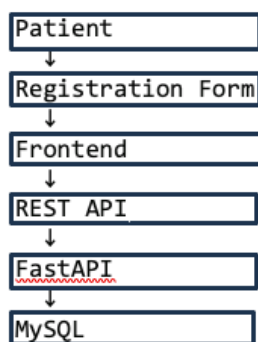
The overall workflow of the healthcare appointment scheduling system can be described in several stages.

5.6.1 User Registration

A new patient provides the required registration information through the frontend.

The frontend sends the information to the FastAPI backend.

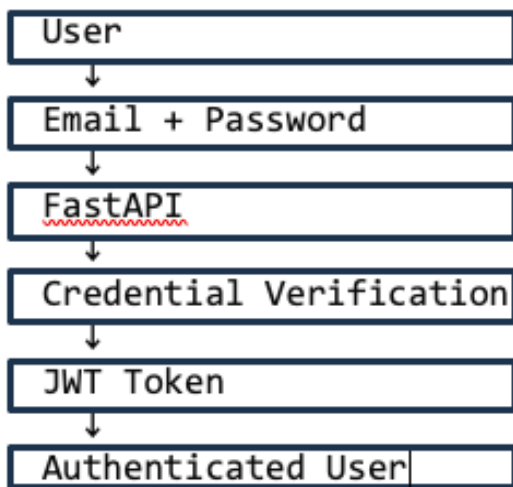
The backend validates the request and stores the required information in the MySQL database.



5.6.2 User Login

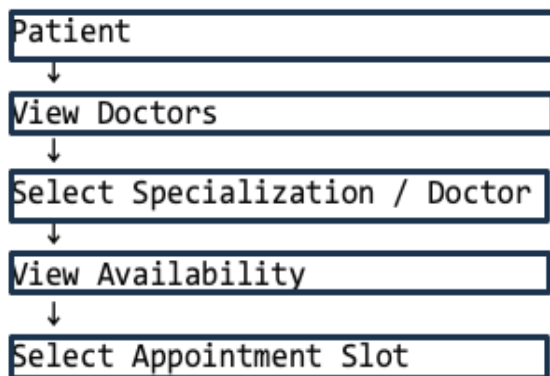
The registered user enters their login credentials.

The backend verifies the credentials. If authentication is successful, a JWT token is generated and provided to the user.



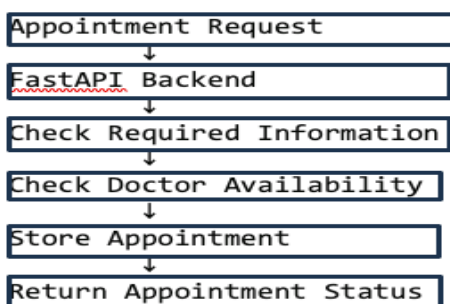
5.6.3 Doctor Selection

After authentication, the patient can view available doctors and their specializations. The patient selects a suitable doctor and checks the available appointment slots.



5.6.4 Appointment Booking

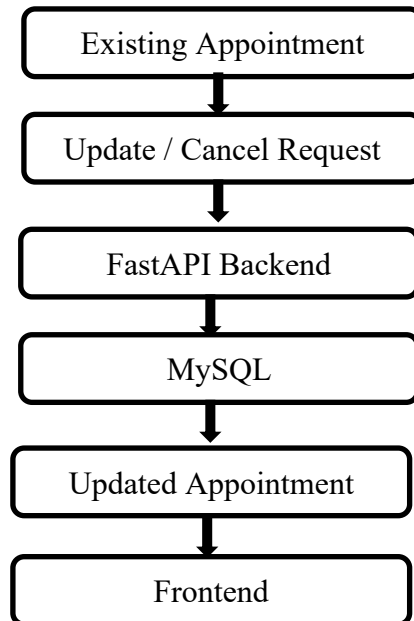
After selecting an available slot, the patient submits the appointment request. The backend processes the request and stores the appointment information in MySQL.



5.6.5 Appointment Update and Cancellation

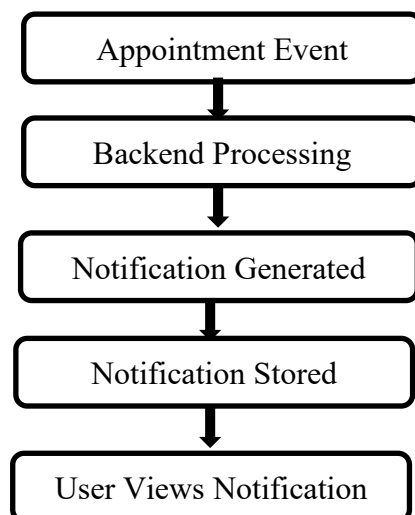
A patient can access their appointment information and perform supported update or cancellation operations.

The request is sent through the REST API to the backend, where the appointment information is updated in the database.



5.6.6 Notification Workflow

When an appointment-related event occurs, the system can generate or display an appropriate notification.



5.6.7 Overall System Workflow

The complete workflow can be summarized as:

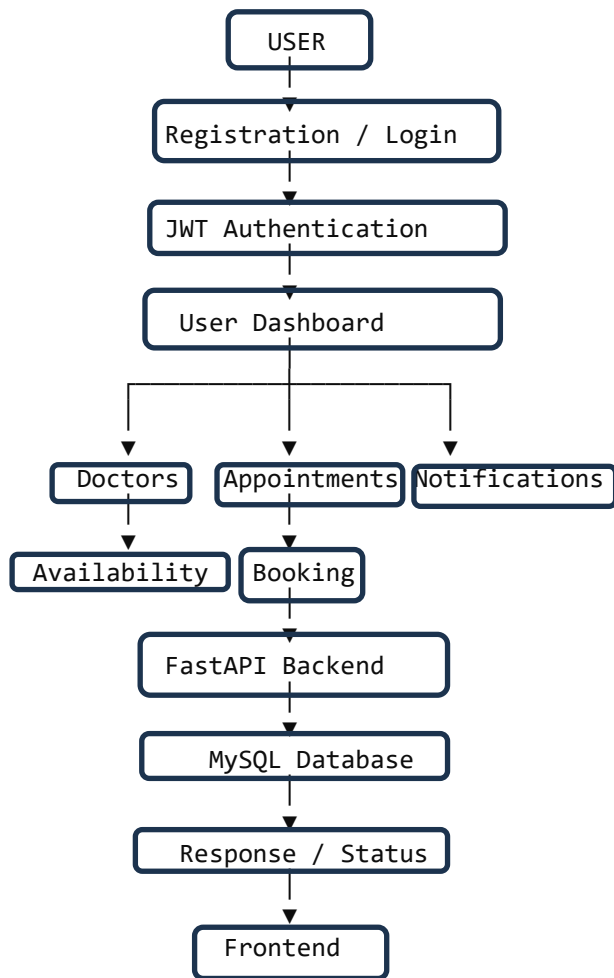


Figure 5.6: Overall System Workflow

5.7 Summary

The system design defines the structure and interaction of the healthcare appointment scheduling application. The design includes the major users, system use cases, database entities, data flow, functional modules, and application workflow.

The Patient, Doctor, and Administrator are the primary users of the system. The frontend provides the user interface, while REST APIs connect the frontend with the FastAPI backend. The backend performs business operations and communicates with the MySQL database. JWT provides authentication and authorization for protected functionality.

The system is divided into authentication and patient, doctor, appointment, notification, and administrator modules. This modular design provides a structured approach for implementing the healthcare appointment scheduling system and supports easier development, testing, maintenance, and future enhancement.

CHAPTER -6 IMPLEMENTATION

6.1 – Module-Wise Implementation

The implementation of the **Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development** system was systematically structured into interconnected modules that together form a unified healthcare appointment management platform. Each module was developed with a specific functional responsibility while maintaining communication with the other modules through a common backend architecture and database. This modular approach improves maintainability, simplifies development, and allows individual components to be modified without affecting the complete system.

The healthcare appointment scheduling platform is organized into interconnected functional modules that together support the complete appointment management workflow. The major implementation areas include authentication and patient management, doctor information and appointment management, notification handling, and administrator management.

The frontend is implemented using **React.js, HTML, CSS, JavaScript, and Vite**. It provides the user interface for patients and administrators. Doctor information and availability are maintained through the administrator functionality rather than through a separate doctor portal. The backend is implemented using **Python with FastAPI** and provides REST API services for authentication, doctor information, appointment management, notifications, and administrative operations. **MySQL** provides persistent data storage, while **JWT-based authentication and role-based authorization** protect authenticated and administrative operations.

These modules collectively support the complete appointment lifecycle, beginning with user registration and authentication and continuing through doctor discovery, appointment booking, appointment management, notifications, and administrative supervision.

The frontend provides the interaction layer through which patients, doctors, and administrators access the system. HTML, CSS, and JavaScript are used to create the user interface and handle user interactions.

The backend is implemented using Python FastAPI, which provides RESTful API endpoints for processing requests, validating data, performing business logic, and communicating with the database. MySQL serves as the central database for storing structured information related to users, doctors, appointments, availability, and notifications.

The implementation follows a layered structure in which the presentation layer communicates with the backend API, while the backend communicates with the database through structured operations. This separation reduces dependency between components and provides a clear development structure. Authentication is implemented using JSON Web Tokens, allowing protected resources to be accessed only by authenticated users with the appropriate permissions.

6.1.1 Authentication and Patient Module

The Authentication and Patient Module is responsible for managing patient registration, login, authentication, and patient-related operations. Since healthcare applications handle personal information, secure access to the platform is an important requirement. The module ensures that only registered users can access protected services and that user information is handled according to the defined access rules.

During registration, a patient provides the required information through the registration interface. The frontend collects the input and sends it to the FastAPI backend through an appropriate REST API request. The backend validates the submitted information before storing it in the MySQL database. Validation helps prevent incomplete or incorrect records from entering the system.

After registration, the patient can log in using the registered credentials. The backend verifies the credentials against the stored user information. Once authentication is successful, a JWT token is generated and returned to the client. The token is used in subsequent requests to access protected resources. This approach avoids relying on a traditional server-side session for every request and provides a structured mechanism for authentication.

The patient module also provides access to patient-specific information. After successful login, the patient can view available doctors, search according to specialization, check doctor availability, and select an appropriate appointment slot. The patient can also view previously created appointments and perform permitted operations such as updating or cancelling an appointment.

Role-based access is maintained so that patient operations remain separate from administrative operations. A patient should not be able to access administrative functions such as deleting doctor records or managing all appointments. The backend therefore verifies the authenticated user's role before processing protected requests.

The module also maintains appropriate responses for invalid input, incorrect login credentials, missing authentication tokens, and unauthorized access. These responses help users understand the reason for unsuccessful operations while providing developers with meaningful API responses during testing.

From a frontend perspective, the patient module includes registration, login, patient dashboard, doctor search, appointment booking, appointment details, and notification views. JavaScript handles form submission and communication with the backend APIs. The interface provides feedback after important operations so that users can understand whether registration, login, booking, updating, or cancellation has been completed successfully.

The patient module therefore acts as the primary entry point into the healthcare appointment scheduling system. It combines authentication, patient information, doctor discovery, and appointment interaction into a single user-oriented workflow.

6.1.2 Doctor and Appointment Module

The Doctor and Appointment Module manages doctor information, specialization, availability, and appointment-related operations. In the implemented system, doctor information and availability are managed through the administrator interface. Patients use this information to

identify suitable doctors and available appointment slots.

Doctor records contain relevant information such as doctor name, specialization, contact information, professional details, and availability. These records are maintained in the database and retrieved through backend API services when patients browse or search for doctors.

The appointment process allows an authenticated patient to select a doctor, view available appointment slots, and submit an appointment request. The backend validates the request and checks the selected appointment information before storing the appointment in the database.

The doctor search process begins when a patient selects or enters a required specialization. The frontend sends the search request to the backend, where the corresponding doctor information is retrieved from MySQL. The backend returns structured data to the frontend, allowing the available doctors to be displayed in an organized format.

Doctor availability is an important component of the appointment process. The system maintains information regarding available appointment slots so that patients can select a suitable time. Before confirming an appointment, the backend checks the selected doctor, date, and time against the available appointment information.

The appointment booking operation follows a controlled sequence. First, the authenticated patient selects a doctor. Next, the patient views the available appointment slots. After selecting a suitable slot, the booking request is submitted to the backend. The backend validates the request and checks whether the selected slot is still available. If the required conditions are satisfied, an appointment record is created in the database.

The appointment record connects the patient and doctor through their respective identifiers. It also stores information such as appointment date, appointment time, and appointment status. This relational approach allows appointments to be retrieved efficiently and provides a clear connection between patients, doctors, and scheduled visits.

Appointment status is used to represent the current state of an appointment. Depending on the operation performed, an appointment may be active, updated, cancelled, or otherwise maintained according to the application's workflow. Status information helps users and administrators understand the current state of each appointment without deleting important historical records unnecessarily.

The update operation allows an authenticated patient to modify permitted appointment details. Before applying an update, the backend verifies that the appointment belongs to the requesting patient and that the requested changes are valid. This prevents unauthorized modification of another patient's appointment.

Cancellation is handled in a similar controlled manner. When a patient requests cancellation, the backend verifies the appointment and updates its status accordingly. The system can then make the corresponding slot available according to the defined scheduling logic.

The appointment module is also responsible for preventing basic scheduling conflicts. A doctor should not have multiple active appointments assigned to the same time slot. Therefore,

appointment creation includes a validation step that checks existing records before confirming a new booking.

From the user perspective, the module provides a simple workflow: search for a doctor, view specialization and availability, select an available slot, confirm the appointment, and receive confirmation. This workflow reduces unnecessary manual coordination and provides a structured digital scheduling process.

From the technical perspective, the module demonstrates the importance of communication between the frontend, backend, and database. JavaScript sends requests to FastAPI, FastAPI performs validation and business logic, and MySQL stores the resulting appointment information. This layered communication forms an important part of the distributed backend structure.

6.1.3 Notification and Admin Module

The Notification and Admin Module provides administrative control over the healthcare appointment platform while also managing communication related to appointment activities. The administrator has a broader level of access than normal patients and can manage important records required for system operation.

The administrative component allows authorized administrators to view and manage patient records, doctor information, appointments, and notification-related information. Access to these operations is protected through authentication and role-based authorization.

The administrator can review registered patients and doctors through the administrative interface. Doctor records can be maintained with relevant information such as specialization and availability. Appointment records can also be reviewed to understand the current scheduling status.

Notifications are generated for important appointment-related events. For example, when an appointment is successfully created, a confirmation notification can be associated with the patient. When an appointment is updated or cancelled, the corresponding notification information can be recorded. Reminder-related notifications can also be incorporated into the workflow to help patients remember upcoming appointments.

The notification system is connected with the appointment module. When a significant appointment event occurs, the backend processes the corresponding notification operation. This separation allows appointment processing and notification handling to remain logically organized while still working together.

The administrator can also review notification records when necessary. This provides better visibility into system communication and helps maintain a clear history of important appointment-related events.

The administrative interface is designed to provide a centralized view of the system. Instead of managing individual records manually through the database, administrators can interact with structured screens that communicate with backend APIs. This improves usability and reduces the possibility of direct database manipulation by non-technical users.

Security is particularly important in this module because administrative operations can affect multiple users. The backend verifies the JWT token and role information before allowing administrative requests. Unauthorized users attempting to access protected administrative endpoints receive an appropriate authorization response.

Together, the Authentication and Patient Module, Doctor and Appointment Module, and Notification and Admin Module provide the primary implementation structure of the healthcare appointment scheduling platform. Their separation into functional components makes the system easier to understand, develop, test, and maintain.

6.2 Front-End Logic and User Interface Implementation

The frontend of the healthcare appointment scheduling system is implemented using **React.js, HTML, CSS, and JavaScript**, with **Vite** used as the frontend development and build tool. The frontend acts as the primary interaction layer between users and the FastAPI backend.

React components are used to organize the user interface into reusable sections. The application provides interfaces for patient registration, login, doctor browsing, appointment scheduling, appointment management, notifications, and administrative operations.

The frontend communicates with the FastAPI backend through REST APIs. User actions such as registration, login, doctor search, appointment booking, updating, cancellation, and notification retrieval result in API requests to the backend.

The frontend includes screens for registration, login, doctor browsing, appointment scheduling, appointment management, notifications, and administrative operations. Each screen is connected to the relevant backend API through JavaScript-based requests.

The frontend includes screens for registration, login, doctor browsing, appointment scheduling, appointment management, notifications, and administrative operations. Each screen is connected to the relevant backend API through JavaScript-based requests. The user can move to the login interface. Following authentication, the patient is directed to the relevant dashboard where available healthcare services can be accessed.

Form handling is an important part of frontend implementation. Registration and login forms collect user input and perform basic validation before sending requests to the backend. Required fields are checked to reduce incomplete submissions. Appointment forms similarly validate selections such as doctor, date, and available time slot.

The frontend communicates asynchronously with the FastAPI backend. When a user searches for a doctor, the browser sends a request to the relevant API and waits for the response without unnecessarily refreshing the entire page. The returned information is then displayed dynamically.

Doctor information can be presented using organized cards or structured sections containing doctor name, specialization, contact information, and availability. This allows patients to compare available options before selecting an appointment.

The appointment booking interface is designed to reduce confusion by presenting only relevant information at each stage. After selecting a doctor, the user can view available appointment slots. Once a slot is selected, the patient can confirm the booking. The frontend then displays the result returned by the backend.

Appointment management also follows the same interaction pattern. Existing appointments are retrieved from the backend and displayed in the patient dashboard. Update and cancellation actions are made available only when permitted by the application workflow.

Real-time validation and contextual feedback are used to improve the user experience. For example, missing fields can be highlighted before a request is submitted. If the backend reports that a selected appointment slot is unavailable, the frontend can display an appropriate message instead of treating the operation as successful.

The interface also maintains clear separation between different user roles. Patients are provided with patient-related functionality, while administrative users can access management screens. This frontend separation complements the backend authorization mechanism.

Error handling is another important part of frontend logic. Network errors, invalid responses, failed authentication, and unsuccessful appointment operations are communicated to users through understandable messages. This reduces confusion and provides a more reliable interaction experience.

The frontend therefore serves more than a visual purpose. It acts as an interaction controller that collects user input, communicates with backend services, processes responses, and updates the interface according to the current state of the system.

6.3 API Integration

The API integration layer forms the communication backbone of the Secure and Scalable Healthcare Appointment Scheduling system. It connects the frontend interface with the FastAPI backend and enables structured communication between users, application logic, and the MySQL database. The API layer provides a clear separation between the user interface and the internal processing of healthcare appointment operations.

The backend is implemented using Python FastAPI, which provides a framework for developing RESTful application programming interfaces. REST principles are used to organize different resources and operations through HTTP requests. Separate API endpoints are created for authentication, patient management, doctor management, appointment scheduling, notifications, and administrative functions.

The authentication APIs are responsible for handling patient registration and login operations. During registration, the information submitted through the frontend is received by the backend and validated before being stored in the database. During login, the backend verifies the submitted credentials and provides an authentication token when the credentials are valid. Patient-related APIs are used to retrieve and manage patient-specific information. After successful authentication, the patient can access permitted information through protected endpoints. These endpoints can also be used to retrieve the patient's appointment records and relevant notifications.

Doctor-related APIs provide information about registered doctors, their specializations, contact

details, and availability. When a patient searches for a doctor, the frontend sends the request to the appropriate FastAPI endpoint. The backend processes the request and retrieves the required doctor information from MySQL before returning the response.

Appointment APIs are responsible for the main scheduling operations. These operations include creating an appointment, viewing appointment details, updating appointment information, and cancelling an appointment. Each operation is processed through a specific backend endpoint so that the frontend does not directly communicate with the database.

HTTP methods are selected according to the purpose of each API operation. GET requests are used for retrieving information, POST requests are used for creating new records, PUT requests are used for modifying existing records, and DELETE requests can be used when an operation requires removal of a record. In appointment management, cancellation can also be implemented by changing the appointment status rather than permanently deleting the record.

The use of structured API endpoints makes communication between different components easier to manage. For example, a patient booking an appointment follows a sequence in which the frontend collects the selected doctor and appointment information, sends the data to the backend, and receives a response indicating whether the booking was successful.

The API layer also performs input validation. Data received from the frontend is checked before any database operation is performed. This is important because frontend validation alone cannot guarantee that every request received by the backend is valid. Backend validation therefore acts as an additional layer of protection.

Appointment validation requires additional processing because an appointment is related to a specific patient, doctor, date, and time. Before creating an appointment, the backend can verify that the selected doctor exists and that the requested appointment slot is available. This helps reduce scheduling conflicts.

The API also verifies authentication for protected operations. A patient must be authenticated before accessing personal appointment information or performing operations on their appointments. Administrative endpoints require the appropriate administrative role.

Responses are returned to the frontend in a structured format. Successful operations provide the relevant information or confirmation message, while unsuccessful operations provide an appropriate response indicating the reason for failure. HTTP status codes can be used to distinguish successful requests from invalid, unauthorized, or forbidden requests.

For example, an unauthenticated request to a protected appointment endpoint should not be processed as a normal request. Similarly, a patient attempting to access an administrative endpoint should not receive administrative information. These checks are performed by the backend before the requested operation is completed.

Postman is used during development to test the REST APIs independently from the frontend. This allows individual endpoints to be verified before complete frontend integration. Registration, login, doctor information, appointment operations, and notification-related endpoints can be tested using appropriate request methods and sample data.

API testing also helps identify incorrect endpoint paths, request formats, missing parameters, authentication problems, and unexpected responses. Testing the backend independently makes it easier to locate the source of a problem before integrating the complete application.

The API architecture also supports maintainability. Since the frontend communicates with the backend through defined endpoints, changes to the internal database logic do not necessarily require changes to the user interface. Similarly, frontend components can be redesigned while continuing to use the same backend API structure.

The API integration therefore establishes a clear communication boundary between the frontend, backend, and database. It supports authentication, doctor discovery, appointment scheduling, notification management, and administrative operations while maintaining a modular structure.

6.4 Database Integration

MySQL is used as the primary database for the healthcare appointment scheduling system. The database provides persistent storage for the information required by the application, including user details, patient records, doctor information, doctor availability, appointments, and notifications.

The database is organized using related tables so that different types of information can be stored separately while maintaining relationships between them. This structure supports the relational database requirements of the project and demonstrates important DBMS concepts such as primary keys, foreign keys, relationships, and structured SQL operations.

Patient information is stored in the appropriate database structure so that registered users can be identified and their appointment information can be associated with their records. Doctor information is maintained separately because doctors have different attributes such as specialization, contact details, and availability.

Each important table contains a primary key that uniquely identifies its records. For example, a patient record can be identified using a patient identifier, while a doctor record can have its own doctor identifier. These identifiers are used when connecting information across different tables. Foreign keys are used to establish relationships between related records. An appointment can contain references to both the patient and doctor associated with that appointment. This creates a relationship between patients, doctors, and appointments without unnecessarily repeating complete patient or doctor information inside every appointment record.

Doctor availability is also maintained as structured information. Availability records can be associated with a particular doctor and can contain information about the available date or time period. This information is used during appointment scheduling to determine whether a requested slot can be selected.

Appointment records store the information necessary to represent a scheduled healthcare appointment. Important information can include appointment identifier, patient identifier, doctor identifier, appointment date, appointment time, and appointment status.

The appointment status helps the system identify the current condition of an appointment. For example, an appointment can be represented as active, updated, or cancelled depending on the operation performed. Maintaining status information provides a better record of appointment activity than simply removing every cancelled record.

Notification records are also stored in the database. These records can be associated with a user and an appointment so that the system can identify which notification belongs to which user and event.

The FastAPI backend acts as the intermediary between the application and the MySQL database. The frontend does not directly connect to the database. Instead, the frontend sends a request to the backend, the backend validates and processes the request, and the required database operation is performed by the server.

For example, when a patient books an appointment, the frontend sends the selected appointment information to the FastAPI backend. The backend checks the request and performs the necessary database operations. If the booking is valid, an appointment record is created and the result is returned to the frontend.

Database queries are also used when retrieving doctor information. When a patient searches for a specific specialization, the backend can query the doctor-related table and return matching records. This allows the frontend to display only the relevant doctors.

Database integration is also important when retrieving a patient's appointments. The backend can use the authenticated patient's identifier to retrieve the appointments associated with that patient. This prevents one patient from automatically receiving another patient's appointment information.

The same principle applies to administrative operations. Administrative requests can retrieve broader information because administrators have the required permissions. However, the backend should still control these operations through authentication and authorization.

Database relationships help maintain data consistency. Instead of storing duplicate doctor information inside every appointment, the appointment record can refer to the doctor through the doctor identifier. If doctor information changes, it can be updated in the doctor record without requiring unnecessary changes to every appointment record.

The relational structure also supports SQL operations such as inserting new records, retrieving existing records, updating information, and joining related tables. These operations demonstrate the practical use of DBMS concepts within the healthcare application.

Database connectivity must also be configured correctly in the backend. The application requires the correct database name, username, password, host, and connection configuration. Incorrect configuration can prevent the backend from retrieving or storing application information.

Error handling is important during database operations. If a database operation cannot be completed, the backend should return an appropriate response instead of incorrectly informing the user that the operation was successful.

Appointment conflict checking is another important database operation. Before creating a new appointment, the backend can check whether an active appointment already exists for the same doctor, date, and time. If the selected slot is already occupied, the booking can be rejected.

The database therefore forms the persistent storage layer of the system. Its structured design allows the healthcare appointment scheduling platform to manage patients, doctors, appointments, availability, and notifications in an organized manner.

6.5 Authentication and Authorization Implementation

Authentication and authorization are implemented to protect the healthcare appointment scheduling system from unauthorized access. Authentication identifies the user who is accessing the application, while authorization determines which operations that authenticated user is permitted to perform.

The system uses JSON Web Token-based authentication for protected API access. When a registered user submits valid login credentials, the FastAPI backend verifies the credentials and generates an authentication token.

The generated JWT token is returned to the client after successful authentication. The client can then include the token when making requests to protected backend endpoints.

When a protected request reaches the backend, the authentication mechanism checks the submitted token. If the token is valid, the backend can identify the authenticated user and continue processing the request. If the token is missing or invalid, the protected operation is rejected.

This approach provides a stateless authentication mechanism in which the backend can verify the identity associated with a request without depending on a traditional session for every operation. Authentication begins with user registration. A new patient provides the required information through the registration interface. The information is validated and stored in the database. The patient can then use the registered credentials to log in.

During login, the backend compares the submitted credentials with the stored authentication information. If the credentials are correct, authentication succeeds and a JWT token is generated. If the credentials are incorrect, the login operation is rejected.

Authorization is implemented using user roles. The healthcare appointment system contains different categories of users, such as patients, doctors, and administrators. Each role has a different set of permitted operations.

Patients are allowed to perform patient-related activities such as viewing doctors, checking availability, creating appointments, viewing their appointments, and updating or cancelling permitted appointments.

Doctors can access functions related to their professional information, availability, and appointment information according to the system's defined permissions.

Administrators have broader access to system management operations. They can manage important patient and doctor information, review appointments, and handle administrative functions.

Role-based authorization is enforced at the backend level. This is important because hiding a button or page on the frontend is not enough to secure an operation. A user could attempt to directly call an API endpoint without using the visible interface.

The backend therefore checks the user's role before allowing access to restricted resources. If a patient attempts to access an administrative operation, the backend rejects the request. Authentication and authorization also protect patient-specific information. A patient should only be able to retrieve or modify appointments associated with their own account. The backend can use the authenticated user identity when performing these operations.

Appropriate HTTP responses are returned when authentication or authorization fails. An unauthenticated request can result in an unauthorized response, while an authenticated user who does not have the required role can receive a forbidden response.

Token-based authentication also supports communication between the frontend and backend through REST APIs. The frontend does not need to send the user's password with every request. Instead, the authentication token is used to identify the authenticated session.

The authentication system contributes significantly to the security of the healthcare appointment scheduling platform. It provides a structured mechanism for verifying users, protecting API endpoints, and controlling access to different system functions.

The implemented application primarily provides authenticated access for **patients and administrators**. Doctor information is managed by authorized administrators and is presented to patients as part of the appointment scheduling workflow. There is no separate doctor login or doctor dashboard in the submitted implementation.

6.6 Notification Implementation

The notification module is implemented to provide appointment-related information to users. Notifications help patients remain informed about important changes to their appointments and provide a structured communication mechanism within the healthcare scheduling system.

The notification process is connected with the appointment management workflow. When an important appointment event occurs, the backend can create an associated notification record.

For example, after a successful appointment booking, the system can create a confirmation notification for the patient. This allows the patient to verify that the appointment has been successfully recorded.

When an appointment is updated, the system can create an update notification. Similarly, when an appointment is cancelled, the cancellation can be reflected through a notification.

Reminder notifications can also be included in the system to inform users about upcoming appointments. These reminders can contain relevant information such as the doctor name, appointment date, and appointment time.

Notification records are stored in the MySQL database. Each notification can be associated with the relevant user and, where required, the corresponding appointment. This relationship allows the system to retrieve notifications for the correct user.

The notification interface displays the available messages to the authenticated user. Patients can therefore review important appointment-related information without having to repeatedly search through the appointment section.

The backend controls notification creation so that notifications are generated according to defined application events. This prevents the frontend from directly creating or modifying notification records in the database.

Notification handling is separated from the core appointment logic while remaining connected to it. This modular approach allows the notification functionality to be extended in the future without requiring major changes to the appointment module.

The administrator can also access notification-related information through the administrative interface when required. This provides additional visibility into appointment communication.

The current notification implementation primarily focuses on application-level appointment notifications. External communication channels such as SMS or email can be considered as future enhancements if additional services and configurations are introduced.

Overall, the notification module improves the communication flow of the healthcare appointment scheduling system. It ensures that important appointment events can be communicated to users in an organized manner.

The implementation of Sections 6.3 to 6.6 demonstrates the integration of the major backend components of the system. FastAPI provides the REST API layer, MySQL provides persistent data storage, JWT supports authentication, role-based authorization controls access, and the notification module provides appointment-related communication. Together, these components support the secure and structured operation of the healthcare appointment scheduling platform.

CHAPTER 7- FEATURES

7.1 List of Main Features

The Secure and Scalable Healthcare Appointment Scheduling system provides a set of features designed to simplify appointment management for patients, doctors, and administrators. The major features of the system include:

1. User Registration and Login
2. Patient Management
3. Doctor Management
4. Doctor Availability
5. Appointment Scheduling
6. Appointment Management
7. Notification Management
8. Admin Management
9. Security and Role-Based Authorization
10. Database Integration and Data Management
11. API-Based System Communication

These features work together to provide a structured healthcare appointment scheduling platform.

7.2How Each Feature Works

1. User Registration and Login Feature

User Perspective:

The registration and login feature allows patients to create an account and securely access the healthcare appointment scheduling system. During registration, the patient provides the required personal and login information. After successful registration, the patient can use the registered credentials to log in to the system.

The login interface provides a simple method for entering the required credentials. After successful authentication, the patient is directed to the appropriate part of the application. This allows authenticated users to access appointment-related services and their personal information.

The registration and login interfaces are implemented using React.js. The frontend communicates with the FastAPI backend through REST APIs, while JWT is used for authenticated access to protected resources.

Technical Description:

The authentication functionality is implemented using the FastAPI backend. Registration requests are sent from the frontend to the backend through REST APIs. The backend validates the submitted information before storing the required user information in the MySQL database. During login, the backend verifies the submitted credentials. After successful authentication, a JSON Web Token (JWT) is generated and provided to the authenticated user. The token is then used when accessing protected API endpoints.

Authentication prevents unauthorized users from accessing protected resources. The backend also performs authorization checks to ensure that users can access only the functions permitted for their roles.

2. Patient Management Feature

User Perspective:

The patient management feature allows registered patients to manage their interaction with the healthcare appointment system. Patients can access their account information and view their appointment-related details after logging in.

Patients can browse available doctors, check their specialization and availability, and select an appropriate doctor for an appointment. They can also view their previously scheduled appointments and their current appointment status.

The patient interface is designed to keep important information organized so that users can easily navigate between doctor information, appointments, and notifications.

Technical Description:

Patient-related information is stored in the MySQL database and accessed through FastAPI endpoints. The backend identifies the authenticated patient using the authentication information associated with the request.

Protected APIs ensure that a patient can access only the information associated with their account. Database queries are used to retrieve patient details and appointment records.

The patient module communicates with the doctor, appointment, and notification modules through the backend API layer. This modular approach allows patient functionality to operate independently while remaining connected to other parts of the application.

3. Doctor Management Feature

User Perspective:

The doctor management feature allows patients to browse registered doctors and view important information before selecting an appointment. Doctor profiles can contain details such as doctor name, specialization, contact information, and availability.

Patients can use the available doctor information to identify a doctor based on their healthcare requirement. The system organizes doctor information so that users can easily understand the

available options.

Doctors can also have their professional information maintained through the system according to the permissions provided to their role.

Technical Description:

Doctor information is maintained in the MySQL database. Each doctor is represented using a unique identifier, allowing doctor information to be connected with appointments and availability records.

The FastAPI backend provides APIs for retrieving doctor information. When a patient requests doctor details, the backend processes the request and retrieves the required information from the database.

Doctor information can also be filtered according to available attributes such as specialization. This allows the frontend to display relevant doctors to the patient.

The use of a separate doctor module prevents unnecessary duplication of doctor information across the system and maintains a clear relationship between doctors and appointments.

4. Doctor Availability Feature

User Perspective:

The doctor availability feature allows patients to identify suitable dates and time slots before booking an appointment. Availability information helps patients select a time that is suitable for both the patient and the doctor.

The system displays available appointment options so that users do not have to depend on manual communication to determine whether a doctor is available.

This feature improves the appointment selection process by providing availability information as part of the booking workflow.

Technical Description:

Doctor availability is maintained as structured data in the backend and database. Availability records are associated with the corresponding doctor using a unique doctor identifier.

When a patient selects a doctor, the frontend requests the available appointment information from the FastAPI backend. The backend retrieves the relevant availability data and returns it to the frontend.

Before confirming an appointment, the backend can check whether the selected date and time are still available. This validation helps reduce appointment conflicts.

The availability feature is closely connected with the appointment module because availability information directly affects whether an appointment can be created.

5. Appointment Scheduling Feature

User Perspective:

Appointment scheduling is one of the main features of the healthcare system. Patients can select a doctor, choose an available date and time, and submit the appointment request through the application.

The system provides a structured booking process instead of requiring patients to call or visit a healthcare facility to request an appointment.

After successful booking, the appointment details can be displayed to the patient. These details may include the selected doctor, appointment date, appointment time, and appointment status.

Technical Description:

The appointment scheduling process is handled by the FastAPI backend and MySQL database. When a patient submits an appointment request, the frontend sends the required information to the appropriate API endpoint.

The backend validates the request before creating the appointment record. It verifies that the patient and doctor information are valid and checks the availability of the selected time slot. If the requested slot is available, a new appointment record is stored in the database. The appointment is associated with both the patient and doctor through their respective identifiers. If the selected slot is not available, the backend rejects the booking request and returns an appropriate response to the frontend.

This validation process helps prevent duplicate or conflicting appointment bookings.

6.Appointment Management Feature

User Perspective:

The appointment management feature allows patients to view and manage their scheduled appointments. Patients can review appointment information and perform permitted actions such as updating or cancelling an appointment.

The appointment status helps users understand the current condition of their booking. This provides a clear record of appointment activity.

Doctors and administrators can also access appointment information according to their respective roles and permissions.

Technical Description:

Appointment records are stored in the MySQL database and linked to patient and doctor records through foreign keys.

The FastAPI backend provides endpoints for retrieving appointment information and performing permitted appointment operations.

When a patient requests their appointments, the backend identifies the authenticated patient and retrieves only the records associated with that patient.

For appointment updates, the backend validates the new information before modifying the existing record. During cancellation, the appointment status can be updated to indicate that the appointment is no longer active.

Maintaining appointment status instead of immediately deleting appointment records provides a more organized way to track appointment activity.

7.Notification Management Feature

User Perspective:

The notification feature keeps users informed about important appointment events. Patients can receive notifications related to successful bookings, appointment updates, cancellations, and reminders.

For example, after successfully scheduling an appointment, the system can provide a confirmation notification. If an appointment is changed or cancelled, the user can receive a corresponding notification.

The notification feature reduces the need for users to repeatedly check their appointment information and helps them remain aware of important updates.

Technical Description:

Notifications are generated by the backend when specific appointment-related events occur. The notification information is stored in the MySQL database.

Each notification can be associated with the relevant user and appointment. This relationship allows the backend to retrieve notifications for the correct user.

The frontend requests notification information through the FastAPI API and displays the available messages to the authenticated user.

The current implementation focuses on application-level notifications. External services such as SMS or email notification systems can be considered as future enhancements.

8. Admin Management Feature**User Perspective:**

The administrator module provides authorized administrators with tools for managing important parts of the healthcare appointment system.

Administrators can monitor patient information, doctor information, appointment records, and notification-related information according to the functions implemented in the system.

The administrative interface provides a centralized way to manage the application rather than requiring separate manual records.

Technical Description:

Administrative operations are protected through authentication and role-based authorization. Only users with the appropriate administrative role can access restricted administrative endpoints.

The FastAPI backend verifies the user's authentication token and role before processing an administrative request.

The administrator can perform permitted database operations such as retrieving, updating, or managing relevant records. These operations are performed through backend APIs rather than allowing direct access to the database from the frontend.

Role-based access control ensures that patients and other unauthorized users cannot access administrative functions.

9. Security and Role-Based Authorization Feature**User Perspective:**

Security is integrated throughout the healthcare appointment scheduling system. Users are required to authenticate before accessing protected features.

Different users receive access according to their roles. Patients, doctors, and administrators do not have identical permissions because their responsibilities within the system are different.

This approach helps protect personal information and prevents users from accessing functions that are not intended for their role.

Technical Description:

JWT-based authentication is used to verify authenticated users. After successful login, the generated token is used to access protected API endpoints.

Role-based authorization is implemented at the backend level. Before processing a protected request, the system verifies the user's identity and role.

For example, a patient can access their appointment information but should not be able to access administrative operations. Similarly, an administrative endpoint is protected from normal patient access.

Backend authorization is important because frontend restrictions alone cannot provide sufficient

protection. A user could attempt to directly send a request to an API endpoint, so authorization must be enforced by the backend.

The security structure therefore provides an additional layer of protection for healthcare-related application data.

10. Database Integration and Data Management Feature

User Perspective:

The database management feature allows the system to store and retrieve healthcare scheduling information in an organized manner.

Patients do not directly interact with the database. Instead, they use the application interface while the backend handles the required database operations.

The system can store information related to patients, doctors, appointments, doctor availability, and notifications.

Technical Description:

MySQL is used as the relational database management system. Different tables are used to maintain different types of application data.

Primary keys provide unique identification for records, while foreign keys establish relationships between related tables.

For example, an appointment can contain references to the patient and doctor associated with the appointment. This creates a structured relationship between the different modules.

The FastAPI backend communicates with MySQL and performs the required operations based on requests received from the frontend.

Database validation and appropriate error handling are used to ensure that invalid operations do not result in incorrect application responses.

The database structure supports the modular design of the application and provides persistent storage for the healthcare scheduling information.

11.API-Based System Communication Feature

User Perspective:

The application provides a smooth experience by allowing the frontend to communicate with the backend through APIs. Users interact with the graphical interface without needing to understand the underlying database or backend processes.

For example, when a patient books an appointment, the patient simply selects the doctor, date, and time and submits the request. The underlying API communication takes place automatically.

Technical Description:

REST APIs developed using FastAPI provide communication between the frontend and backend. Different HTTP methods are used according to the required operation.

GET requests are used to retrieve information, POST requests are used to create new records, PUT requests are used to update existing information, and appropriate deletion or status-update operations can be used for cancellation-related functionality.

The API layer separates frontend presentation from backend processing. This makes the system easier to maintain because changes to the user interface do not require direct changes to database operations.

API testing can be performed using Postman to verify request formats, authentication, responses, and endpoint functionality.

The API-based architecture therefore provides a structured communication mechanism between the different layers of the healthcare appointment scheduling system.

7.3 Feature Integration

The individual features of the healthcare appointment scheduling system are interconnected to form a complete application workflow.

A typical patient workflow begins with registration and login. After successful authentication, the patient can browse doctors and view their specialization and availability. The patient can then select an available appointment slot and submit a booking request.

The backend validates the request and stores the appointment in the MySQL database. After successful scheduling, the notification module can generate an appointment confirmation.

The patient can later access the appointment management section to view, update, or cancel the appointment according to the available functionality. Changes to the appointment can also result in corresponding notification updates.

The doctor module provides the information required during appointment selection, while the admin module provides centralized management of system records.

Authentication and role-based authorization operate across the different modules to ensure that users access only the features permitted for their roles.

The API layer connects the frontend with the backend, while the MySQL database provides persistent storage. This creates a modular architecture in which each feature has a defined responsibility while remaining connected to the overall healthcare scheduling workflow.

The combination of these features provides a structured digital solution for managing healthcare appointments and reduces dependence on manual scheduling processes.

CHAPTER -8 TESTING

8.1. Postman Testing (Frontend/Backend)

Testing is an important stage in the development of the Secure and Scalable Healthcare Appointment Scheduling system. The primary purpose of testing is to verify that the frontend, backend, database, authentication system, and API communication work correctly together.

Postman is used as an API testing tool to verify the communication between the frontend and FastAPI backend. It allows individual API endpoints to be tested without depending completely on the graphical user interface. This makes it easier to identify errors in backend processing, request formats, authentication, and database operations.

The testing process begins by identifying the major API endpoints available in the application. These include endpoints related to user registration, login, patient information, doctor information, doctor availability, appointment scheduling, appointment management, notifications, and administrative operations.

Different HTTP methods are used according to the purpose of each API. GET requests are used to retrieve information, POST requests are used to create new records, PUT requests are used to update existing information, and appropriate DELETE or status-update operations can be used for cancellation-related functionality.

User Registration Testing

The registration API is tested by sending valid patient information through a POST request. The response is checked to determine whether the backend correctly accepts the information and creates the required user record.

Invalid inputs are also considered during testing. Missing required fields, incomplete information, or incorrectly formatted values should be rejected by the backend rather than being stored as valid records.

This testing verifies that the registration process performs the required validation before interacting with the database.

Login and Authentication Testing

The login API is tested using valid and invalid credentials. When correct credentials are provided, the backend should authenticate the user and generate an appropriate JWT authentication token.

When incorrect credentials are provided, the request should not result in successful authentication. The backend should return an appropriate response indicating that authentication was unsuccessful.

Protected API endpoints are also tested without a token to verify that unauthorized requests are rejected.

A valid token is then included in the request header to confirm that an authenticated user can access the permitted resources.

Doctor Information Testing

The doctor-related APIs are tested to verify that registered doctor information can be retrieved correctly.

Requests can be used to retrieve doctor information such as name, specialization, contact details, and availability. The returned information is checked to ensure that the response structure is correct.

Search or filtering functionality based on doctor specialization can also be tested where implemented.

Appointment Booking Testing

Appointment booking is one of the most important parts of the application and therefore requires careful API testing.

A valid appointment request is submitted with the required patient, doctor, date, and time information. The backend is expected to validate the request and create an appointment when the selected slot is available.

The API is also tested using an unavailable or already occupied appointment slot. The backend should prevent conflicting bookings and return an appropriate response.

Testing appointment creation in this way helps ensure that the scheduling system maintains correct appointment information.

Appointment Update and Cancellation Testing

Existing appointment records are tested for update and cancellation operations.

For an update request, the backend verifies whether the requested appointment exists and whether the authenticated user is permitted to modify it.

For cancellation, the system verifies the appointment and updates its status according to the implemented functionality.

The testing also verifies that unauthorized users cannot modify or cancel appointments belonging to another patient.

Notification Testing

Notification-related APIs are tested to verify that appropriate notification information is created

and retrieved.

For example, after a successful appointment booking, the system can generate an appointment confirmation notification. Similar checks can be performed for appointment updates and cancellations.

The notification response is checked to ensure that it is associated with the appropriate user and appointment.

Administrative API Testing

Administrative endpoints are tested using an authorized administrator account. The tests verify whether permitted administrative operations can be performed successfully.

The same endpoints are also tested using a normal patient account. These requests should be rejected when the patient does not have the required administrative role.

This verifies that role-based authorization is being applied at the backend level.

Error and Validation Testing

The APIs are also tested using incorrect or incomplete request data. Examples include missing required fields, invalid identifiers, unavailable appointment slots, invalid authentication tokens, and unauthorized role access.

The objective is to ensure that the backend does not process invalid requests as successful operations.

Appropriate HTTP response codes and messages are checked during testing so that the frontend can correctly identify successful and unsuccessful operations.

Postman therefore provides a controlled environment for verifying the behaviour of the healthcare application's REST APIs.

8.2 Integration Testing

Integration testing is performed to verify that the different modules of the healthcare appointment scheduling system work together correctly.

The application contains several interconnected components, including authentication, patient management, doctor management, appointment scheduling, notifications, database operations, and administrative functions.

Although each module can be tested independently, the complete application requires these modules to communicate correctly. Integration testing therefore focuses on the flow of

information between different components.

Frontend and Backend Integration

The first stage of integration testing verifies communication between the frontend and FastAPI backend.

When a user performs an operation through the frontend, the application sends a request to the appropriate backend API. The backend processes the request and returns a response.

The response is then used by the frontend to update the interface.

For example, when a patient requests a list of doctors, the frontend sends a request to the backend. The backend retrieves the required information from the database and returns the response. The frontend then displays the doctor information.

This process is tested to ensure that the request and response formats remain consistent.

Authentication Integration

Authentication is tested as an integrated process between the frontend, backend, and database. A patient first registers through the frontend. The registration information is sent to the backend and stored in the database.

The patient then logs in using the registered credentials. The backend verifies the information and returns the authentication token.

The frontend uses the token for subsequent protected requests.

Testing this complete sequence verifies that authentication information is correctly transferred between the different layers of the application.

Doctor and Availability Integration

The doctor management and availability modules are connected with appointment scheduling.

When a patient selects a doctor, the system should retrieve the appropriate doctor information and available appointment slots.

The integration test verifies that the selected doctor identifier is correctly transferred to the availability and appointment modules.

Incorrect or unavailable doctor information should not result in a successful appointment.

Appointment and Database Integration

Appointment scheduling requires communication between the frontend, backend, and MySQL database.

When the patient submits a booking request, the backend validates the information and performs the required database operation.

The resulting appointment should then be retrievable through the patient's appointment section.

Integration testing checks whether the appointment created through the booking interface is correctly stored and later retrieved from the database.

Appointment and Notification Integration

The notification module is integrated with appointment operations.

When an appointment is successfully created, the system can generate a confirmation notification.

If an appointment is updated or cancelled, the corresponding notification information can also be generated.

Integration testing verifies that the correct user and appointment information is associated with the notification.

Role-Based Authorization Integration

The authentication and authorization modules are tested together.

After a user logs in, the backend identifies the user's role. The role is then used to determine which endpoints the user can access.

A patient account is tested against patient-specific operations and administrative operations.

Similarly, administrative access is tested using an authorized administrator account.

This ensures that authentication and authorization work together instead of functioning as separate systems.

Database Integration

The database is tested as part of the complete application workflow.

Patient records, doctor records, appointment records, availability information, and notifications are checked to ensure that relationships between them are maintained.

For example, an appointment should contain the correct relationship between the selected patient and doctor.

Database integration testing helps identify problems such as incorrect identifiers, missing records, invalid relationships, or unsuccessful database operations.

End-to-End Integration

The complete patient workflow is also considered during integration testing.

A typical end-to-end sequence includes:

1. Patient registration.
2. Patient login.
3. Doctor search.
4. Viewing doctor availability.
5. Selecting an appointment slot.
6. Booking the appointment.
7. Storing the appointment in the database.
8. Generating the appropriate notification.
9. Viewing the appointment.

Updating or cancelling the appointment when permitted.

Testing this complete sequence helps verify that the modules operate together as one system.

Integration testing therefore ensures that the application does not only contain individually working modules but also provides correct communication between those modules.

8.3 Tools Used for Testing

Different tools are used during the development and testing of the healthcare appointment scheduling system. Each tool supports a particular part of the testing process.

1. Postman

Postman is the primary tool used for API testing.

It allows developers to send HTTP requests directly to FastAPI endpoints and inspect the returned responses. It can be used to test registration, login, doctor information, appointment operations, notifications, and administrative APIs.

Postman also allows authentication headers and request parameters to be added during testing. This is useful for testing protected endpoints that require JWT authentication.

2. Swagger UI

FastAPI provides interactive API documentation through Swagger UI. It can be used to view the available API endpoints, request parameters, request methods, and response structures.

Swagger UI is useful during development because developers can inspect and test API endpoints directly through the generated documentation interface.

It also helps maintain a clear understanding of the backend API structure.

3. Web Browser

A web browser is used for testing the frontend interface of the healthcare application.

The browser allows developers to verify navigation, forms, buttons, appointment interfaces, doctor information, notifications, and other user-facing components.

Different user actions can be performed manually to check whether the frontend communicates correctly with the backend.

4. MySQL Database Tools

MySQL tools are used to verify database records created or modified by the application.

For example, after a patient registers, the corresponding database record can be checked. Similarly, after an appointment is created, the appointment table can be inspected to verify that the correct information has been stored.

Database tools also help identify problems related to relationships, missing records, or incorrect values.

5. Visual Studio Code

Visual Studio Code is used as the development environment for working with the project's frontend and backend source code.

During testing and debugging, developers can inspect application code, identify errors, modify implementation logic, and run the backend application.

The integrated terminal also provides a convenient environment for running development commands and checking application output.

6. Git and GitHub

Git can be used for version control during the development process. It allows team members to maintain different versions of the project source code and track changes.

GitHub can be used to maintain the project repository and support collaboration between team members.

Version control is useful during testing because changes can be tracked and previous versions can be reviewed when required.

7. FastAPI

FastAPI is not only the backend framework but also supports the testing process through its API structure and automatically generated documentation.

Its validation mechanisms help identify invalid request data before backend operations are completed.

The framework also provides clear API responses that can be examined during testing.

8.4 Test Cases and Results

The testing phase of the Secure and Scalable Healthcare Appointment Scheduling system focuses on verifying the main functional modules and their interactions.

The test cases are designed around the major operations performed by patients, doctors, and administrators.

Test Case	Test Scenario	Expected Result
TC01	Register with valid patient details	Patient registration should be accepted
TC02	Register with incomplete details	Invalid registration should be rejected
TC03	Login with valid credentials	User should be authenticated
TC04	Login with incorrect credentials	Authentication should fail
TC05	Access protected API without token	Request should be rejected
TC06	Access protected API with valid token	Authorized request should be processed
TC07	Retrieve doctor information	Correct doctor details should be displayed
TC08	View doctor availability	Available slots should be displayed
TC09	Book an available appointment	Appointment should be created successfully
TC10	Book an unavailable appointment slot	Conflicting booking should be rejected
TC11	View patient's appointments	Patient's appointment records should be displayed
TC12	Update permitted appointment	Appointment should be updated correctly
TC13	Cancel permitted appointment	Appointment status should be updated/cancelled
TC14	Generate appointment notification	Appropriate notification should be created
TC15	Access admin function as patient	Access should be denied
TC16	Access admin function as authorized administrator	Request should be processed
TC17	Retrieve notification information	Correct notifications should be displayed
TC18	Verify database appointment record	Appointment data should be stored correctly

Authentication Test Results

Authentication testing verifies registration, login, token generation, and protected endpoint access.

Valid registration and login requests are expected to proceed successfully when the required information is provided. Invalid credentials and incomplete requests are expected to be rejected.

Protected endpoints are checked with and without authentication tokens to verify that unauthorized access is not permitted.

Appointment Test Results

Appointment testing focuses on the core scheduling functionality.

The system is expected to create an appointment when a valid doctor, patient, date, and available time slot are provided.

When the selected slot is already occupied or otherwise unavailable, the booking request should not be accepted.

Appointment retrieval is also tested to verify that users can access the appropriate appointment records.

Authorization Test Results

Role-based authorization testing verifies that users cannot access functions outside their permitted roles.

A patient account is tested against administrative endpoints to confirm that restricted operations are rejected.

An authorized administrator is tested against the same operations to verify that permitted access is available.

Database Test Results

Database testing verifies that information submitted through the application is correctly stored and retrieved.

Patient registration should create the appropriate patient record. Doctor information should be retrievable from the relevant database records.

When an appointment is created, the appointment record should contain the correct patient and doctor relationships.

Notification records should also be associated with the appropriate user and appointment.

Integration Test Results

Integration testing verifies that the frontend, backend, authentication, database, appointment, doctor, and notification modules communicate correctly.

The complete patient workflow is tested from registration and login through doctor selection and appointment booking.

The purpose of these tests is to ensure that data remains consistent as it moves between the different layers of the application.

Overall Testing Outcome

The testing process provides a structured method for identifying functional, integration, authentication, authorization, and database-related issues in the healthcare appointment scheduling system.

Postman is used to verify backend APIs, the browser is used to validate the frontend, Swagger UI supports API inspection, and MySQL tools help verify stored data.

The test cases are designed to ensure that the main healthcare scheduling operations work according to the intended system design.

Since the project documentation does not provide a verified numerical pass percentage or production test report, no specific success percentage is claimed here. The results should be updated with the team's actual execution results if formal test records are available.

Overall, the testing stage helps verify the reliability of the system and provides confidence that the major modules can communicate correctly through the FastAPI backend and MySQL database.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The deployment of the Secure and Scalable Healthcare Appointment Scheduling system represents the transition of the developed application from the development environment to an environment where it can be accessed and used as intended. Deployment involves preparing the frontend, backend, database, configuration files, and required dependencies so that all components can communicate correctly.

The deployment process begins with preparing the project environment. The development team verifies that the source code, required dependencies, database configuration, and application settings are available and correctly configured. The project is maintained using a structured source-code organization so that the frontend and backend components can be managed independently.

The first step is to prepare the backend application developed using Python FastAPI. The required Python packages are installed according to the project's dependency configuration. The backend is then started using the appropriate development server command. Once the server is running, the available API endpoints can be accessed for testing.

The next step involves preparing the MySQL database. The database must be created and configured before the backend application can perform database operations. The required tables for patients, doctors, appointments, doctor availability, and notifications are created according to the database design.

Database connection information is configured in the backend so that FastAPI can communicate with MySQL. The connection configuration includes the required database host, database name, username, and authentication information.

After database configuration, the backend connection is verified. The application is tested to ensure that it can successfully establish a connection with the database and perform basic operations such as inserting, retrieving, and updating records.

The frontend application is then configured to communicate with the FastAPI backend. The frontend must use the correct backend API address so that requests generated through the user interface reach the appropriate API endpoints.

After configuring the frontend and backend, the complete application workflow is tested. This includes registration, login, doctor browsing, availability checking, appointment booking, appointment management, and notification operations.

Authentication is also verified during deployment preparation. JWT-based authentication is checked to ensure that protected endpoints require valid authentication information.

Role-based access is tested to ensure that patients, doctors, and administrators receive the appropriate permissions. Administrative functions should remain protected from unauthorized users.

Once the application has been configured, API testing is performed using Postman. Important

endpoints are tested to verify that the backend is responding correctly and that database operations are functioning as expected.

The frontend is then tested through a web browser. The main pages are opened and user workflows are performed to verify that the frontend successfully communicates with the backend.

A final verification is performed after all components have been connected. This ensures that the frontend, FastAPI backend, MySQL database, authentication mechanism, and notification module work together as a complete system.

The deployment process therefore consists of environment preparation, dependency installation, database configuration, backend setup, frontend configuration, authentication verification, API testing, and final system validation.

9.2 Environment Configuration

Environment configuration is an important part of deploying the healthcare appointment scheduling system because the frontend, backend, and database must operate using compatible settings.

The development environment contains the software and tools required to build and test the application. The backend uses Python and FastAPI, while MySQL is used for persistent data storage.

The frontend environment contains the technologies required to display the application interface and communicate with the backend APIs. The frontend sends requests to FastAPI rather than directly connecting to the MySQL database.

The backend environment requires the appropriate Python version and project dependencies. These dependencies are installed before starting the application.

The FastAPI application is configured with the required database connection information. The configuration allows the backend to establish communication with the MySQL database.

Sensitive configuration information should not be directly written into application source code. Database credentials, authentication secrets, and other private configuration values should be maintained separately from publicly shared source files.

Environment variables can be used to store configuration values that may change between development and deployment environments. This makes the application easier to configure without modifying the main source code.

The database environment contains the tables required by the application. The main data categories include patient information, doctor information, appointment information, doctor availability, and notification information.

The database connection is verified before the application is started. If the connection is not correctly configured, database-dependent functions such as registration and appointment booking

cannot operate correctly.

The API environment is also configured so that the frontend knows the address of the FastAPI backend. This is particularly important when the frontend and backend are running separately.

Authentication configuration is another important part of the environment. JWT-related configuration values must be available to the backend so that authentication tokens can be generated and validated correctly.

The development environment can be used for implementing and debugging the system. A separate deployment environment can later be configured when the application is moved to a hosted server.

The configuration should remain consistent across environments wherever possible. Differences in database configuration, API addresses, or dependencies can result in unexpected behaviour after deployment.

Version control can also be used to maintain the application source code and configuration templates. Sensitive values should not be committed to a public repository.

The environment configuration therefore provides the foundation required for reliable execution of the healthcare appointment scheduling application.

9.3 Hosting of Frontend and Backend

Hosting is the process of making the developed application available through an appropriate server or hosting environment. The current project documentation focuses on the development and backend architecture of the healthcare appointment scheduling system and does not specify a particular production hosting provider.

Therefore, the application can be prepared for hosting without claiming that it has already been deployed to a specific cloud platform.

Frontend Hosting

The frontend contains the user interface through which patients, doctors, and administrators interact with the application.

For deployment, the frontend application can be built into the required production files and hosted on a suitable web server or frontend hosting platform.

The frontend must be configured with the correct FastAPI backend address. This allows user actions such as login, doctor search, appointment booking, and appointment management to

generate requests to the backend.

Before hosting the frontend, the interface should be tested to ensure that all required API calls use the correct backend configuration.

The frontend should also be checked for responsive behaviour so that the healthcare scheduling interface remains usable on different screen sizes.

Backend Hosting

The backend is developed using Python FastAPI. It acts as the central application layer between the frontend and the MySQL database.

For deployment, the FastAPI application can be run using a suitable production server configuration. The backend server must have access to the required Python dependencies and database configuration.

The backend should expose the API endpoints required by the frontend. These endpoints include authentication, patient management, doctor management, appointment scheduling, appointment management, notification management, and administrative operations.

The backend should not expose the MySQL database directly to frontend users. All database operations should pass through the backend API layer.

Database Hosting

The MySQL database provides persistent storage for the application.

In a deployment environment, the database can be hosted on the same server or on a separately managed database server depending on the final infrastructure configuration.

Database access should be restricted so that only the backend application has the required permissions to perform application-level operations.

Regular database backups should also be considered when the system is moved to a production environment.

Frontend–Backend Communication

After hosting the frontend and backend, communication between them must be verified.

A patient registering through the frontend should generate a request to the FastAPI backend. The backend should validate the request and store the information in MySQL.

Similarly, when a patient books an appointment, the request should travel from the frontend to the backend, be validated by the backend, and then be stored in the database.

The response from the backend should then be returned to the frontend so that the user can see the result of the operation.

Security During Hosting

Security is an important consideration when hosting a healthcare-related application.

Authentication and authorization must remain active after deployment. Protected API endpoints should continue to require valid JWT authentication.

Database credentials and authentication secrets should be kept confidential and should not be exposed through the frontend.

HTTPS should be used when the application is deployed publicly so that information exchanged between users and the server is protected during transmission.

Access to administrative functions should also remain restricted through role-based authorization.

Post-Deployment Verification

After hosting, the main application workflows should be tested again.

The verification process should include:

- Patient registration
- Patient login
- Doctor information retrieval
- Doctor availability
- Appointment booking
- Appointment viewing
- Appointment update
- Appointment cancellation
- Notification retrieval
- Administrative access
- Unauthorized access attempts

The API endpoints can be tested using Postman, while the complete user workflow can be tested through the frontend.

Any configuration issue discovered during deployment should be corrected before the application is considered ready for regular use.

Maintenance

After deployment, the application requires regular maintenance. Backend dependencies, frontend components, database configuration, and security-related settings should be reviewed periodically.

Application logs can be monitored to identify unexpected errors or failed requests. Database backups should be maintained to reduce the risk of data loss.

When new features or changes are introduced, they should first be tested before being included in the deployed version.

The deployment process therefore provides a structured transition from development to a usable application environment. It ensures that the frontend, FastAPI backend, MySQL database, authentication system, and API layer are correctly configured and able to operate together.

The current project documentation does not provide evidence of a completed public production deployment, specific cloud hosting provider, domain name, or live deployment URL. Therefore, these details are intentionally not claimed as completed deployment activities.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Challenges Faced During Development

During the development of the **Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development** system, the team needs to integrate several components, including the frontend, FastAPI backend, MySQL database, authentication, REST APIs, appointment management, and notifications. Integrating these components into a single working application presents several technical and organizational challenges.

10.1.1 Frontend–Backend Integration

One of the major challenges is connecting the frontend developed using HTML, CSS, and JavaScript with the FastAPI backend.

The frontend sends requests to the backend through REST APIs. The backend processes these requests, communicates with the MySQL database, and returns the required response to the frontend.

Incorrect API URLs, request formats, parameters, or response handling can result in communication errors.

One of the major challenges was integrating the React.js frontend with the FastAPI backend. The frontend was developed using React.js, HTML, CSS, and JavaScript with Vite as the development and build tool. The frontend communicates with the backend through REST APIs. Incorrect API URLs, request formats, parameters, authentication information, or response handling can result in communication errors. Maintaining consistent communication between the React.js components and FastAPI endpoints was therefore an important part of the development process.

10.1.2 Database Connectivity

The FastAPI backend needs to communicate correctly with the MySQL database.

The system performs database operations such as:

- Creating users.
- Storing patient information.
- Storing doctor information.
- Managing doctor availability.
- Creating appointments.
- Updating appointments.

- Storing notifications.
- Retrieving appointment information.

Correct database configuration and proper SQL queries are therefore important.

10.1.3 Authentication and Authorization

The implemented application provides authenticated functionality for patients and administrators. Doctor information and availability are managed through the administrator functionality rather than through a separate doctor login or doctor dashboard.

The system needs to ensure that:

- Users can log in using valid credentials.
- Invalid credentials are rejected.
- Protected resources are not accessible without authentication.
- Patients can access only permitted patient-related functions.
- Administrative functions are restricted to authorized administrators.

10.1.4 Appointment Scheduling

Appointment scheduling requires careful management of doctor availability, appointment dates, and appointment times.

The system must check whether a doctor is available before an appointment is booked. It should also reduce the possibility of two patients attempting to book the same doctor and time slot.

10.1.5 Notification Management

The system includes a notification module to provide appointment-related information.

Notifications may be generated for events such as:

- Appointment booking.
- Appointment confirmation.
- Appointment update.
- Appointment cancellation.

Ensuring that the correct notification is associated with the correct user and appointment is an important part of the system.

10.1.6 API Testing

The REST APIs need to be tested before complete frontend integration.

Postman can be used to test:

- Registration.
- Login.

- Doctor information.
- Doctor availability.
- Appointment operations.
- Notifications.

Testing APIs individually helps identify backend problems before they affect the complete application.

10.1.7 Team Integration

Since the project contains frontend, backend, and database components, combining the work of multiple team members can be challenging.

Differences in:

- File names.
- Folder structures.
- API names.
- Database column names.
- Coding approaches.

can cause integration problems.

Git and GitHub can be used to maintain a common codebase and coordinate development.

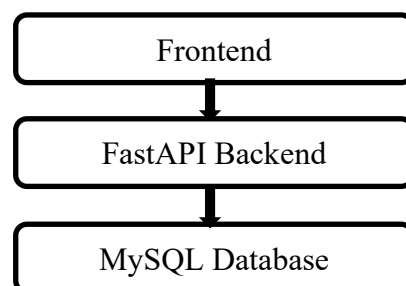
10.2 Solutions Applied

The project uses several approaches to handle the development challenges.

10.2.1 Layered Architecture

The frontend–backend integration challenge was addressed by using React.js and Vite for the frontend and FastAPI REST APIs for backend communication. The frontend sends structured requests to the backend, while the backend validates the requests and returns structured responses.

The system separates the frontend, backend, and database.



This makes the system easier to understand, develop, test, and maintain.

10.2.2 REST API Communication

FastAPI REST APIs are used to provide communication between the frontend and backend.

This creates a clear interface between the user interface and backend operations.

10.2.3 Database Relationships

Primary keys and foreign keys are used to maintain relationships between:

- Users.
- Patients.
- Doctors.
- Doctor Availability.
- Appointments.
- Notifications.

10.2.4 Authentication

Authentication is used to control access to protected resources.

Role-based authorization is used to distinguish between patient, doctor, and administrator operations.

10.2.5 Appointment Validation

Before creating an appointment, the backend can check doctor availability and existing appointment records.

This helps reduce appointment conflicts.

10.2.6 API Testing

Postman is used to test APIs independently before connecting all frontend components.

API testing was performed using FastAPI's Swagger UI and browser-based API verification, with Postman used where applicable during development. Testing the API independently helped identify request-format, authentication, validation, and response-handling issues before complete frontend integration.

10.2.7 Version Control

Git and GitHub are used to maintain the source code and coordinate work between team members.

10.3 Current Limitations or Known Bugs

The current system includes a demonstration payment workflow and receipt-related functionality. However, it does not integrate with a live external payment gateway or process real financial transactions.

Then keep the genuinely out-of-scope items:

- Video consultation
- Pharmacy management
- Ambulance management
- Medical billing, if not implemented
- Electronic medical records
- AI-based medical diagnosis
- Advanced hospital management

These features can be considered for future development but are not necessary for the current DBMS project.

10.3.1 Deployment Limitation

The initial version of the project is primarily intended for academic demonstration and local development. Large-scale production deployment would require additional infrastructure and security configuration.

10.3.2 Notification Limitation

The basic notification functionality focuses on appointment-related notifications. Integration with external SMS or email providers may require additional services and configuration.

10.3.3 Scalability Limitation

The system architecture is designed with scalability in mind, but actual large-scale performance would require further load testing and infrastructure optimization.

CHAPTER -11 FUTURE ENHANCEMENTS

The **Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development** project provides the basic functionality required for managing patients, doctors, appointments, authentication, and notifications. However, additional features can be introduced in the future to improve the system's usability, scalability, security, and overall performance.

11.1 Advanced Notification Features

The notification system can be enhanced to provide more advanced and timely communication between the healthcare system and its users. At present, the system focuses on appointment-related notifications.

Future enhancements can include:

- Appointment confirmation notifications.
- Appointment cancellation notifications.
- Appointment rescheduling notifications.
- Automatic appointment reminders.
- Notifications when a doctor changes availability.
- Email notifications for important appointment updates.
- SMS notifications for appointment reminders.
- Notifications for doctors when a new appointment is booked.
- Notifications for patients when an appointment status changes.

These enhancements can help patients and doctors stay informed about important appointment-related activities and reduce missed appointments.

11.2 Reports and Analytics

A reporting and analytics module can be added to provide useful information about the healthcare appointment system.

The system can generate reports such as:

- Total number of registered patients.
- Total number of doctors.
- Total number of appointments.
- Completed appointments.
- Cancelled appointments.

- Pending appointments.
- Doctor-wise appointment statistics.
- Patient-wise appointment history.
- Daily, weekly, and monthly appointment reports.
- Frequently selected doctors or specializations.

The administrator can use these reports to understand appointment patterns and monitor system usage. Graphs and dashboards can also be introduced to make the information easier to understand.

11.3 Cloud Deployment

In the future, the healthcare appointment system can be deployed on a cloud platform instead of running only in a local development environment.

Cloud deployment can provide:

- Remote access to the application.
- Better availability.
- Easier system maintenance.
- Scalable backend infrastructure.
- Centralized database access.
- Improved deployment and backup capabilities.

The FastAPI backend, frontend, and MySQL database can be deployed using suitable cloud services when the system is ready for larger-scale usage.

Cloud deployment can therefore help the system support a larger number of users and provide better availability.

11.4 MongoDB Integration if Required

MySQL is used as the primary database for the current project because the system requires structured relationships between patients, doctors, appointments, availability, and notifications.

If required in the future, **MongoDB** can be evaluated as an alternative or additional database technology for selected use cases.

MongoDB may be useful for storing flexible or semi-structured information such as:

- Notification records.
- Activity logs.
- Application logs.
- Additional user preferences.

- Other dynamically structured data.

However, MongoDB integration is considered a **future enhancement and is not part of the current implementation unless it is specifically approved and developed.**

The current project continues to use **MySQL as the primary database.**

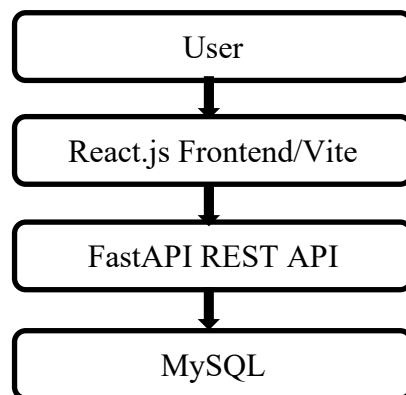
CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development project provides a web-based platform for managing healthcare appointments. The system allows patients to register, authenticate, browse doctor information, view availability, book and manage appointments, and receive application-level notifications.

The system also provides administrative functionality for managing patient information, doctor information, availability, appointments, and other implemented administrative records. The application separates frontend presentation, backend processing, authentication and authorization, and database storage into distinct layers.

The system separates the frontend, backend, and database responsibilities.



This architecture provides a clear separation between the user interface, application logic, and data storage.

12.2 Major Features Implemented

The proposed system contains the following major features:

Patient Management

Patients can register, log in, view doctors, check availability, and manage appointments.

Doctor Management

[Doctor Information Management]

Doctor information, specialization, and availability are maintained through the administrator functionality. Patients can view available doctor information while selecting an appointment. The submitted implementation does not provide a separate doctor login or doctor dashboard. Doctors can maintain their information and availability and view their appointments.

Appointment Management

The system supports:

- Appointment booking.
- Appointment viewing.
- Appointment updating.
- Appointment cancellation.
- Appointment status management.

Authentication

The system provides authentication and authorization to control access to protected functions.

Notification Management

The system provides appointment-related notifications such as:

- Booking confirmation.
- Appointment updates.
- Cancellation notifications.

Admin Management

The administrator can manage important system information and monitor appointment-related activities.

Demonstration Payment Workflow

The application includes a simulated payment workflow with payment-related records and receipt information. It does not integrate with a live external payment gateway.

12.3 DBMS Concepts Applied

The project demonstrates several important Database Management System concepts.

Database Design

The system uses a relational database structure to organize healthcare appointment information.

Primary Keys

Primary keys uniquely identify records in each table.

Foreign Keys

Foreign keys establish relationships between related tables.

Relationships

The system contains relationships between:

- Users and Patients.
- Users and Doctors.
- Doctors and Availability.
- Patients and Appointments.
- Doctors and Appointments.
- Users and Notifications.
- Appointments and Notifications.

SQL

SQL is used for:

- Creating tables.
- Inserting records.
- Retrieving records.
- Updating records.
- Joining related tables.

Normalization

The database separates different types of information into appropriate tables to reduce unnecessary data duplication and maintain data consistency.

12.4 Backend and API Concepts Applied

The project also demonstrates backend development concepts.

FastAPI is used to create REST APIs that connect the React.js frontend with the application logic and MySQL database. The backend is responsible for processing requests, validating input, performing JWT-based authentication and authorization, applying business logic, communicating with MySQL, and returning responses to the frontend.

The backend is responsible for:

- Processing requests.
- Validating input.
- Performing authentication.

- Applying business logic.
- Communicating with MySQL.
- Returning responses to the frontend.

12.5 Overall Outcome

The project provides a structured approach to healthcare appointment scheduling by integrating:

- Frontend development.
- Backend development.
- REST APIs.
- Database management.
- Authentication.
- Appointment scheduling.
- Notifications.
- Administrative management.

The project demonstrates how DBMS concepts can be combined with backend and frontend technologies to create a practical healthcare application.

12.6 Skills Applied

Through the project, the team applies knowledge of:

- Database Management Systems.
- SQL.
- ER modelling.
- Database normalization.
- Primary and foreign keys.
- HTML.
- CSS.
- JavaScript.
- Python.
- FastAPI.
- REST APIs.
- Authentication.

- API testing.
- Git and GitHub.
- Frontend–backend integration.
- React.js
- Vite
- Node.js/npm, if actually used in the project

CHAPTER -13 REFERENCES

13.1 Books

1. Elmasri, R. and Navathe, S. B., *Fundamentals of Database Systems*, Pearson.
2. Silberschatz, A., Korth, H. F. and Sudarshan, S., *Database System Concepts*, McGraw-Hill.

These books are relevant to the **DBMS concepts, database design, ER diagrams, relationships, normalization, and SQL** used in your project.

13.2 Tutorials

The team referred to relevant tutorials for understanding and implementing:

- Python backend development
- FastAPI REST API development
- MySQL and SQL queries
- HTML, CSS, and JavaScript
- Frontend–backend integration
- REST API testing
- Git and GitHub

13.3 FastAPI Documentation

FastAPI documentation was referred to for implementing the backend REST APIs, including:

- API routing
- Request and response handling
- Data validation
- API endpoints
- Automatic API documentation
- Backend application development

13.4 MySQL Documentation

MySQL documentation was referred to for:

- Database creation
- Table creation
- Primary keys
- Foreign keys
- Relationships
- SQL queries
- Data insertion
- Data retrieval
- Data updating
- Database management

13.5 Postman Documentation

Postman documentation was referred to for testing and validating the REST APIs of the healthcare appointment scheduling system.

The API testing included operations related to:

- User registration
- User login
- Doctor information
- Doctor availability
- Appointment management
- Notifications

13.6 Other Resources Used

Other technical resources used during the development of the project included:

- Python documentation
- HTML documentation
- CSS documentation
- JavaScript documentation
- Git documentation
- GitHub documentation
- VS Code resources

- Other technical resources consulted during development and testing

CHAPTER -14 APPENDICES

The appendices contain supporting materials related to the **Secure and Scalable Healthcare Appointment Scheduling Through Distributed Backend Development** project.

Appendix A – Screenshots of the Application

This appendix contains screenshots of the developed healthcare appointment scheduling system.

A.1 Home Page

Include the screenshot of the main home page of the application.

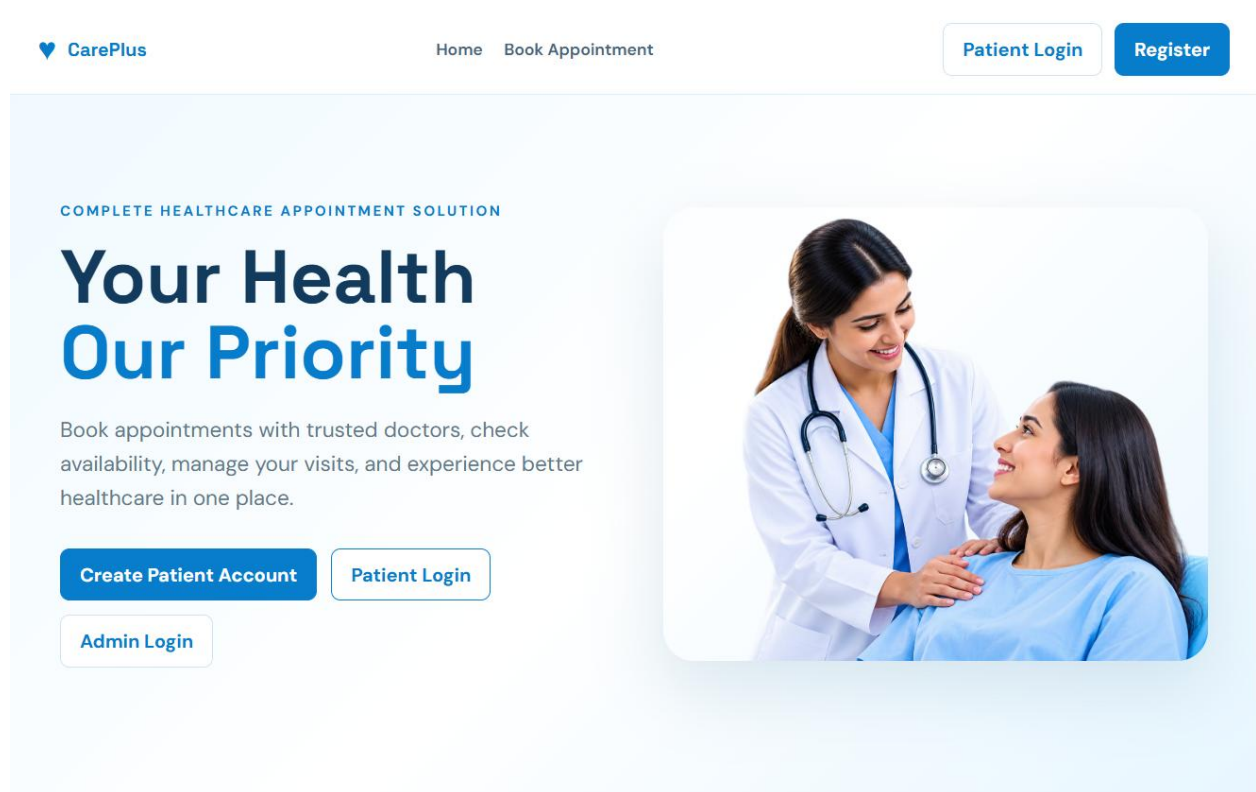


Figure A.1: Healthcare Appointment System Home Page

A.2 Registration Page

Include the screenshot of the patient registration page.

Create Your CarePlus Account

Full Name

Date of Birth

mm/dd/yyyy



Age

Phone

Email

Address

Emergency Contact Name

Emergency Contact Number

Password

Confirm Password

Gender

Select gender ▼

Create Patient Account

Create Your CarePlus Account

Email already registered

This account already exists. Your new password was not saved.

[Go to Patient Login](#)

Full Name

CarePlus E2E Duplicate Attempt

Date of Birth

01/15/2000



Age

26

Phone

900029490572

Email

careplus.e2e.1790529490572@example.com

Address

E2E test address

Emergency Contact Name

Test Contact

Emergency Contact Number

9000000001

Password

.....

Confirm Password

CarePlusTest!246

Gender

Other



Create Patient Account

Registration Successful!

Welcome, **CarePlus E2E Patient**.

Your Patient ID is:

P202600053

Please save this ID for login.

[Go to Patient Login](#)

Figure A.2: Patient Registration Page

A.3 Login Page

Include the screenshot of the user login page.

CAREPLUS PATIENT PORTAL

Welcome Back

Login to access your account

Patient ID or Email

P202600001 or patient@example.com

Password

Enter your password

☐ Remember me

[Forgot Password?](#)

Login

New here? [Create an account](#)

CAREPLUS ADMIN PORTAL

Admin Login

Secure access to manage doctors, appointments, patients and payments.



Admin Access

Enter your admin credentials.

Admin Email *

admin1@careplus.com

Password *

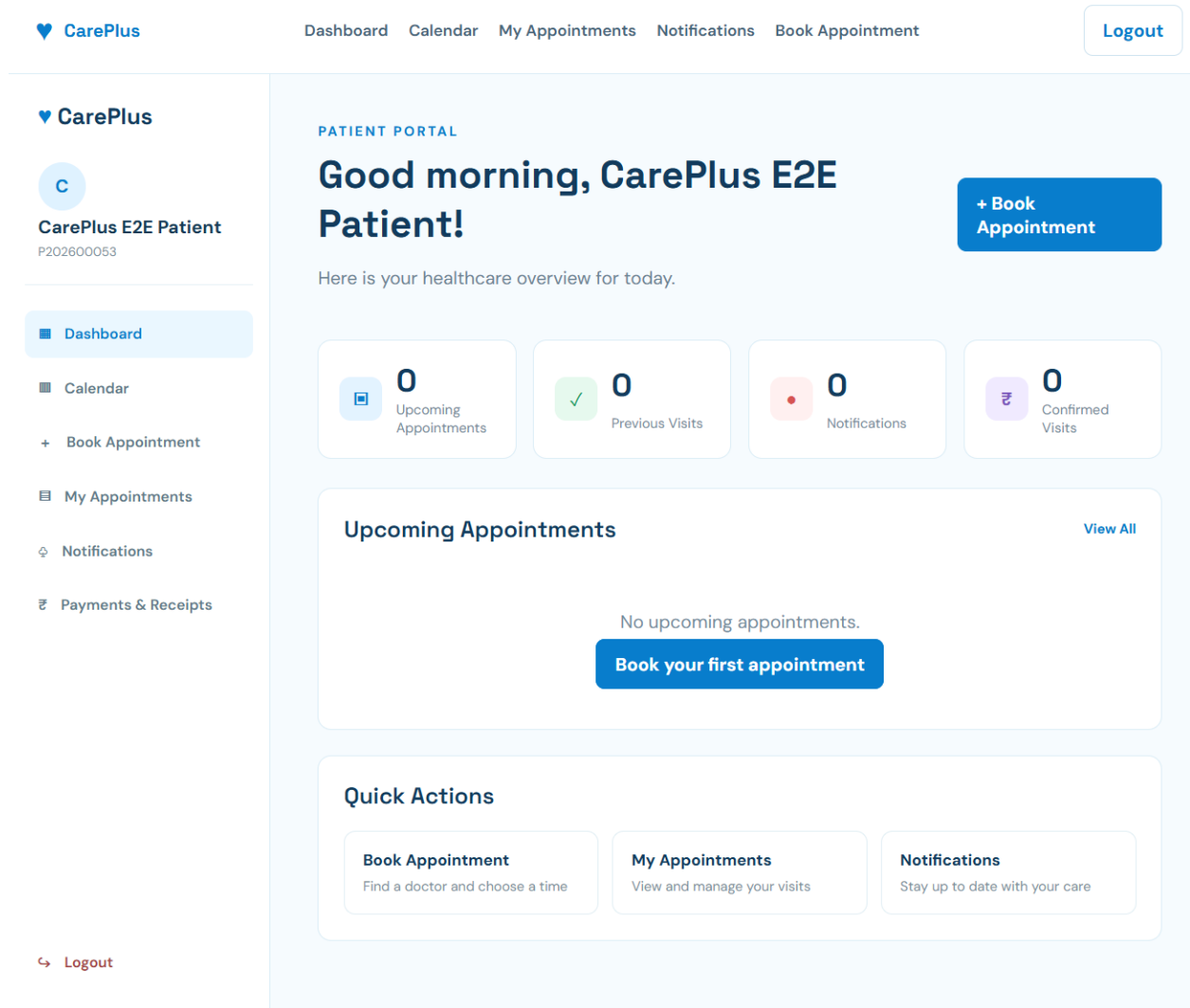
Enter admin password

Login as Admin →

Figure A.3: User Login Page

A.4 Patient Dashboard

Include the screenshot of the patient dashboard after successful login.



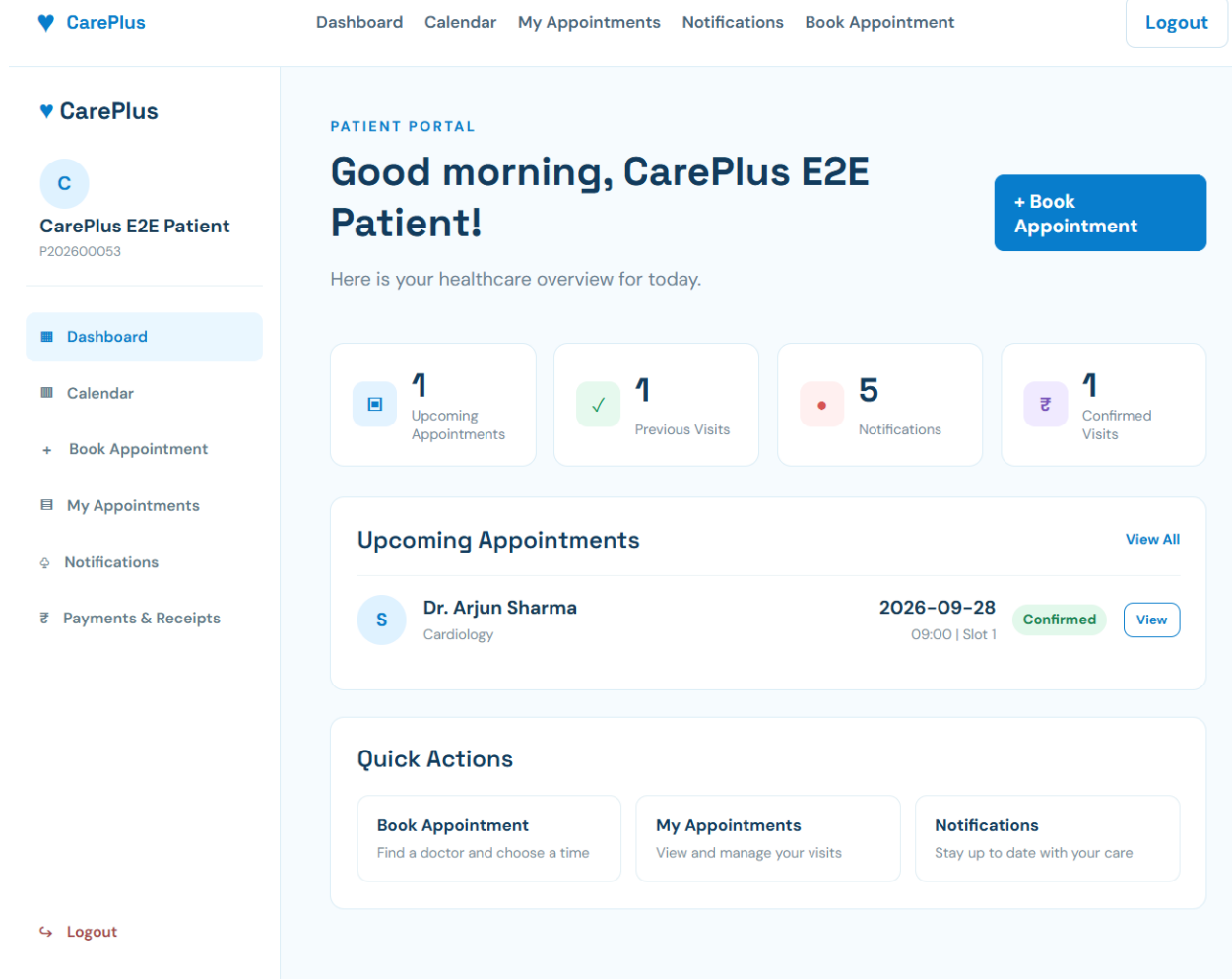


Figure A.4: Patient Dashboard

A.5 Doctor List

Include the screenshot showing the available doctors.

DASHBOARD / DOCTORS

Find a Doctor

Filter by specialization

Cardiology

Dr. Arjun Sharma**Cardiology**

MBBS, MD Cardiology

12 years experience

Available: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday | 09:00 – 11:30

Consultation fee: ₹500

[View Profile](#)

DASHBOARD / DOCTORS

Find a Doctor

Filter by specialization

Dentistry

Dr. Karan Shah**Dentistry**

BDS, MDS

7 years experience

Available: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday | 09:00 – 11:30

Consultation fee: ₹500

[View Profile](#)

DASHBOARD / DOCTORS

Find a Doctor

Filter by specialization

Dermatology ▾

Dr. Priya Mehta

Dermatology

MBBS, MD Dermatology

9 years experience

Available: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday | 09:00 – 11:30

Consultation fee: ₹500

View Profile

DASHBOARD / DOCTORS

Find a Doctor

Filter by specialization

General Medicine ▾

Dr. Neha Kapoor

General Medicine

MBBS, MD

13 years experience

Available: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday | 09:00 – 11:30

Consultation fee: ₹500

View Profile

DASHBOARD / DOCTORS

Find a Doctor

Filter by specialization

Neurology

Dr. Rahul Verma

Neurology

MBBS, DM Neurology

11 years experience

Available: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday | 09:00 – 11:30

Consultation fee: ₹750

[View Profile](#)

DASHBOARD / DOCTORS

Find a Doctor

Filter by specialization

Ophthalmology

Dr. Ananya Patel

Ophthalmology

MBBS, MS Ophthalmology

8 years experience

Available: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday | 09:00 – 11:30

Consultation fee: ₹500

[View Profile](#)

DASHBOARD / DOCTORS

Find a Doctor

Filter by specialization

Orthopedics ▾

Dr. Sneha Rao

Orthopedics

MBBS, MS Orthopedics

10 years experience



Available: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday | 09:00 – 11:30

Consultation fee: ₹500

[View Profile](#)

Figure A.5: Doctor List

A.6 Doctor Availability

Include the screenshot showing the available dates and time slots of a doctor.

← Back to Doctor Profile
SELECT DATE AND TIME

Calendar & Slot Selection

● Available ● Full ● Holiday ● My Upcoming Appointment ● Previous Visit ● Visited – Not Completed

Dr. Arjun Sharma

09-27 Unavailable	09-28 My Upcoming Appointment	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 8/10	10-06 Available 0/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Available 0/10	10-14 Available 0/10	10-15 Available 0/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

← Back to Doctor Profile
SELECT DATE AND TIME

Calendar & Slot Selection

● Available ● Full ● Holiday ● My Upcoming Appointment ● Previous Visit ● Visited – Not Completed

Dr. Priya Mehta

09-27 Unavailable	09-28 Available 0/10	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 0/10	10-06 Available 1/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Available 0/10	10-14 Available 0/10	10-15 Available 0/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

← Back to Doctor Profile
SELECT DATE AND TIME

Calendar & Slot Selection

● Available ● Full ● Holiday ● My Upcoming Appointment ● Previous Visit ● Visited – Not Completed

Dr. Rahul Verma

09-27 Unavailable	09-28 Available 0/10	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 0/10	10-06 Available 1/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Available 0/10	10-14 Available 0/10	10-15 Available 0/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

← Back to Doctor Profile
SELECT DATE AND TIME

Calendar & Slot Selection

● Available ● Full ● Holiday ● My Upcoming Appointment ● Previous Visit ● Visited – Not Completed

Dr. Sneha Rao

09-27 Unavailable	09-28 Available 0/10	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 0/10	10-06 Available 0/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Available 0/10	10-14 Available 0/10	10-15 Available 0/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

← Back to Doctor Profile
SELECT DATE AND TIME

Calendar & Slot Selection

● Available ● Full ● Holiday ● My Upcoming Appointment ● Previous Visit ● Visited – Not Completed

Dr. Ananya Patel

09-27 Unavailable	09-28 Available 0/10	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 0/10	10-06 Available 0/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Available 0/10	10-14 Available 0/10	10-15 Available 1/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

← Back to Doctor Profile
SELECT DATE AND TIME

Calendar & Slot Selection

● Available ● Full ● Holiday ● My Upcoming Appointment ● Previous Visit ● Visited – Not Completed

Dr. Karan Shah

09-27 Unavailable	09-28 Available 0/10	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 0/10	10-06 Available 0/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Full	10-14 Available 0/10	10-15 Available 1/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

← Back to Doctor Profile
SELECT DATE AND TIME

Calendar & Slot Selection

● Available ● Full ● Holiday ● My Upcoming Appointment ● Previous Visit ● Visited – Not Completed

Dr. Neha Kapoor

09-27 Unavailable	09-28 Available 0/10	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 0/10	10-06 Available 0/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Available 0/10	10-14 Available 0/10	10-15 Available 0/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

← Back to Doctor Profile
SELECT DATE AND TIME

Calendar & Slot Selection

● Available ● Full ● Holiday ● My Upcoming Appointment ● Previous Visit ● Visited – Not Completed

Dr. Arjun Sharma

09-27 Unavailable	09-28 Available 0/10	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 8/10	10-06 Available 0/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Available 0/10	10-14 Available 0/10	10-15 Available 0/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

← Back to Doctor Profile
SELECT DATE AND TIME

Calendar & Slot Selection

● Available ● Full ● Holiday ● My Upcoming Appointment ● Previous Visit ● Visited – Not Completed

Dr. Arjun Sharma

09-27 Unavailable	09-28 Available 0/10	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 8/10	10-06 Available 0/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Available 0/10	10-14 Available 0/10	10-15 Available 0/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

2026-09-28

09:00	Available
09:15	Available
09:30	Available
09:45	Available
10:00	Available
10:15	Available
10:30	Available
10:45	Available
11:00	Available
11:15	Available

Figure A.6: Doctor Availability

A.7 Appointment Booking

Include the screenshot of the appointment booking page.

Book Appointment

What brings you here today?

Heart / Chest Problem

Available Doctors

Dr. Arjun Sharma

MBBS, MD Cardiology | 12 years experience

[View Profile](#)

Dr. Arjun Sharma

Cardiology | MBBS, MD Cardiology

Experienced cardiologist providing cardiovascular care.

Available days: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday

Calendar & Slots

2026-09-27 Unavailable	2026-09-28 My Upcoming Appointment	2026-09-29 0/10 booked
2026-09-30 0/10 booked	2026-10-01 0/10 booked	2026-10-02 0/10 booked
2026-10-03 0/10 booked	2026-10-04 Unavailable	2026-10-05 8/10 booked
2026-10-06 0/10 booked	2026-10-07 0/10 booked	2026-10-08 0/10 booked
2026-10-09 0/10 booked	2026-10-10 0/10 booked	2026-10-11 Unavailable
2026-10-12 0/10 booked	2026-10-13 0/10 booked	2026-10-14 0/10 booked
2026-10-15 0/10 booked	2026-10-16 0/10 booked	2026-10-17 0/10 booked
2026-10-18 Unavailable	2026-10-19 0/10 booked	2026-10-20 0/10 booked
2026-10-21 0/10 booked	2026-10-22 0/10 booked	2026-10-23 0/10 booked
2026-10-24 0/10 booked	2026-10-25 Unavailable	2026-10-26 0/10 booked

Appointment Details

Available time

09:15

Reason for appointment

E2E appointment for update and cancellation test

[Confirm Appointment](#)

Figure A.7: Appointment Booking Page

A.8 Appointment Details

Include the screenshot showing the patient's appointment information.

The screenshot displays the 'Appointment Details' page within the CarePlus application. The top navigation bar includes the CarePlus logo, a dashboard menu, and links to 'Calendar', 'My Appointments', 'Notifications', 'Book Appointment', and a 'Logout' button. The main content area features a breadcrumb trail 'DASHBOARD / BOOK APPOINTMENT' and a large heading 'Appointment Details'. Below this, a form is presented with several input fields: 'Patient ID' (P202600053), 'Patient Name' (CarePlus E2E Patient), 'Doctor' (Dr. Arjun Sharma), 'Specialization' (Cardiology), 'Appointment Date' (2026-09-28), and 'Appointment Time' (09:00). A larger text area for 'Reason for Appointment' is also present. At the bottom of the form is a prominent blue 'Confirm Appointment' button.

Figure A.8: Appointment Details

A.9 Notifications

Include the screenshot showing appointment-related notifications.

Notifications

Appointment Cancelled

Your appointment with Dr. Arjun Sharma on 2026-10-01 has been cancelled.

2026-09-27T22:55:33

Appointment Updated

Your appointment has been updated to 2026-10-01 at 09:45.

2026-09-27T22:55:26

Mark as read

Appointment Updated

Your appointment has been updated to 2026-09-30 at 09:30.

2026-09-27T22:53:21

Mark as read

Appointment Booked

Your appointment with Dr. Arjun Sharma on 2026-09-29 at 09:15 is booked. Please complete payment.

2026-09-27T22:53:00

Mark as read

Payment Successful

Payment of ₹5000 received. Receipt: RCPT-20260927225014-6801. Appointment confirmed.

2026-09-27T22:50:14

Mark as read

Appointment Booked

Your appointment with Dr. Arjun Sharma on 2026-09-28 at 09:00 is booked. Please complete payment.

2026-09-27T22:50:05

Mark as read

Figure A.9: Notification Page

A.10 Doctor Dashboard

Include the screenshot of the doctor dashboard.

[← Back to doctors](#)

DOCTOR PROFILE

Dr. Arjun Sharma**Cardiology**

Experienced cardiologist providing cardiovascular care.

Qualification

MBBS, MD Cardiology

Experience

12 years

Available days

Monday, Tuesday, Wednesday, Thursday, Friday, Saturday

Consultation fee

₹500

Contact

9000000001

[Book Appointment](#)[← Back to doctors](#)

DOCTOR PROFILE

Dr. Priya Mehta**Dermatology**

Experienced dermatologist specializing in skin care.

Qualification

MBBS, MD Dermatology

Experience

9 years

Available days

Monday, Tuesday, Wednesday, Thursday, Friday, Saturday

Consultation fee

₹500

Contact

9000000002

[Book Appointment](#)

[← Back to doctors](#)

DOCTOR PROFILE

Dr. Rahul Verma**Neurology**

Neurologist.

Qualification

MBBS, DM Neurology

Available days

Monday, Tuesday, Wednesday, Thursday, Friday, Saturday

Contact

9000000003

Experience

11 years

Consultation fee

₹750

[Book Appointment](#)[← Back to doctors](#)

DOCTOR PROFILE

Dr. Sneha Rao**Orthopedics**

Orthopedic specialist providing bone and joint care.

Qualification

MBBS, MS Orthopedics

Available days

Monday, Tuesday, Wednesday, Thursday, Friday, Saturday

Contact

9000000004

Experience

10 years

Consultation fee

₹500

[Book Appointment](#)

[← Back to doctors](#)

DOCTOR PROFILE

Dr. Ananya Patel

Ophthalmology

Eye specialist providing vision and eye care.

Qualification

MBBS, MS Ophthalmology

Experience

8 years

Available days

Monday, Tuesday, Wednesday, Thursday, Friday, Saturday

Consultation fee

₹500

Contact

9000000005

[Book Appointment](#)[← Back to doctors](#)

DOCTOR PROFILE

Dr. Karan Shah

Dentistry

Dentist providing comprehensive dental care.

Qualification

BDS, MDS

Experience

7 years

Available days

Monday, Tuesday, Wednesday, Thursday, Friday, Saturday

Consultation fee

₹500

Contact

9000000006

[Book Appointment](#)

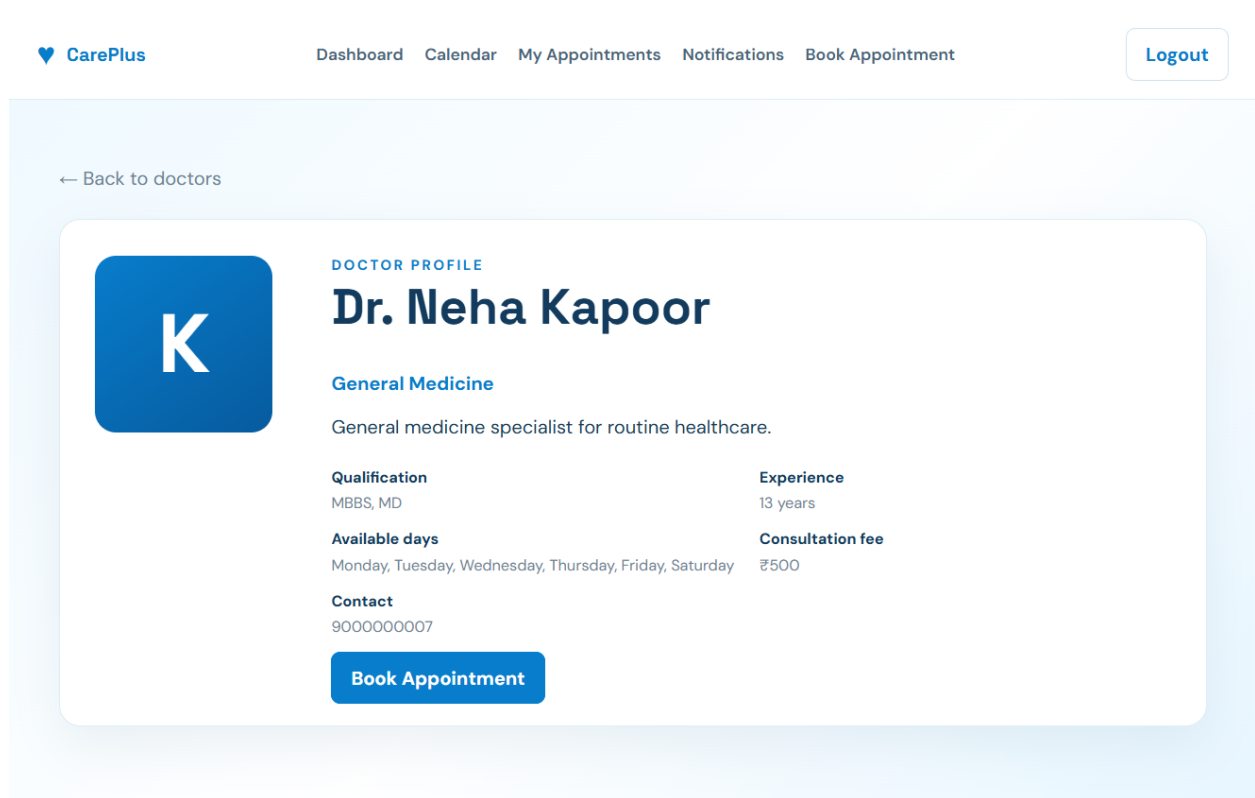


Figure A.10: Doctor Dashboard

A.11 Admin Dashboard

Include the screenshot of the administrator dashboard.

CarePlus Admin

OverviewAppointmentsDoctorsPatientsHolidaysPayments

Logout

ADMIN PORTAL

CarePlus Administration

Logout

Patients53

Doctors7

Appointments53

Today0

Appointments

Patient	Doctor	Date	Time	Status
moni	Dr. Arjun Sharma	2026-09-21	09:00	No Show
	Dr. Priya Mehra	2020-09-21	09:00	Cancelled
QA Test Patient	Dr. Sneha Rao	2026-09-21	09:00	Confirmed
QA Test Patient	Dr. Rahul Verma	2020-09-21	09:00	Cancelled
QA Test Patient	Dr. Arjun Sharma	2020-09-21	09:15	Booked
QA Test Patient	Dr. Karan Shah	2026-09-21	09:30	Confirmed
QA Test Patient	Dr. Ananya Patel	2020-09-21	10:00	Booked
moni	Dr. Ananya Patel	2026-09-22	09:00	Completed
	Dr. Karan Shah	2020-09-22	09:15	Cancelled
moni	Dr. Sneha Rao	2026-09-22	09:15	Confirmed
CarePlus CTO Patient	Dr. Arjun Sharma	2026-09-26	09:00	Confirmed
CarePlus L20 Patient	Dr. Arjun Sharma	2020-10-01	09:45	Cancelled
Slot Filter 0	Dr. Arjun Sharma	2026-10-05	09:00	Booked
Slot Filter 1	Dr. Arjun Sharma	2020-10-05	09:15	Booked
Slot Filter 2	Dr. Arjun Sharma	2026-10-05	09:30	Booked
Slot Filter 3	Dr. Arjun Sharma	2026-10-05	09:45	Booked
Slot Filter 4	Dr. Arjun Sharma	2020-10-05	10:00	Booked
Slot Filter 5	Dr. Arjun Sharma	2026-10-05	10:15	Booked
Slot Filter 6	Dr. Arjun Sharma	2020-10-05	10:30	Booked
Slot Filter 7	Dr. Arjun Sharma	2020-10-05	10:45	Booked
Audit Slot	Dr. Priya Mehra	2026-10-06	09:00	Booked
Audit Slot	Dr. Rahul Verma	2020-10-06	09:15	Booked
Slot Filter 0	Dr. Karan Shah	2026-10-13	09:00	Booked
Slot Filter 1	Dr. Karan Shah	2020-10-13	09:15	Booked
Slot Filter 2	Dr. Karan Shah	2020-10-13	09:30	Booked
Slot Filter 3	Dr. Karan Shah	2026-10-13	09:45	Booked
Slot Filter 4	Dr. Karan Shah	2020-10-13	10:00	Booked
Slot Filter 5	Dr. Karan Shah	2026-10-13	10:15	Booked
Slot Filter 6	Dr. Karan Shah	2020-10-13	10:30	Booked
Slot Filter 7	Dr. Karan Shah	2020-10-13	10:45	Booked
Slot Filter 8	Dr. Karan Shah	2026-10-13	11:00	Booked
Slot Filter 9	Dr. Karan Shah	2020-10-13	11:15	Booked
Register 888372	Dr. Karan Shah	2026-10-15	09:00	Visited - Appointment Not Completed
Register 888372	Dr. Ananya Patel	2020-10-15	09:00	Confirmed
Register 888372	Dr. Karan Shah	2020-10-15	10:00	Cancelled
Audit User 0	Dr. Arjun Sharma	2026-11-09	09:00	Cancelled
Audit User 2	Dr. Priya Mehra	2020-11-09	09:15	Booked
Full Filter 0	Dr. Sneha Rao	2026-11-16	09:00	Booked
Full Filter 1	Dr. Sneha Rao	2020-11-16	09:15	Booked
Full Filter 2	Dr. Sneha Rao	2020-11-16	09:30	Booked
Full Filter 3	Dr. Sneha Rao	2026-11-16	09:45	Booked
Full Filter 4	Dr. Sneha Rao	2020-11-16	10:00	Booked
Full Filter 5	Dr. Sneha Rao	2026-11-16	10:15	Booked
Full Filter 6	Dr. Sneha Rao	2020-11-16	10:30	Booked
Full Filter 7	Dr. Sneha Rao	2020-11-16	10:45	Booked
Full Filter 8	Dr. Sneha Rao	2026-11-16	11:00	Booked
Full Filter 9	Dr. Sneha Rao	2020-11-16	11:15	Booked
Audit Payment Tester	Dr. Ananya Patel	2026-11-23	09:00	Confirmed
Regression Tester	Dr. Arjun Sharma	2026-12-07	09:00	Booked
Regression Tester	Dr. Priya Mehra	2020-12-07	09:00	Booked
Regression Tester	Dr. Arjun Sharma	2026-12-07	09:15	Booked
Regression Tester	Dr. Arjun Sharma	2020-12-07	10:00	Cancelled
Regression Tester	Dr. Ananya Patel	2026-12-14	09:00	Confirmed

Patients

53 registered patients

Doctors

7 doctors configured

Holidays

0 holidays configured

Payments

Receipt	Patient	Amount	Status
RCPT-20200920225014-6801	CarePlus E28 Patient	500	Success
RCPT-2020092024816-426	Register 888372	500	Success
RCPT-20200920234564-9604	Regression Tester	500	Success
RCPT-202009202356-4540	Audit Payment Tester	500	Success
RCPT-2020092020059-4522	QA Test Patient	500	Success
RCPT-2020092025880-6435	moni	500	Success
RCPT-2020092023706-9984	moni	500	Success
RCPT-2020092022034-2363	QA Test Patient	500	Success
RCPT-2020092025804-8155	moni	500	Success
RCPT-2020092025035-0738	moni	500	Success

Figure A.11: Admin Dashboard

Use only the screens that you actually implement. If a particular page is not present in your final application, don't include it.

Appendix B – Sample SQL Queries

This appendix contains the SQL queries used for managing the healthcare appointment database.

B.1 Database Creation

```
CREATE DATABASE healthcare_appointment_db;
```

B.2 Table Creation

Include the actual CREATE TABLE queries from your schema.sql file.

The database should contain the tables that are actually implemented in your project, such as:

- Users
- Patients
- Doctors
- Doctor Availability
- Appointments
- Notifications

B.3 Insert Queries

Include the actual queries used to insert sample or required records.

```
INSERT INTO doctors (...);
```

B.4 Select Queries

Include queries used to retrieve information.

```
SELECT * FROM doctors;
```

```
SELECT * FROM appointments;
```

B.5 Update Queries

Include queries used to update appointment or other system information.

```
UPDATE appointments
```

```
SET status = 'Cancelled'
```

```
WHERE appointment_id = 1;
```

B.6 Join Queries

Include important queries used to retrieve information from related tables.

```
SELECT ...
```

```
FROM appointments
```

JOIN doctors ...

JOIN patients ...;

Important: Replace these examples with your **actual SQL queries** when your database is completed.

Appendix C – API Testing Screenshots

This appendix contains screenshots of the REST API testing performed using Postman.

C.1 FastAPI Swagger Documentation

Screenshot: API Testing/Swagger_API_Documentation.png

FastAPI provides interactive API documentation through Swagger UI. The screenshot shows the available backend endpoints and their supported HTTP methods.

Authorize 

default

GET	/	Root	
GET	/health	Health	
POST	/auth/register	Register Patient	
POST	/auth/login	Login Patient	
GET	/auth/me	Current Patient	
POST	/auth/admin/login	Login Admin	
GET	/doctors/specializations	List Specializations	
GET	/doctors	List Doctors	
GET	/doctors/by-specialization/{spec_id}	Doctors By Spec	
GET	/doctors/{doctor_id}	Doctor Profile	
GET	/doctors/{doctor_id}/availability	Doctor Availability	
GET	/appointments/calendar/{doctor_id}	Doctor Calendar	
GET	/appointments/available-slots/{doctor_id}/{appt_date}	Available Slots	
POST	/appointments/book	Book Appointment	
GET	/appointments/my	My Appointments	
GET	/appointments/{appt_id}	My Appointment Detail	
PUT	/appointments/{appt_id}	Update My Appointment	
DELETE	/appointments/cancel/{appt_id}	Cancel My Appointment	
GET	/notifications/my	My Notifications	
PUT	/notifications/{notification_id}/read	Mark Notification Read	
POST	/payments/pay	Make Payment	
GET	/payments/my	My Payments	
GET	/payments/{payment_id}	Payment Detail	
GET	/payments/appointment/{appointment_id}	Appointment Payment	
GET	/payments/receipt/{receipt_number}	Payment Receipt	
GET	/admin/dashboard	Admin Dashboard	
GET	/admin/appointments	Admin All Appointments	
GET	/admin/patients	Admin All Patients	
GET	/admin/doctors	Admin All Doctors	
POST	/admin/doctors	Admin Create Doctor	
PUT	/admin/doctors/{doctor_id}	Admin Update Doctor	
POST	/admin/doctors/{doctor_id}/availability	Admin Add Availability	
DELETE	/admin/availability/{availability_id}	Admin Delete Availability	
GET	/admin/holidays	Admin All Holidays	
POST	/admin/holidays	Admin Create Holiday	
DELETE	/admin/holidays/{holiday_id}	Admin Delete Holiday	
GET	/admin/payments	Admin All Payments	
PUT	/admin/appointments/{appt_id}/status	Admin Update Status	

Schemas

AdminLogin > Expand all object

AppointmentBook > Expand all object

AppointmentStatusUpdate > Expand all object

AppointmentUpdate > Expand all object

AvailabilitySlot > Expand all object

DoctorCreate > Expand all object

HTTPValidationError > Expand all object

HolidayCreate > Expand all object

PatientLogin > Expand all object

PatientRegister > Expand all object

PaymentRequest > Expand all object

TokenResponse > Expand all object

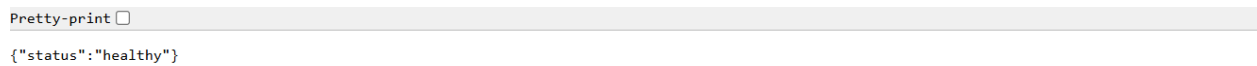
ValidationError > Expand all object

Figure C.1: FastAPI Swagger API Documentation

C.2 API Health Check

Screenshot: API Testing/API_Health_Response.png

The API health-check endpoint was accessed to inspect the backend service response. The screenshot records the response returned by the running application.



```
Pretty-print ☐  
{"status":"healthy"}
```

Figure C.2: API Health Check Response

C.3 Doctor API Response

Screenshot: API Testing/Public_Doctor_API_Response.png

The public doctor API was accessed to inspect the doctor information returned by the backend. This endpoint supports the application's doctor-listing functionality.

Pretty-print ☐

```
[{"doctor_id":5,"full_name":"Dr. Ananya Patel","specialization_id":5,"qualification":"MBBS, MS Ophthalmology","experience_years":8,"phone":"9000000005","email":"ananya.patel@careplus.com","bio":"Eye specialist providing vision and eye care.","is_active":1,"created_at":"2026-09-12T13:44:51","consultation_fee":500.0,"specialization":"Ophthalmology"}, {"doctor_id":1,"full_name":"Dr. Arjun Sharma","specialization_id":1,"qualification":"MBBS, MD Cardiology","experience_years":12,"phone":"9000000001","email":"arjun.sharma@careplus.com","bio":"Experienced cardiologist providing cardiovascular care.","is_active":1,"created_at":"2026-09-12T13:44:51","consultation_fee":500.0,"specialization":"Cardiology"}, {"doctor_id":9,"full_name":"Dr. E2E Legacy QA","specialization_id":6,"qualification":"Legacy QA Qualification","experience_years":3,"phone":"770030981841","email":"careplus.e2e.legacydoctor.1790530981841@example.com","bio":"Temporary legacy doctor for E2E cleanup.","is_active":1,"created_at":"2026-09-27T23:13:01","consultation_fee":500.0,"specialization":"Dentistry"}, {"doctor_id":8,"full_name":"Dr. E2E Temporary QA Updated","specialization_id":6,"qualification":"E2E QA Qualification","experience_years":4,"phone":"880030104677","email":"careplus.e2e.doctor.1790530104677@example.com","bio":"Temporary doctor record for browser acceptance testing.","is_active":1,"created_at":"2026-09-27T22:58:25","consultation_fee":575.0,"specialization":"Dentistry"}, {"doctor_id":6,"full_name":"Dr. Karan Shah","specialization_id":6,"qualification":"BDS, MDS","experience_years":7,"phone":"9000000006","email":"karan.shah@careplus.com","bio":"Dentist providing comprehensive dental care.","is_active":1,"created_at":"2026-09-12T13:44:51","consultation_fee":500.0,"specialization":"Dentistry"}, {"doctor_id":7,"full_name":"Dr. Neha Kapoor","specialization_id":7,"qualification":"MBBS, MD","experience_years":13,"phone":"9000000007","email":"neha.kapoor@careplus.com","bio":"General medicine specialist for routine healthcare.","is_active":1,"created_at":"2026-09-12T13:44:51","consultation_fee":500.0,"specialization":"General Medicine"}, {"doctor_id":2,"full_name":"Dr. Priya Mehta","specialization_id":2,"qualification":"MBBS, MD Dermatology","experience_years":9,"phone":"9000000002","email":"priya.mehta@careplus.com","bio":"Experienced dermatologist specializing in skin care.","is_active":1,"created_at":"2026-09-12T13:44:51","consultation_fee":500.0,"specialization":"Dermatology"}, {"doctor_id":3,"full_name":"Dr. Rahul Verma","specialization_id":3,"qualification":"MBBS, DM Neurology","experience_years":11,"phone":"9000000003","email":"rahul.verma@careplus.com","bio":"Neurologist.","is_active":1,"created_at":"2026-09-12T13:44:51","consultation_fee":750.0,"specialization":"Neurology"}, {"doctor_id":4,"full_name":"Dr. Sneha Rao","specialization_id":4,"qualification":"MBBS, MS Orthopedics","experience_years":10,"phone":"9000000004","email":"sneha.rao@careplus.com","bio":"Orthopedic specialist providing bone and joint care.","is_active":1,"created_at":"2026-09-12T13:44:51","consultation_fee":500.0,"specialization":"Orthopedics"}]
```

Figure C.3: Public Doctor API Response

C.4 Protected Admin API — Unauthorized Access

Screenshot: API Testing/Protected_Admin_API_Unauthorized.png

A protected administrator API endpoint was accessed without the required authorization. The screenshot records the backend's response to the unauthorized request, providing evidence of access-control testing.

Pretty-print ☐

```
{"detail": "Authentication required"}
```

Figure C.4: Unauthorized Access Response from Protected Admin API

C.5 OpenAPI Schema

Screenshot: API Testing/OpenAPI_Raw_Schema.png

FastAPI generates an OpenAPI schema describing the application's documented endpoints, request structures and response definitions. The screenshot shows the schema returned by the backend.

[illegible]

Figure C.5: FastAPI OpenAPI Schema

Appendix D – Installation and Setup Instructions

This appendix explains how to install and run the **Healthcare Appointment System**.

D.1 Required Software

The following software is required:

- Python
- MySQL
- Visual Studio Code
- Postman
- Git
- Web Browser

D.2 Database Setup

1. Install and start MySQL.
2. Create the healthcare appointment database.
3. Execute the schema.sql file.
4. Execute the sample_data.sql file if sample data is required.
5. Verify that the required tables have been created.

D.3 Backend Setup

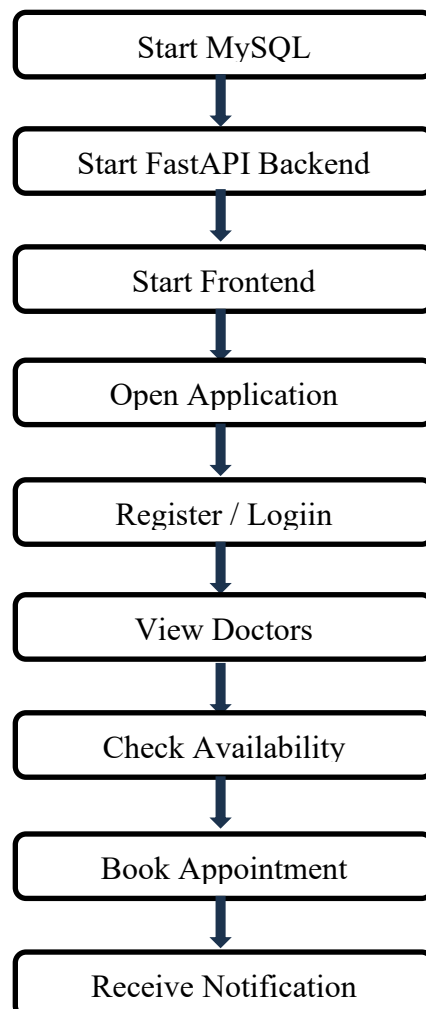
1. Open the project in Visual Studio Code.
2. Open the backend directory.
3. Create a Python virtual environment if required.
4. Activate the virtual environment.
5. Install the packages listed in requirements.txt.
6. Configure the database connection.
7. Configure the .env file.
8. Start the FastAPI backend.

D.4 Frontend Setup

1. Open the frontend directory.
2. Verify that the frontend files are connected to the correct backend API.
3. Start the frontend using the method selected for the project.
4. Open the application in a web browser.

D.5 Application Execution

The basic execution flow is:



Appendix E – User Manual / Guide

This appendix provides basic instructions for using the healthcare appointment scheduling system.

E.1 Patient Guide

Step 1: Registration

The patient creates an account by entering the required registration details.

Create Your CarePlus Account

Full Name

Date of Birth

mm/dd/yyyy



Age

Phone

Email

Address

Emergency Contact Name

Emergency Contact Number

Password

Confirm Password

Gender

Create Patient Account

Step 2: Login

The patient logs into the system using the registered credentials.

CAREPLUS PATIENT PORTAL

Welcome Back

Login to access your account

Patient ID or Email

P202600001 or patient@example.com

Password

Enter your password

☐ Remember me

[Forgot Password?](#)

Login

New here? [Create an account](#)

Step 3: View Doctors

The patient can view available doctors and their relevant information.

DASHBOARD / DOCTORS

Find a Doctor

Filter by specialization

Cardiology

Dr. Arjun Sharma

Cardiology

MBBS, MD Cardiology

12 years experience

Available: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday | 09:00 – 11:30

Consultation fee: ₹500

View Profile

Step 4: Check Availability

The patient checks the doctor's available appointment dates and time slots.

 [Dashboard](#) [Calendar](#) [My Appointments](#) [Notifications](#) [Book Appointment](#) [Logout](#)

[← Back to Doctor Profile](#)
[SELECT DATE AND TIME](#)

Calendar & Slot Selection

Available Full Holiday My Upcoming Appointment Previous Visit Visited – Not Completed

Dr. Arjun Sharma

09-27 Unavailable	09-28 Available 0/10	09-29 Available 0/10	09-30 Available 0/10	10-01 Available 0/10
10-02 Available 0/10	10-03 Available 0/10	10-04 Unavailable	10-05 Available 8/10	10-06 Available 0/10
10-07 Available 0/10	10-08 Available 0/10	10-09 Available 0/10	10-10 Available 0/10	10-11 Unavailable
10-12 Available 0/10	10-13 Available 0/10	10-14 Available 0/10	10-15 Available 0/10	10-16 Available 0/10
10-17 Available 0/10	10-18 Unavailable	10-19 Available 0/10	10-20 Available 0/10	10-21 Available 0/10
10-22 Available 0/10	10-23 Available 0/10	10-24 Available 0/10	10-25 Unavailable	10-26 Available 0/10

Available Time Slots

Step 5: Book Appointment

The patient selects a suitable available slot and books an appointment.

Book Appointment

What brings you here today?

Heart / Chest Problem

Available Doctors

Dr. Arjun Sharma

MBBS, MD Cardiology | 12 years experience

[View Profile](#)

Dr. Arjun Sharma

Cardiology | MBBS, MD Cardiology

Experienced cardiologist providing cardiovascular care.

Available days: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday

Calendar & Slots

2026-09-27 Unavailable	2026-09-28 My Upcoming Appointment	2026-09-29 0/10 booked
2026-09-30 0/10 booked	2026-10-01 0/10 booked	2026-10-02 0/10 booked
2026-10-03 0/10 booked	2026-10-04 Unavailable	2026-10-05 8/10 booked
2026-10-06 0/10 booked	2026-10-07 0/10 booked	2026-10-08 0/10 booked
2026-10-09 0/10 booked	2026-10-10 0/10 booked	2026-10-11 Unavailable
2026-10-12 0/10 booked	2026-10-13 0/10 booked	2026-10-14 0/10 booked
2026-10-15 0/10 booked	2026-10-16 0/10 booked	2026-10-17 0/10 booked
2026-10-18 Unavailable	2026-10-19 0/10 booked	2026-10-20 0/10 booked
2026-10-21 0/10 booked	2026-10-22 0/10 booked	2026-10-23 0/10 booked
2026-10-24 0/10 booked	2026-10-25 Unavailable	2026-10-26 0/10 booked

Appointment Details

Available time

09:15

Reason for appointment

E2E appointment for update and cancellation test

[Confirm Appointment](#)

Step 6: View Appointment

The patient can view the details and status of the booked appointment.

♥ CarePlus Dashboard Calendar My Appointments Notifications Book Appointment Logout

My Appointments

Dr. Arjun Sharma
Cardiology
2026-09-29 at 09:15 | Slot 2
Booked
Cancel Reschedule Pay

Dr. Arjun Sharma
Cardiology
2026-09-28 at 09:00 | Slot 1
Confirmed Payment: Success
Cancel

Step 7: Manage Appointment

The patient can update or cancel an appointment according to the functionality provided by the system.

My Appointments

Dr. Arjun Sharma

Cardiology

2026-10-01 at 09:45 | Slot 4

Booked

Cancel

Reschedule

Pay

Dr. Arjun Sharma

Cardiology

2026-09-28 at 09:00 | Slot 1

Confirmed

Payment: Success

Cancel

My Appointments

Dr. Arjun Sharma

Cardiology

2026-10-01 at 09:45 | Slot 4

Cancelled

Dr. Arjun Sharma

Cardiology

2026-09-28 at 09:00 | Slot 1

Confirmed

Payment: Success

Cancel

Step 8: Check Notifications

The patient can view notifications related to appointments.

Notifications

Appointment Cancelled

Your appointment with Dr. Arjun Sharma on 2026-10-01 has been cancelled.

2026-09-27T22:55:33

Mark as read

Appointment Updated

Your appointment has been updated to 2026-10-01 at 09:45.

2026-09-27T22:55:26

Mark as read

Appointment Updated

Your appointment has been updated to 2026-09-30 at 09:30.

2026-09-27T22:53:21

Mark as read

Appointment Booked

Your appointment with Dr. Arjun Sharma on 2026-09-29 at 09:15 is booked. Please complete payment.

2026-09-27T22:53:00

Mark as read

Payment Successful

Payment of ₹5000.0 received. Receipt: RCPT-20260927225014-6801. Appointment confirmed.

2026-09-27T22:50:14

Mark as read

Appointment Booked

Your appointment with Dr. Arjun Sharma on 2026-09-28 at 09:00 is booked. Please complete payment.

2026-09-27T22:50:05

Mark as read

E.2 Doctor Guide

Step 1: Login

The doctor logs into the system using the appropriate credentials.

Step 2: Manage Availability

The doctor can manage available dates and appointment time slots.

Step 3: View Appointments

The doctor can view appointments associated with their schedule.

Step 4: Manage Appointment Information

The doctor can perform the appointment-related operations provided by the system.

E.3 Admin Guide

Step 1: Admin Login

The administrator logs into the administrative section.

CarePlus Admin

OverviewAppointmentsDoctorsPatientsHolidaysPayments

Logout

ADMIN PORTAL

CarePlus Administration

Logout

Patients53

Doctors7

Appointments53

Today0

Appointments

Patient	Doctor	Date	Time	Status
moni	Dr. Arjun Sharma	2026-09-21	09:00	No Show
	Dr. Priya Mehra	2026-09-21	09:00	Cancelled
QA Test Patient	Dr. Sneha Rao	2026-09-21	09:00	Confirmed
QA Test Patient	Dr. Rahul Verma	2026-09-21	09:00	Cancelled
QA Test Patient	Dr. Arjun Sharma	2026-09-21	09:15	Booked
QA Test Patient	Dr. Karan Shah	2026-09-21	09:30	Confirmed
QA Test Patient	Dr. Ananya Patel	2026-09-21	10:00	Booked
moni	Dr. Ananya Patel	2026-09-22	09:00	Completed
	Dr. Karan Shah	2026-09-22	09:15	Cancelled
moni	Dr. Sneha Rao	2026-09-22	09:15	Confirmed
CarePlus CTO Patient	Dr. Arjun Sharma	2026-09-26	09:00	Confirmed
CarePlus L26 Patient	Dr. Arjun Sharma	2026-10-01	09:45	Cancelled
Slot Filter 0	Dr. Arjun Sharma	2026-10-05	09:00	Booked
Slot Filter 1	Dr. Arjun Sharma	2026-10-05	09:15	Booked
Slot Filter 2	Dr. Arjun Sharma	2026-10-05	09:30	Booked
Slot Filter 3	Dr. Arjun Sharma	2026-10-05	09:45	Booked
Slot Filter 4	Dr. Arjun Sharma	2026-10-05	10:00	Booked
Slot Filter 5	Dr. Arjun Sharma	2026-10-05	10:15	Booked
Slot Filter 6	Dr. Arjun Sharma	2026-10-05	10:30	Booked
Slot Filter 7	Dr. Arjun Sharma	2026-10-05	10:45	Booked
Audit Slot	Dr. Priya Mehra	2026-10-06	09:00	Booked
Audit Slot	Dr. Rahul Verma	2026-10-06	09:15	Booked
Slot Filter 0	Dr. Karan Shah	2026-10-13	09:00	Booked
Slot Filter 1	Dr. Karan Shah	2026-10-13	09:15	Booked
Slot Filter 2	Dr. Karan Shah	2026-10-13	09:30	Booked
Slot Filter 3	Dr. Karan Shah	2026-10-13	09:45	Booked
Slot Filter 4	Dr. Karan Shah	2026-10-13	10:00	Booked
Slot Filter 5	Dr. Karan Shah	2026-10-13	10:15	Booked
Slot Filter 6	Dr. Karan Shah	2026-10-13	10:30	Booked
Slot Filter 7	Dr. Karan Shah	2026-10-13	10:45	Booked
Slot Filter 8	Dr. Karan Shah	2026-10-13	11:00	Booked
Slot Filter 9	Dr. Karan Shah	2026-10-13	11:15	Booked
Register 888372	Dr. Karan Shah	2026-10-15	09:00	Visited - Appointment Not Completed
Register 888372	Dr. Ananya Patel	2026-10-15	09:00	Confirmed
Register 888372	Dr. Karan Shah	2026-10-15	10:00	Cancelled
Audit User 0	Dr. Arjun Sharma	2026-11-09	09:00	Cancelled
Audit User 2	Dr. Priya Mehra	2026-11-09	09:15	Booked
Full Filter 0	Dr. Sneha Rao	2026-11-16	09:00	Booked
Full Filter 1	Dr. Sneha Rao	2026-11-16	09:15	Booked
Full Filter 2	Dr. Sneha Rao	2026-11-16	09:30	Booked
Full Filter 3	Dr. Sneha Rao	2026-11-16	09:45	Booked
Full Filter 4	Dr. Sneha Rao	2026-11-16	10:00	Booked
Full Filter 5	Dr. Sneha Rao	2026-11-16	10:15	Booked
Full Filter 6	Dr. Sneha Rao	2026-11-16	10:30	Booked
Full Filter 7	Dr. Sneha Rao	2026-11-16	10:45	Booked
Full Filter 8	Dr. Sneha Rao	2026-11-16	11:00	Booked
Full Filter 9	Dr. Sneha Rao	2026-11-16	11:15	Booked
Audit Payment Tester	Dr. Ananya Patel	2026-11-23	09:00	Confirmed
Regression Tester	Dr. Arjun Sharma	2026-12-07	09:00	Booked
Regression Tester	Dr. Priya Mehra	2026-12-07	09:00	Booked
Regression Tester	Dr. Arjun Sharma	2026-12-07	09:15	Booked
Regression Tester	Dr. Arjun Sharma	2026-12-07	10:00	Cancelled
Regression Tester	Dr. Ananya Patel	2026-12-14	09:00	Confirmed

Patients

53 registered patients

Doctors

7 doctors configured

Holidays

0 holidays configured

Payments

Receipt	Patient	Amount	Status
RCPT-20260920225014-6801	CarePlus E26 Patient	500	Success
RCPT-2026092024816-426	Register 888372	500	Success
RCPT-2026092023456-9604	Regression Tester	500	Success
RCPT-202609202356-4545	Audit Payment Tester	500	Success
RCPT-2026092020059-4522	QA Test Patient	500	Success
RCPT-2026092023480-6435	moni	500	Success
RCPT-2026092023706-9984	moni	500	Success
RCPT-2026092022034-2363	QA Test Patient	500	Success
RCPT-2026092025804-8155	moni	500	Success
RCPT-2026092025035-0738	moni	500	Success

Step 2: Manage Users

The administrator can manage the users supported by the system.

Manage Patients

Patient ID	Name	Email	Phone	Age	Gender
P202600053	CarePlus EZE Patient	careplus.eze.1790529490572@example.com	900029490572	26	Other
P202600052	RegUser 888572	reguser_888572@example.com	98885727	32	Female
P202600051	Regression Tester	regtest_888490@example.com	960888490	32	Female
P202600050	Regression Tester	regtest_888310@example.com	960888310	32	Female
P202600049	Audit Payment Tester	auditpay_886635@example.com	951886635	34	Male
P202600048	Eleventh Patient	eleventh_886562@example.com	9430886562	30	Other
P202600047	FullFiler 9	fullfiller_886562_9@example.com	9428865629	30	Other
P202600046	FullFiler 8	fullfiller_886562_8@example.com	9428865628	30	Other
P202600045	FullFiler 7	fullfiller_886562_7@example.com	9428865627	30	Other
P202600044	FullFiler 6	fullfiller_886562_6@example.com	9428865626	30	Other
P202600043	FullFiler 5	fullfiller_886562_5@example.com	9428865625	30	Other
P202600042	FullFiler 4	fullfiller_886562_4@example.com	9428865624	30	Other
P202600041	FullFiler 3	fullfiller_886562_3@example.com	9428865623	30	Other
P202600040	FullFiler 2	fullfiller_886562_2@example.com	9428865622	30	Other
P202600039	FullFiler 1	fullfiller_886562_1@example.com	9428865621	30	Other
P202600038	FullFiler 0	fullfiller_886562_0@example.com	9428865620	30	Other
P202600037	Audit User2	audituser2b_886562@example.com	941886562	34	Male
P202600036	Audit User	audituser_886503@example.com	940886503	33	Female
P202600035	Overbook Test	overbook_85662@example.com	98566299	30	Other
P202600034	Slot Filler2 9	slotfiller2_85662_9@example.com	98566209	30	Other
P202600033	Slot Filler2 8	slotfiller2_85662_8@example.com	98566208	30	Other
P202600032	Slot Filler2 7	slotfiller2_85662_7@example.com	98566207	30	Other
P202600031	Slot Filler2 6	slotfiller2_85662_6@example.com	98566206	30	Other
P202600030	Slot Filler2 5	slotfiller2_85662_5@example.com	98566205	30	Other
P202600029	Slot Filler2 4	slotfiller2_85662_4@example.com	98566204	30	Other
P202600028	Slot Filler2 3	slotfiller2_85662_3@example.com	98566203	30	Other
P202600027	Slot Filler2 2	slotfiller2_85662_2@example.com	98566202	30	Other
P202600026	Slot Filler2 1	slotfiller2_85662_1@example.com	98566201	30	Other
P202600025	Slot Filler2 0	slotfiller2_85662_0@example.com	98566200	30	Other
P202600024	Slot Filler2 0	slotfiller2_0@example.com	9300000000	30	Other
P202600023	Slot Filler 10	slotfiller10@example.com	9200000010	35	Other
P202600022	Slot Filler 9	slotfiller9@example.com	9200000009	35	Other
P202600021	Slot Filler 8	slotfiller8@example.com	9200000008	35	Other
P202600020	Slot Filler 7	slotfiller7@example.com	9200000007	35	Other
P202600019	Slot Filler 6	slotfiller6@example.com	9200000006	35	Other
P202600018	Slot Filler 5	slotfiller5@example.com	9200000005	35	Other
P202600017	Slot Filler 4	slotfiller4@example.com	9200000004	35	Other
P202600016	Slot Filler 3	slotfiller3@example.com	9200000003	35	Other
P202600015	Slot Filler 2	slotfiller2@example.com	9200000002	35	Other
P202600014	Slot Filler 1	slotfiller1@example.com	9200000001	35	Other
P202600013	Slot Filler 0	slotfiller0@example.com	9200000000	35	Other
P202600012	Audit Bot	audit.bot@example.com	9199999999	35	Other
P202600011	QA Test Patient	qatestpatient001@example.com	9123456780	30	Male
P202600010	moni	moni@gmail.com	8989895678	19	Female
P202600009	Connectivity Test Patient	connectivity-1789881244073@example.com	911244073	31	Other
P202600008	Smoke Patient	smoke1789880810@example.com	9009880810	31	Other
P202600007	UI Test Patient	ui.test@example.com	9876500101	32	Female
P202600006	QA Patient	qa.patient@example.com	9876500001	30	Female
P202600005	Monisha Raghini	monishachelle@gmail.com	8125357979	19	Female
P202600004	B Devi charitha	devicharithab@gmail.com	1234567891	18	Female
P202600003	Sony	Sony@gmail.com	9980807575	19	Female
P202600002	Monisha Raghini Chelle	monishachelle@gmail.com	9876543219	19	Female
P202600001	Monisha Raghini Chelle	monisharaghinichelle@gmail.com	9876543210	19	-

Step 3: Manage Doctors

The administrator can manage doctor-related information.

ADMINISTRATION

Manage Doctors

[+ Add Doctor](#)

ID	Doctor	Specialization	Fee	Contact	Status	Actions
5	Dr. Ananya Patel	Ophthalmology	₹500	9000000005	Active	Edit Availability
1	Dr. Arjun Sharma	Cardiology	₹500	9000000001	Active	Edit Availability
8	Dr. E2E Temporary QA	Dentistry	₹550	880030104677	Active	Edit Availability
6	Dr. Karan Shah	Dentistry	₹500	9000000006	Active	Edit Availability
7	Dr. Neha Kapoor	General Medicine	₹500	9000000007	Active	Edit Availability
2	Dr. Priya Mehta	Dermatology	₹500	9000000002	Active	Edit Availability
3	Dr. Rahul Verma	Neurology	₹750	9000000003	Active	Edit Availability
4	Dr. Sneha Rao	Orthopedics	₹500	9000000004	Active	Edit Availability

Add Doctor

Name

Specialization

Choose



Qualification

Experience years

Phone

Email

Consultation fee

Bio

[Create Doctor](#)

Step 4: Manage Appointments

The administrator can view and manage appointment information.

Appendix F – Geo-Tag Photos with Guide

This section contains the **actual geo-tagged photographs** required by the project guide or department.

Include the photographs provided or requested by your guide.

For each photograph, add a caption such as:

Figure F.1: Project Development/Review Session

Figure F.2: Project Review with Guide

Review -1



Geo Tag photos with guide

